



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校

浙江工商大学

参赛队号

24103530099

队员姓名

1.

余乐

2.

郑浩东

3.

王灏

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生 数学建模竞赛

题 目： 基于特征对齐与多重预测驱动的 WLAN 信号预测方案

摘 要：

无线局域网基于 IEEE802.11 标准不断发展演变，然而在高密度部署场景下信号干扰、碰撞等问题显著导致实际部署带宽和数据传输速率大幅下降，针对此类现象，本文综合运用**自适应器、随机森林、GBDT、虚拟变量、DBSCAN 算法、CNN 系列**等理论和方法，建立分频预测模型，预测 AP 发送数据帧序列的总时长、MCS,NSS 以及网络吞吐量等信息，从而对 WLAN 进行优化。

针对问题一，本题要求分析各参数对 AP 发送机会的影响，给出影响性强弱的顺序，对 AP 发送机会进行预测。本文首先通过**目标编码与虚拟变量**的方式将特征转换为数值形式，采用 **DBSCAN 聚类算法**将同步传输和异步传输按照高低密度不同的聚类区域进行区分，同时引入一个**扰动机制以控制随机性**，接着分别构建基于**随机森林、GBDT** 的预测模型，比较二者在训练集上性能指标 **MSE** 和**拟合优度**，进而确定模型的应用，最后依据模型输出的特征重要性评分，按影响力大小对 seq_time 的因素进行排序，并使用拟合好的随机森林模型对测试集上 AP 发送数据帧序列总时长进行了预测。

针对问题二，本题要求对测试中 AP 发送数据选用最多次数 MCS（调制编码方案）,NSS（空间流数），并通过测试集预测 MCS,NSS。本文首先将标签 NSS 和 MCS 进行融合，形成一个新的复合标签，减少模型训练时的复杂性，同时考虑到**编码数字大小**会对模型的输入产生影响，继而采用**独热编码**的方式消除该影响，确保每个类别特征都以**独立且平等**的方式呈现给模型。接着构建新特征 SINR（信号与干扰加噪声比），采用截断操作使训练集中两个数据集长度保持一致，并用**过采样**的方法丰富样本量，根据同步传输和异步传输类型的不同分别采取不同的方法来计算，并对两个类型的 SINR 分别取平均值得到最终标签，最后分别输入到 **BP 神经网络**和**卷积神经网络（CNN）**中加以训练，选用预测效果更好的模型加以预测。

针对问题三，本文首先结合题目中所给出的相关信息，吞吐量的计算中仅 MSDU 的总字节数属于有效传输数据，但是 **AMSDU 和 AMPDU 聚合的准确评估**对吞吐量预测也有影响，由此可以明显判断 AMSDU 和 AMPDU 聚合的准确评估会影响 num_ppdu 和 ppdu_dur (s)，通过观察数据集中 pkt_len 代表的列的意思为数据报文的长度数值相同，即说明

AMSDU 和 AMPDU 聚合的准确性会直接影响到数据帧的传输时间和时长。所以本题中，我们团队认为 throughput 受到的影响来自数据帧统计信息，而这些 num_ppdu 数据等是作为测试过程中的观测值，测试集中并没有标出，所以团队认为该类数据近似受到网络拓扑、业务流量、门限、节点间 RSSI 等信息的影响，我们将建立一个**双层交叉预测模型**，单层中我们使用经过第一问检验的**随机森林模型**进行数据帧统计信息的预测，通过验证之后，我们采用**深度残差神经网络**进行数据帧统计信息和部分门限等信息来进行系统吞吐量的判断。

关键词： 双层交叉预测模型，特征对齐，DBSCAN 算法，随机森林，卷积神经网络

目 录

一、前言	5
1.1 问题背景	5
1.2 问题研究现状	5
1.3 问题重述	6
1.4 技术路线图	8
2.1 模型假设	9
2.2 符号说明	9
三、数据预处理	10
3.1 缺失值处理	10
3.2 异常值处理	11
3.3 数据标准化	11
四、问题一：基于控制随机性的随机森林模型进行 seq_time 预测与特征重要性排序.....	13
4.1 问题一分析	13
4.2 相关理论	14
4.2.1 特征工程	14
4.2.2 随机森林	15
4.2.3GBDT	16
4.2.4 虚拟变量	17
4.3 随机传输特征处理：DBSCAN 密度聚类	18
4.3.1 数据处理	18
4.3.2 数据分布与算法确定	19
4.3.3 DBSCAN 算法求解	21
4.4 2ap 情况下模型建立与求解.....	21
4.4.1 特征选择与处理	21
4.4.2 模型建立	22
4.4.3 指标相关性分析——热力图	23
4.4.4 模型分析	25
4.4.5 预测结果分析	26
4.5 3ap 情况下模型建立与求解.....	28
4.5.1 特征选择与处理	28
4.5.2 指标相关性分析——热力图	28
4.5.3 模型建立与重要性排序	29
4.5.4 预测结果分析	30
五、问题二：基于过采样方法与卷积神经网络的（MCS, NSS）预测模型	32
5.1 问题二分析	32
5.2 相关理论	33
5.2.2 BP 神经网络.....	33
5.2.2 CNN 系列	33
5.2.3 过采样	34
5.3 模型建立与分析	35
5.3 2ap 情况下模型建立与求解.....	36

5.3.1 特征选择与处理	36
5.3.2 建立 BP 神经网络以及 CNN 卷积神经网络模型	36
5.3.3 在 CNN 卷积神经网络中融入过采样方法	38
5.4 3ap 情况下模型建立与求解	40
六、问题三：基于双层交叉预测模型的系统吞吐量分析预测	42
6.1 问题三分析	42
6.2 深度残差神经网络相关理论	43
6.3 2ap 情况下模型建立与求解	44
6.3.1 特征选取与处理	44
6.3.2 第一阶段——预测中间变量	44
6.3.3 第二阶段——预测网络吞吐量	45
6.4 3ap 情况下模型建立与求解	47
七、模型评价与改进	51
7.1 模型的优点	51
7.2 模型的缺点	51
参考文献	52
附录	53
附录 A：各小题预测结果	53
附录 B：代码	60

一、前言

1.1 问题背景

无线局域网（WLAN）是一种利用无线通信技术构建的计算机网络，允许设备在一定范围内进行无线连接和通信，其技术背景源于人们对移动性和网络灵活性的需求。基于 IEEE 802.11 系列标准，WLAN 不断演进，从最初的 802.11b、802.11g 到 Wi-Fi 4（802.11n）、Wi-Fi 5（802.11ac），再到最新的 Wi-Fi 6（802.11ax）和即将到来的 Wi-Fi 7（802.11be），提供了更高的带宽、更低的延迟和更好的用户体验。这些标准的升级满足了从语音、邮件到网页、视频以及虚拟现实/增强现实等日益复杂的业务需求。

然而，随着 WLAN 在家庭、办公、教育、医疗、工业制造和仓储等场景的广泛部署，高密度环境下的节点密集度增加，导致相邻小区覆盖范围重叠，引发干扰和碰撞问题，实际带宽和数据传输速率大幅下降，系统吞吐量受到限制，需要进一步优化。安全性方面，WLAN 采用了从 WEP、WPA 到 WPA2 和 WPA3 的加密协议，提升了数据传输的安全性，但仍需警惕未经授权的接入、中间人攻击等安全威胁。

提升无线局域网（WLAN）性能的关键在于对网络吞吐量进行精确预测。吞吐量代表了节点在单位时间内成功传输的数据量，是衡量网络效率的重要指标，快速而准确地预测吞吐量，有助于设计出更加稳健、高效的 WLAN 系统。近年来，研究人员尝试利用机器学习技术，通过提取诸如信道条件、带宽配置、流量特征、发射功率、接入控制机制等基本信息，以及节点之间的接收信号强度（RSSI）、信号干扰噪声比（SINR）、传输时延等架构信息，来构建模型预测网络吞吐量。然而，在实际应用中，WLAN 的通信行为受到信道环境和干扰的快速变化，以及复杂多样的业务流量影响，导致仿真模型与真实情况存在差异。因此，单纯依赖仿真数据训练的模型难以满足商业应用的需求，需要开发能够适应实时变化的预测方法，以进一步优化 WLAN 系统的性能。

因此，为了满足用户对高速无线通信的需求，需要在结合实际部署情况的基础上，对 WLAN 系统进行进一步的优化，包括改进技术标准、加强安全措施和优化网络架构，以提升系统的吞吐量和整体性能。

1.2 问题研究现状

有关 WLAN 吞吐量影响因素的探索，国内外早已有不少学者展开了研究。在国外学者中，Nazabal 等人^[1]认为室内场景中元件的布局和组成在无线传感器网络和系统中使用的传播信道损耗方面起着关键作用，为此他们开发了一种基于 3D 射线发射的内部确定性代码，以分析室内拓扑和形态在两种不同的现实场景中对无线链路运行的影响；Radhakrishna K S 等人^[2]通过分析 WLAN 在 5240 MHz 和移动网络在 2100 MHz 下的接收功率（dBm）和自由空间路径损耗（FSPL），认为随着距离的增加，FSPL 的值增加，这会导致接收的功率减少，障碍物越密集，信号衰减越多；Qureshi I A 等人^[3]对标准 IEEE802.11 论文

和同行评审期刊提供系统评价，阐述哪些措施迫使在 WLAN 领域引入这些改进，认为最近的研究并不关注 PHY 层的数据速率改进，而是集中在 MAC 层，试图最大限度地减少开销以提高整体性能。

在国内学者中，宋向东^[4]认为 Wi-Fi CERTIFIED ac Wave 2 是基于 802.11ac 标准的 Wi-Fi 规范，是 Wave 1 的升级版，与 Wave 1 相比有很大改进，提高了性能、添加了方便性；徐勇军等人^[5]研究了智能超表面辅助的认知反向散射通信网络吞吐量最大化问题，提出了一种基于迭代的吞吐量最大化多维资源分配算法；李爽^[6]尝试将参数估计与假设检验引入到软件性能指标的描述和处理中，以网络吞吐量测试为例，尝试对随机变量的软件性能指标从统计学的角度作探讨；曹莹等人^[7]建立了反向神经网络（BP）和差分整合移动平均自回归模型（ARIMA）的组合预测模型来提高港口吞吐量预测精准性；何剑等人^[8]为了提高 IEEE 802.11e 增强分布式信道接入(EDCA)机制在高密场景下的全局吞吐量，研究了基于多关联模型的软件定义 WLAN 系统和无线接入点(AP)关联算法明显提高 EDCA 机制下的全局带宽；陈禹等人^[9]基于一定程度的强化学习策略，针对 IEEE802.11 无线网络中的速率自适应技术提出多臂老虎机问题的速率自适应算法，提高了 WLAN 无线网络节点的吞吐量。

1.3 问题重述

基于上述问题背景和研究现状，该赛题需要建立并运用数学模型，研究 WLAN 网络拓扑结构、节点间的接收信号强度指示（RSSI）、信道接入方式、干扰等因素如何影响 WLAN 的数据传输和速度。具体地，该赛题包括以下 3 个小题：

问题一：根据附件 WLAN 网络实测训练集中所提供的各参数基本信息，分析各参数对 AP 发送机会的影响，给出影响性强弱的顺序，对 AP 发送机会进行预测。

包括三方面内容：

（1）根据提供的网络拓扑、业务流量、门限、节点间 RSSI 等信息给出影响性强弱顺序；

（2）对 AP 发送数据帧序列的总时长进行预测；

具体要求如下：

（1）根据 WLAN 网络实测训练集中的数据，包括网络拓扑、业务流量、门限（如包检测门限 PD、能量检测门限 ED、NAV 门限）、节点间 RSSI 等基本信息，详细分析这些参数如何影响 AP（接入点）的发送机会。需要给出这些参数对 AP 发送机会的影响强度排序；

（2）基于上述分析，构建一个模型来预测每个 AP 的发送机会，具体表现为 AP 发送数据帧序列的总时长（seq_time）；

（3）利用已建立的模型，通过提供的测试集 test_set_1_2ap 和 test_set_1_3ap 来预测在不同数量同频 AP 情况下的 AP 发送数据帧序列的总时长。

问题二：根据训练集中测试的基本信息，对 MCS,NSS 进行建模，并通过测试集预测 MCS,NSS。

包括两方面内容：

（1）根据节点间 RSSI 信息和门限信息和 AP 发送机会的分析，对测试中 AP 发送数据选用最多次数的 MCS,NSS 进行建模；

（2）通过测试集预测 MCS,NSS。

具体要求如下：

（1）基于实测训练集中的测试基本信息，特别是节点间 RSSI 信息和门限信息，并结合问题一中对 AP 发送机会的分析，构建一个模型来预测在实际传输过程中 AP 最常选用的调制编码方案（MCS）与空间流数（NSS）组合；

（2）使用建立的模型通过提供的测试集 test_set_2_2ap 和 test_set_2_3ap 预测 AP 发送数据时最常采用的(MCS, NSS)组合。

问题三：对系统吞吐量进行建模并预测网络吞吐量。

包括两方面内容：

（1）结合问题 1 和 2 的分析对系统吞吐量进行建模；

（2）通过测试集预测网络吞吐量。

具体要求如下：

（1）结合问题一和问题二的分析，对 WLAN 系统的吞吐量进行建模。这包括综合考虑 AP 发送机会（即数据帧序列的总时长）以及 AP 发送数据时所选用的(MCS, NSS)组合等因素；

（2）利用已构建的模型，通过测试集 test_set_1_2ap 和 test_set_1_3ap 来预测网络吞吐量。可以按照同频 AP 的数量（如 2 个或 3 个 AP）分类建模，也可以尝试建立一个统一的模型来处理所有情况。

1.4 技术路线图

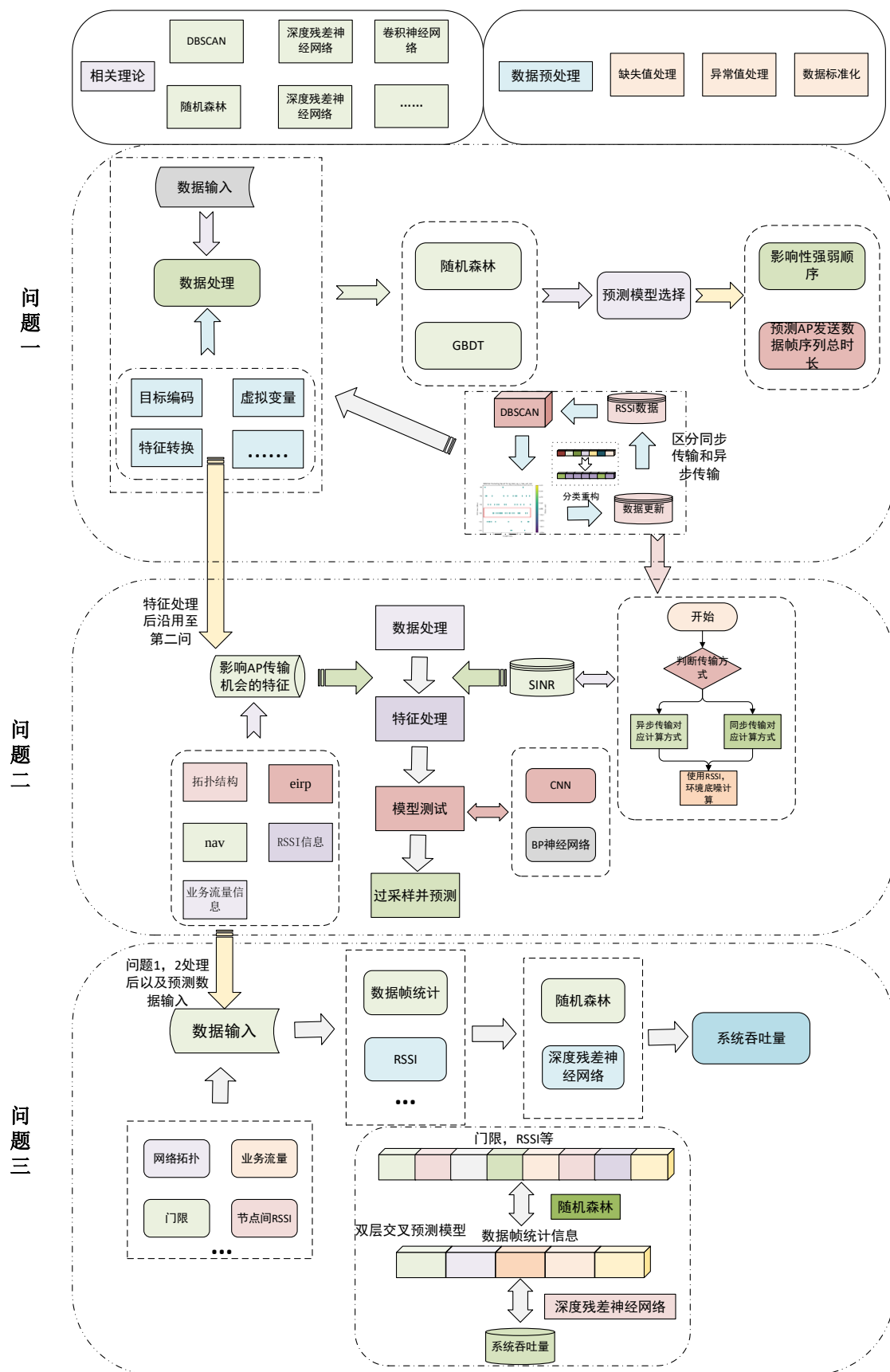


图 1-1 技术路线图

二、模型假设与符号说明

2.1 模型假设

- (1) 本题中不考虑同步传输时发生的偶发性回退为 0 的情况；
- (2) 假设除本文考虑因素外，无其他扰动因素；
- (3) 假设环境噪声水平相对稳定且已知；
- (4) 假设假定信号传播主要受到直射路径的影响，忽略反射、折射等多路径效应；
- (5) 假设 RSSI 等信道状态信息的更新频率足够高，能够反映信道条件的变化；
- (6) 假设训练集中提供的数据能够代表实际部署场景中的各种情况；
- (7) 假设每个数据帧的传输成功率较高，允许忽略由于错误引起的重传对整体吞吐量的影响。

2.2 符号说明

表 2-1 符号说明表

符号	说明
B	表示随机选择 B 个样本子集
$F_m(x)$	表示第 m 轮的模型预测值
$T_m(x)$	第 m 轮构建的决策树
M	表示树的总数
S_j	分割阈值
$W^{(l)}$	全连接层中的权重矩阵
X	模型的输入变量
$\delta^{(l)}$	表示第 l 层的误差
Nss	空间流数
Mcs	MCS 阶数
Per	丢包率
throughput	实测每个 AP 的吞吐量

三、数据预处理

3.1 缺失值处理

缺失数据在现实世界的各种数据集中是一个常见的问题，可能由于数据收集过程中的错误、遗漏或其他原因导致。在进行数据分析和建模时，处理缺失值是一个重要的步骤。

在附件“training_set_3ap_loc30_nav86”中可能由于未监测到、设备故障、环境干扰等原因导致 N、O、P 三列数据缺失，由于在本数据集中，数据并没有标签，且该数据列全部缺失，相邻字段有一部分相关性，并且不能使用常规的方式进行处理，我们团队选择了自编码器进行该数据的缺失值处理。

自编码器（Autoencoder）作为一种强大的神经网络模型，能够通过学习数据的隐含结构来预测和补全缺失数据，可以用于对缺失值进行有效的补全。编码过程如下：

$$z = f(W_{enc}x + b_{enc}) \quad (3.1)$$

$$\hat{x} = g(W_{dec}z + b_{dec}) \quad (3.2)$$

$$L_{train} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2 \quad (3.3)$$

$$L_{val} = \frac{1}{M} \sum_{i=1}^M (x_i^{val} - \hat{x}_i^{val})^2 \quad (3.4)$$

在公式（3.1）和公式（3.2）中， $\hat{x}$ 代表重构后的输入， W_{enc} 和 W_{dec} 分别是编码器和解码器的权重矩阵， b_{enc} 和 b_{dec} 分别是相应的偏置项， f 和 g 分别是编码器和解码器使用的激活函数（本团队使用 ReLU）。在公式（3.3）和公式（3.4）我们采用均方误差计算误差。

模型的训练损失和验证损失如图 3-1，从损失曲线来看，验证损失和训练损失没有明显的波动，表明模型在训练过程中能够较好地拟合数据，没有出现非常不稳定情况，同时这也表明模型学到的数据模式比较稳定，但是模型在验证集上的损失略大于测试集，说明模型具有一定的泛化能力。

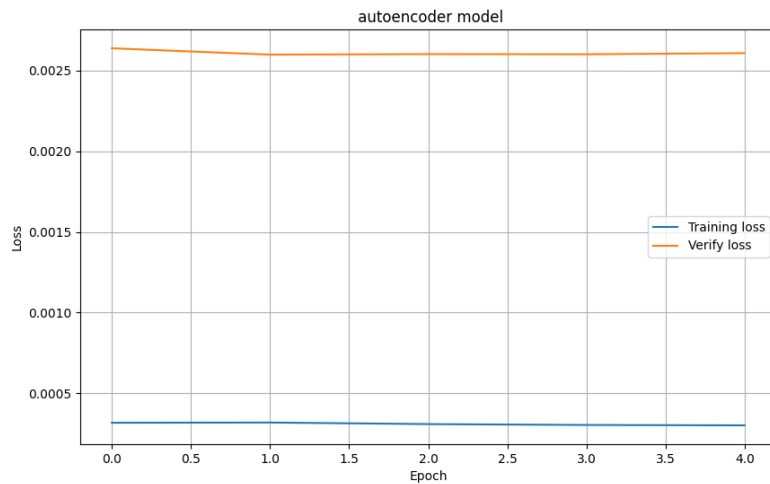


图 3-1 自适应器损失图

3.2 异常值处理

异常值检查是数据分析中的关键步骤，旨在识别和处理数据集中与其他数据点显著不同的异常数据点。按照附件“training set 2ap loc2 nav82”提供的数据，逐一进行异常值检查，发现该附件 80-81 两行数据中的 N、O、P、Q、R、S、T、U、V、W 列数据结构和形式存在明显异于其它值的情况，考虑到在 WLAN 性能分析和吞吐量预测中，异常值可能源于测量误差、设备故障、环境干扰或数据录入错误，如果不处理这些异常值，可能会导致模型偏差，降低预测的准确性和可靠性，故对这两行的数据进行了删除处理。在附件“training_set_3ap_loc33_nav88.csv”中，发现 loc_id 列的数据与数据名不一致，更正为 loc33。

在附件“training_set_2ap_loc1_nav86.csv”中第 13 条记录，nss 为 0，标记为异常值，该条数据进行删除处理。“training_set_3ap_loc33_nav82.csv”附件中第 7 条及第 63 条数据出现 nss 为 0 的情况，对这些数据进行删除处理。测试集“test_set_1_3ap.csv”第 21 条数据出现 nss 为 0 的情况，对该条数据进行删除处理。

3.3 数据标准化

数据标准化是一种数据预处理技术，它在数据分析、机器学习和其他数据密集型任务中被广泛应用。其主要目的是将不同量纲或不同数量级的数据转换为统一的标准形式，以便于比较和分析。标准化有助于算法更快地收敛，并且可以提高模型的性能。常见的数据标准化方法主要有以下几种：

（1）最小-最大缩放（Min-Max Scaling）

通过特征值缩放到一个给定的最大值和最小值之间（通常是 0 到 1）来实现。

公式是： $X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$ ，该方法对异常值非常敏感，因为最大值或最小值的变化会改变整个数据集的分布。

（2）Z-score 标准化（Standardization，Z-score Normalization）

也称为零均值标准化，它将数据转换成均值为 0，标准差为 1 的形式，

公式是： $X' = \frac{X - \mu}{\sigma}$ ，其中 μ 是平均值， σ 是标准差。

此方法不会限定数据的范围，因此适合那些后续处理不希望受数据具体范围限制的情况。

（3）小数定标标准化（Decimal Scaling）

通过移动数据的小数点位置来进行缩放，通常选择一个合适的整数 j 作为缩放因子，使得所有绝对值最大的数值不超过 1。

公式是： $X' = \frac{X}{10^j}$ ，其中 j 满足条件 $\max(|X|) < 1$ 时取最小正整数。

该方法适用于不知道数据分布情况下的初步处理。

（4）鲁棒放缩（Robust Scaling）

使用统计学中的四分位间距（IQR）而不是传统的均值和方差来计算中心趋势和离散度。

公式为： $X' = \frac{X - \text{median}(X)}{IQR}$ 。

当存在大量异常值时，这种基于中位数和 IQR 的方法比 $Z\text{-score}$ 更加稳健。

由于在第一题中我们引入了随机扰动以增强模型对噪声数据的鲁棒性，并提高其泛化能力。为了确保模型训练过程中各特征之间的可比性和一致性，同时考虑到随机扰动可能带来的数据分布变化，我们决定采用 $Z\text{-score}$ 标准化方法对数据进行预处理。

四、问题一：基于控制随机性的随机森林模型进行 seq_time 预测与特征重要性排序

4.1 问题一分析

首先，依据同频 AP 数量（2 个或 3 个）对数据进行分类，因为不同数量的 AP 直接影响信道竞争模式及发送机会。接着，在特征选择与转换方面，对于非连续型特征如协议类型（UDP/TCP）、位置编号（loc）、BSS ID 等，则通过目标编码与虚拟变量的方法将其转化为数值形式，以便采用的机器学习模型能够有效利用这些信息。

针对数据处理部分，本团队发现当 RSSI 值在 PD 和 ED 之间时，由于包长的差异较大，导致在 CCA 机制中错过对方 Preamble 的情况从而判定信道为空闲，导致异步传输，但是在此区间中发生异步传输和同步传输的概率未知。我们采用了 DBSCAN 算法将 RSSI 值进行聚类，在高密度的聚类区域代表同步传输，因为在高密度聚类区域中节点之间的信号强度相对较强且较为稳定；低密度的区域代表了异步传输，反映了节点间信号较弱或不稳定的传输状态。为了更好地应对这种随机性，我们在 DBSCAN 聚类分类之后，加入了一个扰动机制。该扰动机制通过随机翻转一部分分类结果，模拟无线传输中的不确定性和随机性。

在模型建立阶段，分别构建了基于随机森林和 GBDT 的预测模型。随机森林因其在处理高维、非线性关系时表现出色且具有良好的抗过拟合能力而被选中作为基准模型，通过调整参数如树的数量、最大深度等优化其性能。另一方面，GBDT 通过梯度提升的方式逐步改进基础模型来提高预测准确性，重点在于调整学习率、树的复杂度以及正则化参数等，寻找最佳配置。随后，在训练集上比较两种模型的性能指标 MSE 和拟合优度，分析它们在面对不同类型数据（2AP 和 3AP）时的具体表现，确定哪种模型更适合特定场景下的应用。

最终，基于选定的模型输出的特征重要性评分，我们列出了影响 AP 发送数据帧序列总时长（seq_time）的主要因素，并按其对模型预测结果的影响力从大到小进行排序。特征重要性评分的计算依赖于模型中每个特征对决策过程的贡献程度。在确定了主要影响因素后，我们使用拟合好的随机森林模型，对测试集上的 AP 发送数据帧序列总时长进行了预测。

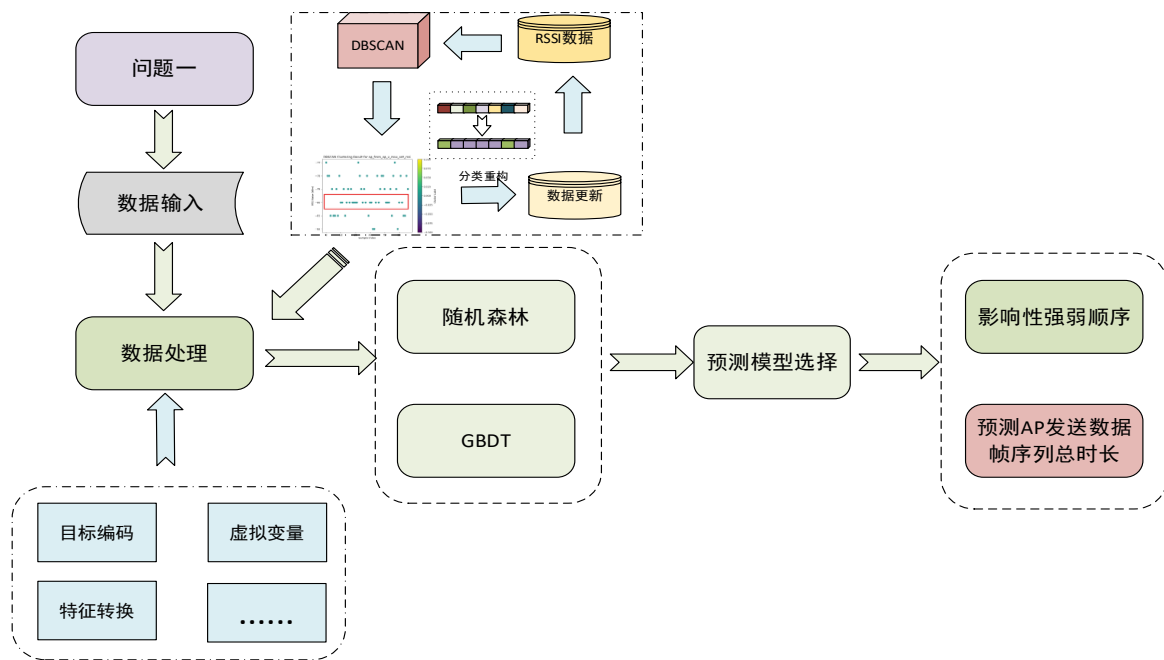


图 4-1 问题一建模流程图

4.2 相关理论

4.2.1 特征工程

(1) 特征选择

特征选择是特征工程中的关键步骤，旨在通过挑选对模型效果最显著的特征，去除冗余或无关特征，从而提升模型的性能。常见方法包括过滤法、包裹法和嵌入法。过滤法依赖于统计分析手段，如相关系数、卡方检验等；包裹法通过评估特征组合对模型的直接影响，常见的技术有向前选择和递归特征消除；嵌入法结合模型训练过程自动选择特征，如决策树等。特征选择可以有效的减少过拟合、提高模型性能、降低计算开销，并且提升模型的可解释性。

(2) 特征提取

特征提取是特征工程中的重要步骤，旨在从原始数据中生成能够有效表达数据核心信息的新特征，以提升模型的性能表现。与特征选择不同，特征提取通过对现有特征进行转换、组合或降维，构造出新的特征，使模型能够更好地识别数据中的模式。常见的方法包括线性和非线性降维技术，如主成分分析（PCA）和 t-SNE，它们可以简化高维数据结构；在文本处理中，词袋模型、TF-IDF 和词向量模型能够有效提取文本中的语义信息；在信号处理中，傅里叶变换和小波变换用于提取频域和时域特征；图像处理中，边缘检测、SIFT 等技术能够提取图像中的关键特征，卷积神经网络（CNN）则通过自动提取高级特征在图像分类等任务中广泛应用。此外，在时间序列分析中，移动平均、标准差等统计特征常用于捕捉序列的趋势和波动性。特征提取不仅能够减少特征的维度、提

高模型的计算效率，还能通过捕捉数据中的隐藏信息，显著提高模型的准确性和鲁棒性，是机器学习和数据分析中的核心技术之一。

（3）特征缩放

特征缩放是特征工程中的一种重要预处理方法，旨在将不同特征的数据缩放到相同的尺度，以消除特征之间量纲差异带来的影响。这对于基于距离或梯度的机器学习模型（如支持向量机、KNN、神经网络等）尤为重要，因为这些模型对特征值的大小较为敏感。常见的特征缩放方法包括：标准化（归一化），即将特征值缩放到 $[0, 1]$ 的范围公式表示为： $X' = (X - X_{\min}) / (X_{\max} - X_{\min})$ ；最大最小缩放，即将数据压缩到特定范围（如 $[0, 1]$ 或 $[-1, 1]$ ）；以及 Robust Scaler，基于中位数和四分位数来缩放，能够有效减少

异常值的影响，其公式表示为： $x' = \frac{x - \text{median}(x)}{IQR}$ 。通过特征缩放，可以避免某些特

征因值域较大而对模型产生过大的影响，帮助提高模型的训练效率和预测性能。

（4）特征转换

特征转换是特征工程中的一重要技术，旨在通过对原始数据进行变换，使其更适合机器学习模型处理，从而提升模型的表现。常见的特征转换方法包括：数据缩放，如标准化和归一化，用于将特征值调整到相同尺度；对数变换，通过取对数减少数据的偏态分布，适合处理极值或指数增长的数据；平方根变换，通过取平方根减小数据的幅度，减少极端值影响；多项式特征，生成原始特征的高次项和交叉项，用于捕捉非线性关系；离散化，将连续数据分段，适用于处理区间特征；类别编码，如独热编码和目标编码，将类别变量转换为数值数据；特征交互，通过特征之间的乘积或其他组合生成新特征；以及时间特征提取，用于从时间序列数据中提取有意义的时间信息。特征转换的目的在于增强模型对数据的理解，解决数据分布问题，进而提高模型的泛化能力和预测性能。

由于测试条件不一致，包括但不限于测试采样时间、测试轮次等问题，测试过程中发生噪声的发生概率不同进而导致去除噪声后相关数据长度不一致，无法使用简单的分类进行说明，故本题中我们选择以百分比的形式对特征进行类别编码，同时对训练集中 loc_bss_id 进行了目标编码的处理。

4.2.2 随机森林

随机森林(Random Forest)是一种集成学习方法，广泛应用于分类和回归任务中，属于 Bagging 的一种。它通过构建多个决策树并结合它们的预测结果来提升模型的准确性和稳健性。随机森林首先从原始数据集中通过有放回的抽样方法生成多个不同的子集，每个子集用于训练一棵独立的决策树。在每棵树的构建过程中，随机森林不仅在样本选择上引入随机性，还在每个节点的特征选择上进行随机抽取，这样每棵树在分裂节点时只考

考虑部分特征，从而增加了模型的多样性，减少了各树之间的相关性。训练完成后，对于分类任务，随机森林通过多数投票机制决定最终的类别预测；对于回归任务，则通过平均各树的预测值来得到最终结果。具体示意图如图 4-2 所示。

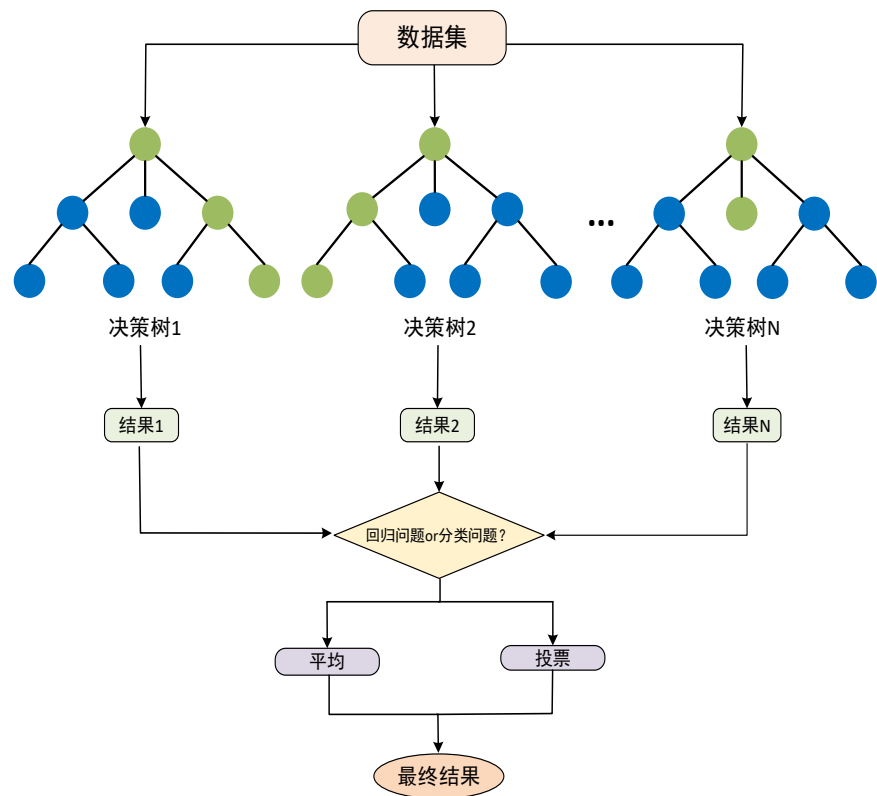


图 4-2 随机森林示意图

此外，随机森林模型能够评估各个特征的重要性，明确哪些特征对模型性能的影响最大。这一过程基于单个决策树在选择特征时所使用的信息增益或不纯度减少等度量标准。具体来说，随机森林通过计算每个特征被用于节点分裂的频率以及每次分裂后不纯度减少的程度来量化特征的重要性。特征重要性的数值通常介于 0 和 1 之间，数值越大表明该特征对于分类或回归任务的贡献越显著。

由于本题中需要将各参数对 AP 发送机会影响性强弱进行排序，并且本题属于回归问题的范畴，因此考虑使用经典的随机森林算法进行处理。

4.2.3GBDT

GBDT 的全称为 Gradient Boosting Decision Trees，梯度提升决策树。属于 Boosting 的一种，是机器学习算法中的集成学习方法。基本思想为：通过迭代地构建一系列弱预测模型（通常是决策树），并将这些模型以加权的方式组合起来，形成一个强大的预测模型。GBDT 的核心思想在于每次迭代时都试图减少前一轮模型的残差，即前一轮模型未能准确预测的部分。通过这种方式，每一轮新加入的模型都在努力纠正前面所有模型犯下的错误，从而逐步提高整体模型的性能。GBDT 具有许多优点，如鲁棒性较强；可解释性

较好；能够有效地处理数据集中的缺失值；训练过程并行化，适用于大规模数据，可以加快训练速度等。具体示意图如图 4-3 所示。

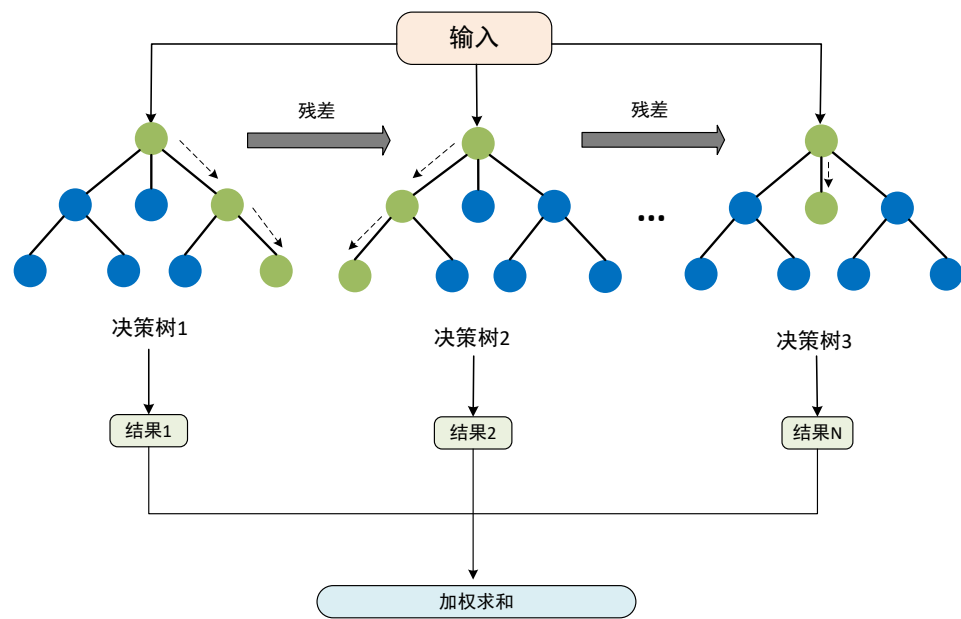


图 4-3 GBDT 示意图

本题属于回归问题的也可考虑使用 GBDT 算法，将本算法预测结果与随机森林预测结果进行对比，选择预测效果更好的算法预测结果作为本作品本题的结果。

4.2.4 虚拟变量

虚拟变量也称为哑变量，是一种用于将分类变量转换为可以被统计模型和机器学习算法处理的形式的方法，用字母 D 表示。虚拟变量通过创建一组二进制（0 或 1）变量来表示每一个类别，从而实现这一转换。本题将虚拟变量设定为：

$$D = \begin{cases} 1, & protocol \text{ 特征取值为 } tcp \text{ 时} \\ 0, & protocol \text{ 特征取值为 } udp \text{ 时} \end{cases}$$

本题中首先依据同频 AP 数量（2 个或 3 个）对数据进行分类，因为不同数量的 AP 直接影响信道竞争模式及发送机会。接下来，在特征选择与转换方面，针对 protocol 特征列中的 TCP 和 UDP 值，创建虚拟变量，即将协议类型转化为二进制形式。具体来说，我们为 protocol 创建一个新列，命名为 is_TCP，当协议类型为 TCP 时赋值为 1，当协议类型为 UDP 时赋值为 0。这样就实现了对 protocol 特征的编码，使得它可以直接被用于训练模型。

本题需要对附件中 protocol 特征列的 tcp 和 udp 两个变量进行分类，以二进制（0 或 1）的形式用以将二者进行分类，从而能够输入到模型当中。

4.3 随机传输特征处理：DBSCAN 密度聚类

4.3.1 数据处理

在无线局域网（WLAN）中，RSSI 是影响节点间通信的重要参数。根据题目中所给的信息结合 IEEE 802.11 系列标准，当两个节点之间的 $RSSI > ED$ 时，一个节点可以检测到另一个节点的数据传输。这种情况下，两个节点可以“听到”彼此，表示它们能够侦听到对方正在传输数据。本团队不考虑偶然同时回退到 0 时，多 AP 同时开始数据传输极小的概率。当 RSSI 值在 PD 和 ED 之间时，由于包长的差异较大，导致在 CCA 机制中错过对方 Preamble 的情况从而判定信道为空闲，导致异步传输，但是在此区间中发生异步传输和同步传输的概率未知。当 RSSI 值小于 PD 时，则说明该信号非常弱，可能来自于一个较远的节点或者干扰源，此时节点会判断信道是空闲的，会导致产生异步传输。

由于无法判断 RSSI 值在 PD 和 ED 之间时的传输方式，本团队通过查阅相关文献，决定使用 DBSCAN 来进行传输方式的划分，将 RSSI 值进行 DBSCAN 聚类，在高密度的聚类区域代表同步传输，因为在高密度聚类区域中节点之间的信号强度相对较强且较为稳定；低密度的区域代表了异步传输，反映了节点间信号较弱或不稳定的传输状态。同时我们还考虑到这部分同步传输和异步传输的判断取决于随机回退的随机性，然而，这一机制也带来了不可预测的随机性，尤其是在信号强度接近 PD 和 ED 的临界情况下。节点在回退到 0 时可能会同时开始传输，或者因错过其他节点的 Preamble（前导信号）而错误地判断信道为空闲，导致同步和异步传输的混合发生。

为了更好地应对这种随机性，我们在 DBSCAN 聚类分类之后，加入了一个 **扰动机制**。该扰动机制通过随机翻转一部分分类结果，模拟无线传输中的不确定性和随机性。具体来说，在高密度区域（DBSCAN 聚类识别为同步传输）和低密度区域（识别为异步传输）之间，加入随机扰动能够更真实地反映实际网络中由于随机回退和干扰导致的传输方式不稳定性。

通过这种方式，我们不仅通过 DBSCAN 聚类模型区分了不同密度的传输方式，还进一步引入了随机扰动，增强了模型对实际网络环境中随机性和不确定性的鲁棒性。该方法可以更精确地模拟 WLAN 中复杂的传输模式，尤其是在多节点环境中，信道竞争激烈时的情况。

本团队数据具体的数据处理如图 4-4 所示。

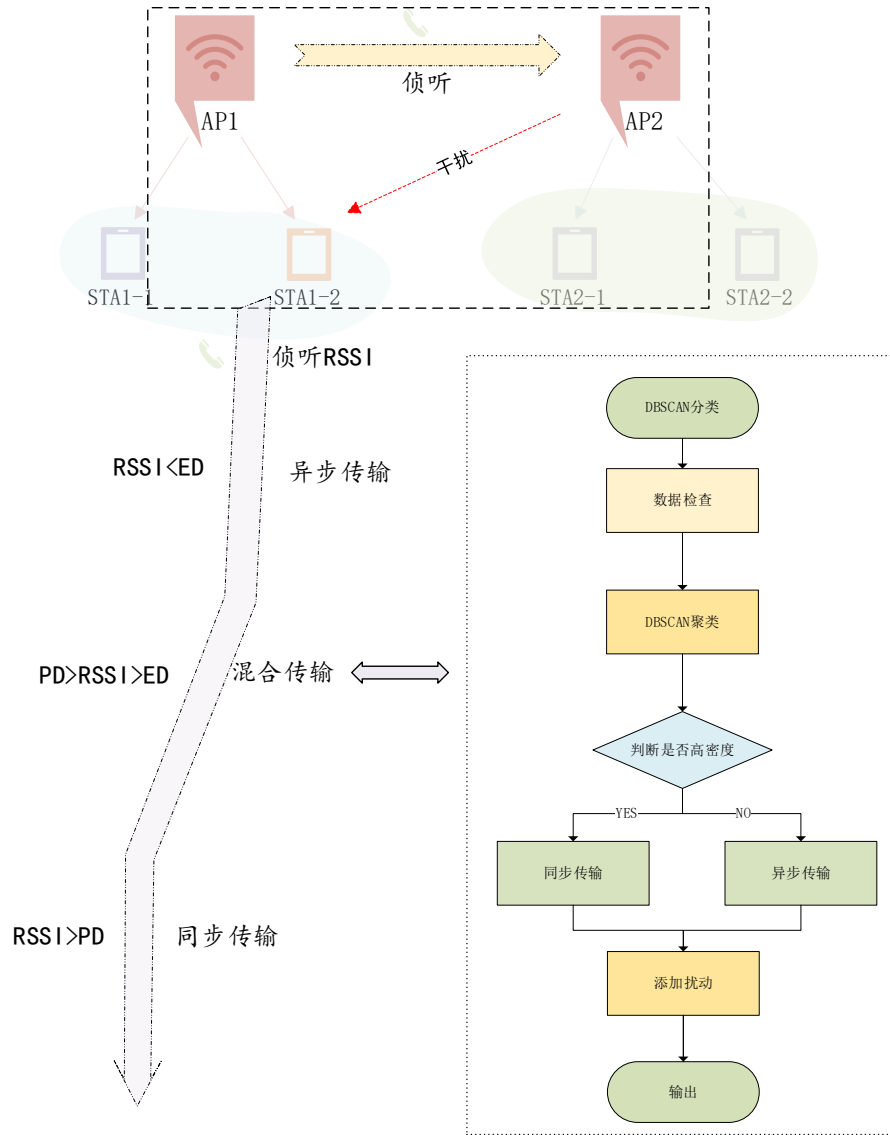


图 4-4 RSSI 对应数据转换

4.3.2 数据分布与算法确定

DBSCAN 是一种基于密度的聚类算法，它能够发现任意形状的簇，同时还能识别噪声点（即不属于任何簇的数据点）。DBSCAN 通过基于局部密度的聚类方法，定义簇为高密度区域内的点集，并将低密度区域的数据点标记为噪声。具体的公式如下：

$$N_{\epsilon}(P) = \{q \in D \mid \text{dist}(P, q) \leq \epsilon\} \quad (4.1)$$

$$|N_{\epsilon}(P)| \geq \text{min_samples} \quad (4.2)$$

$$|N_{\epsilon}(P)| < \text{min_samples} \quad \text{and} \quad \exists \text{ core point } q, P \in N_{\epsilon}(q) \quad (4.3)$$

$$P \text{ is noise if } P \text{ is neither a core point nor a border point.} \quad (4.4)$$

公式(4.1)中为领域定义， $\text{dist}(P, q)$ 表示点 P 到 q 的欧几里得距离，公式(4.2)表示点 P 的 ϵ 邻域内的点数大于等于 min_samples ，则该点 P 是一个核心点，公式(4.3)点 P 不满足

核心点的条件，但它位于某个核心点的邻域内。公式(4.4)表示如果点 P 既不是核心点，也不是边界点，则它是噪声点。

DBSCAN 的方法优势如下：与 K-means 不同，DBSCAN 不需要提前知道簇的数量，它通过局部密度来自动发现簇的数量。同时，DBSCAN 通过密度连接点来发现簇，因此能识别任意形状的簇，适合处理形状不规则的数据集。本团队使用的方法示意图如图 4-6 所示。

在基础的 DBSCAN 方法上，我们额外添加了扰动因素，公式如下所示：

$$\text{Label}_i^{\text{perturbed}} = \begin{cases} 1 - \text{Label}_i, & \text{if } i \in S \\ \text{Label}_i, & \text{if } i \notin S \end{cases} \quad (4.5)$$

在公式(4.5)中， $\text{Label}_i^{\text{perturbed}}$ 为加入扰动后的标签， S 属于选中进行扰动的集合。

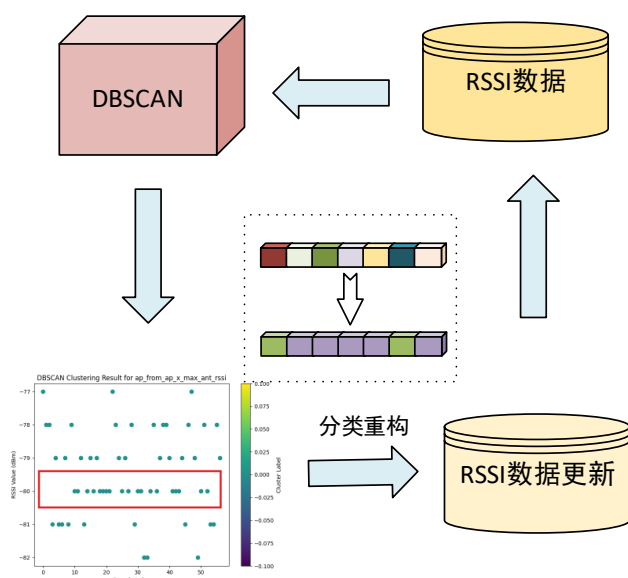


图 4-6 DBSCAN 数据处理示意图

4.3.3 DBSCAN 算法求解

本节中截取了部分字段的部分测试结果进行标注，将 DBSCAN 部分结果高密度区域可视化，如图 4-7 所示。高密度聚类区域中节点之间的信号强度相对较强且较为稳定，代表了同步传输，低密度的区域代表了异步传输，反映了节点间信号较弱或不稳定的传输状态。

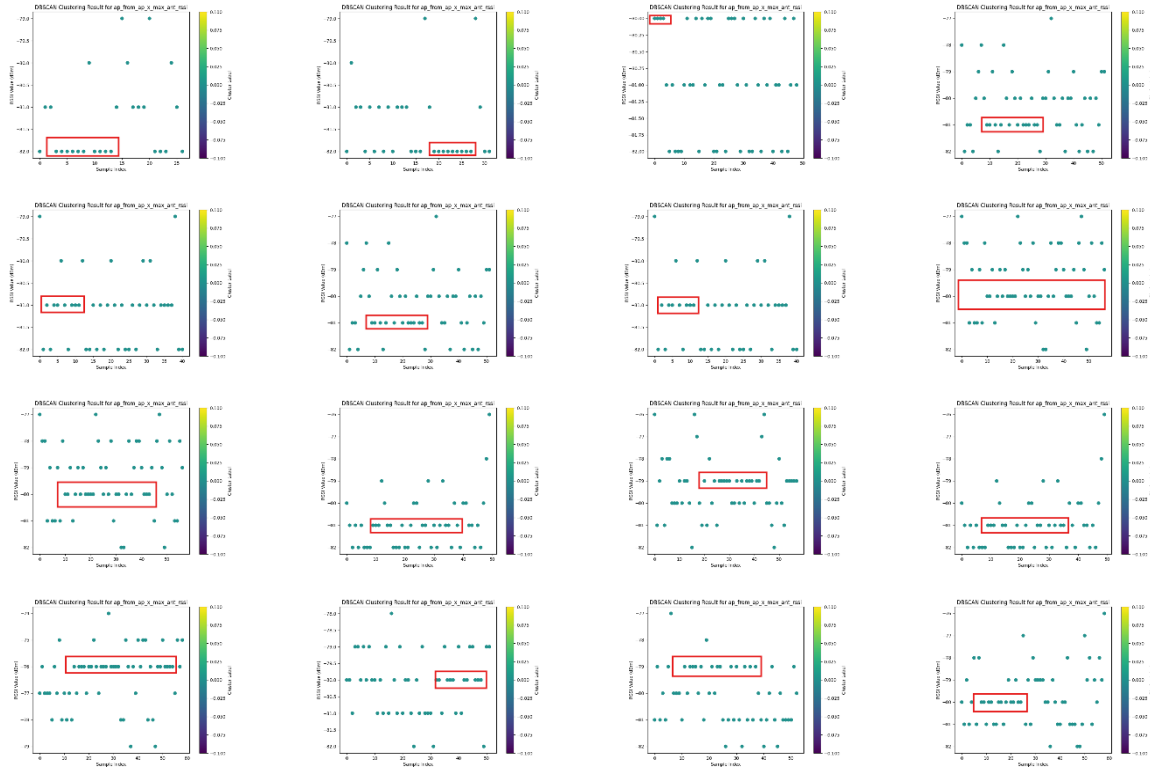


图 4-7 DBSCAN 部分结果高密度区域可视化

4.4 2ap 情况下模型建立与求解

4.4.1 特征选择与处理

在本文中，根据题目所述的网络拓扑、业务流量、门限、节点间 RSSI 四个基本信息，选取 loc_id 和 bss_id 两个特征代表网络拓扑信息；选取 protocol 特征代表业务流量信息；选取 nav 特征代表门限信息；选取 eirp、ap_from_ap_x_max_ant_rssi、ap_from_ap_x_mean_ant_rssi、sta_to_ap_x_max_ant_rssi、sta_to_ap_x_mean_ant_rssi、sta_from_ap_x_max_ant_rssi、sta_from_ap_x_mean_ant_rssi 等特征代表节点间 RSSI 信息。输出变量为 seq_time，即 AP 发送数据帧序列的总时长。

根据本文业务流量模式 TCP 的工作原理，接收方需要通过回复 TCP ACK 确认收到的数据，并且题目中所有测试的 AP 发射模式都是 RTS-CTS 模式，帧序列时长的计算方式是指从发送 RTS 到接收 ACK，结合 IEEE 802.11 系列标准，本团队将 STA 侦听 AP、STA 的信号也作为影响因素考虑。

对于代表网络拓扑信息的 `loc_id` 与 `bss_id` 两个特征，本文将这两个特征融合为一个定性特征 `loc_bss_id`，而后采用目标编码的方法将 `loc_bss_id` 转换为一个定量特征。对于代表业务流量信息的 `protocol` 特征，将其转换为一个虚拟变量。具体地，如果 `protocol` 取值为 `tcp`，那么取值为 1；反之则取值为 0。对于代表门限信息的 `nav` 特征，由于其本身就是连续特征，所以不对其进行处理。对于代表节点间 RSSI 信息的特征，首先根据 DBSCAN 算法将数据分类为同步传输与异步传输两类，而后计算异步传输的所占比例作为输入特征。对于 `ap_from_ap_0_max_ant_rssi` 以及 `ap_from_ap_1_max_ant_rssi` 两个特征，由于它们可以相互补充，并且都代表一个 AP 检测到来自另一个的 RSSI 的最大值，所以本文将这两个特征融合为一个特征，即 `ap_from_ap_x_max_ant_rssi`。对于 `ap_from_ap_0_mean_ant_rssi` 以及 `ap_from_ap_1_mean_ant_rssi`，进行同样的操作，将它们也融合为一个特征，即 `ap_from_ap_x_mean_ant_rssi`。

4.4.2 模型建立

随机森林的模型如下：

$$x_j \leq s_j \quad (\text{左子节点}) \quad (4.6)$$

$$x_j > s_j \quad (\text{右子节点}) \quad (4.7)$$

$$\hat{y} = \frac{1}{B} \sum_{k=1}^B T_k(x) \quad (4.8)$$

随机森林是指每个决策树通过不断地递归分割特征空间来进行构建的一种方法，对于特征空间分割，在公式(4.6)和公式(4.7)中表示当在点 j 时，选择了特征 x_j 和分割阈值 s_j 来将数据分为两部分。

公式(4.8)中， $T_k(x)$ 表示第 k 棵树对输入 x 的预测结果。

GBDT 的模型如下：

$$L(y, F(x)) = \frac{1}{2} (y - F(x))^2 \quad F_m(x) = F_{m-1}(x) + \eta \cdot T_m(x)$$

$$L(y, F(x)) = \frac{1}{2} (y - F(x))^2 \quad (4.9)$$

$$F_m(x) = F_{m-1}(x) + \eta \cdot T_m(x) \quad (4.10)$$

在公式(4.9)和公式(4.10)中， $F_m(x)$ 表示第 m 轮的模型预测值， $T_m(x)$ 表示在第 m 轮构建的决策树。 η 是学习率，用于控制每棵树的贡献。对于均方误差损失函数而言，负梯度表示残差：

$$r_i^{(m)} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} = y_i - F_{m-1}(x_i) \quad (4.11)$$

公式(4.11)表示每轮的树 $T_m(x)$ 用于拟合 $r_i^{(m)}$ 。然后模型通过不断的累加构建，最后形成了如下的模型：

$$F(x) = F_0(x) + \sum_{m=1}^M \eta \cdot T_m(x) \quad (4.12)$$

在公式(4.12)中， $F_m(x)$ 表示第 m 轮的模型预测值， $T_m(x)$ 表示在第 m 轮构建的决策树， M 表示树的总数，一般这个参数是用于控制复杂度的。

由于该模型中，我们考虑到 WLAN 传输过程中有许多不可控的因素，除了我们模型假设中的以及通过 DBSCAN 聚类进行传输模式的划分之外，我们在模型中额外加入了高斯扰动，最终的模型如下：

$$\hat{y} = \frac{1}{B} \sum_{k=1}^B T_k(x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1) \quad (4.13)$$

$$F(x) = F_0(x) + \sum_{m=1}^M \eta \cdot T_m(x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1) \quad (4.14)$$

在公式(4.13)和公式(4.14)中，额外添加了一部分的 ϵ 作为高斯扰动，并且该部分扰动服从(0,1)正态分布。

4.4.3 指标相关性分析——热力图

经过上一部分的处理，已经将各定性特征转换为定量特征，计算各特征间以及特征与输出变量间的相关系数，并绘制热力图，如下图 4-8 所示：

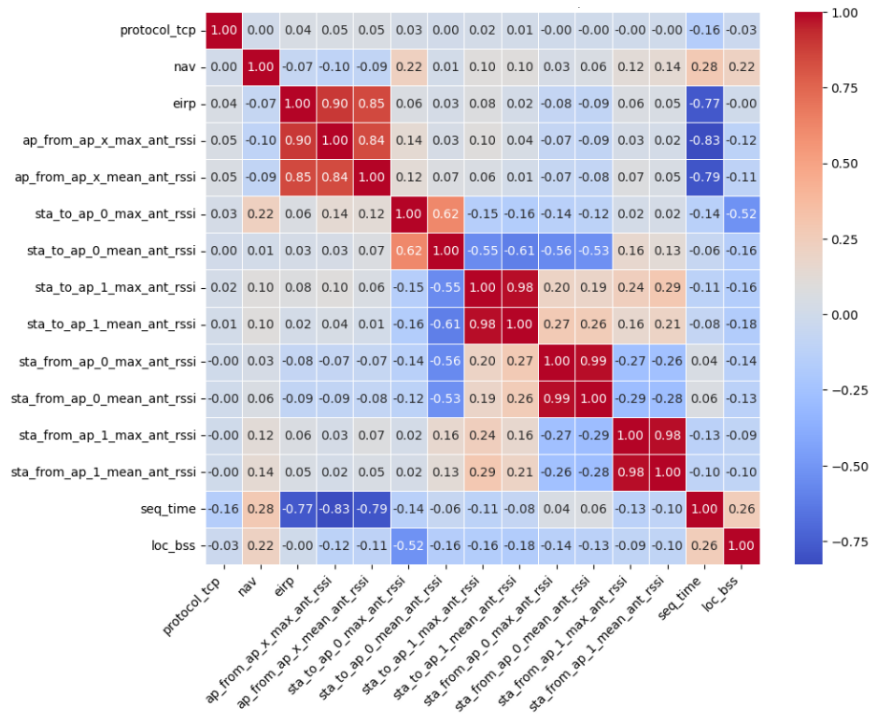


图 4-8 2ap 情况下各特征与标签热力图

通过热力图可以发现，eirp、ap_from_ap_x_max_ant_rssi 以及 ap_from_ap_x_mean_ant_rssi 三个特征与 seq_time 之间的相关性较大，相关系数绝对值在 0.8 左右。因此可以初步判断出节点间 RSSI 信息对 AP 发送数据帧序列总时长的影响程度高于其余三类信息。代表网络拓扑信息的 loc_bss 特征与 seq_time 之间的相关系数为 0.26；代表业务流量信息的 protocol 特征与 seq_time 之间的相关系数为-0.16；代表门限信息的 nav 特征与 seq_time 之间的相关系数为 0.28。相关系数绝对值均小于 0.3，说明相关性不高，同时对输出变量 AP 发送数据帧序列总时长的影响程度不及节点间 RSSI 信息。同时可以看出，在代表节点间 RSSI 信息的特征中，eirp 与 ap_from_ap_x_max_ant_rssi 以及 ap_from_ap_x_mean_ant_rssi 间有着较强的相关性；各天线检测到的 RSSI 中的最大值与平均值间也存在着较强的相关性。

4.4.4 模型分析

根据上述预处理后的数据，按照 80%:20%划分训练集与测试集，在训练集上拟合随机森林以及 GBDT 模型，而后在测试集上测试，计算预测值与真实值之间的残差平方和 MSE 以及拟合优度 R^2 ，最终结果如下所示：

表 4-1 2ap 情况下随机森林与 GBDT 结果对比

模型	残差平方和 MSE	拟合优度 R^2
随机森林	9.09	0.91
GBDT	10.74	0.90

通过对比可以发现，随机森林的残差平方和小于 GBDT，同时拟合优度高于 GBDT，这说明随机森林取得了比 GBDT 更好的效果。绘制随机森林以及 GBDT 模型下的特征重要性排序图，如下所示。

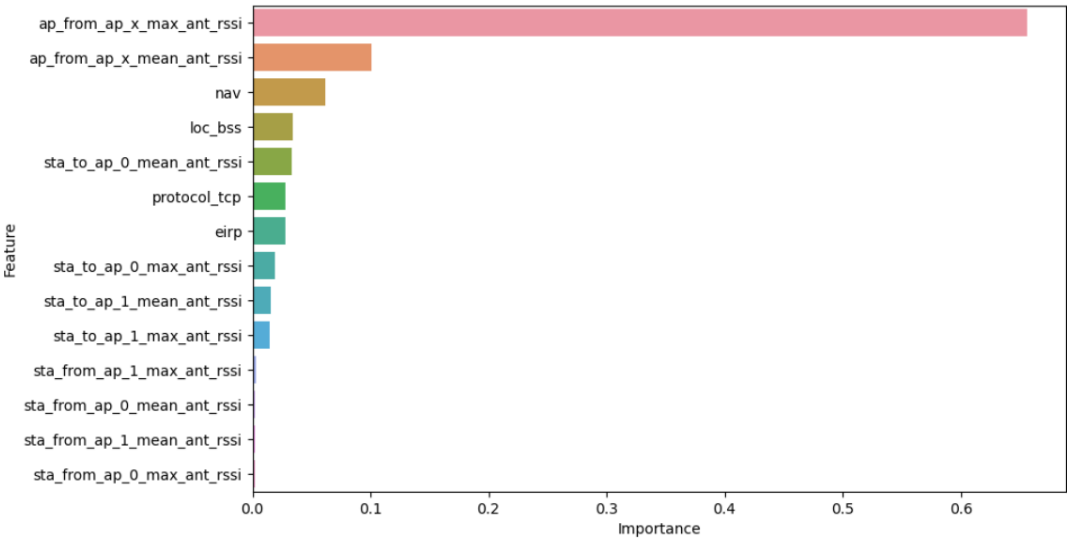


图 4-9 2ap 情况下随机森林特征重要性排序

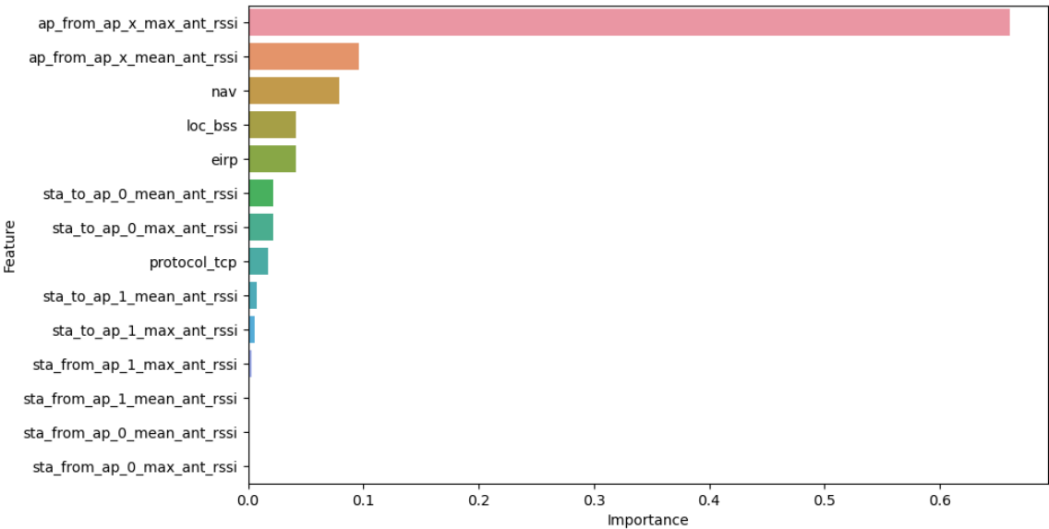


图 4-10 2ap 情况下 GBDT 特征重要性排序

由以上两幅图可以看出，在随机森林中，`ap_from_ap_x_max_ant_rssi` 这一特征的重要性占比最高，重要性占比超过六成；随后是 `ap_from_ap_x_mean_ant_rssi` 这一特征，重要性占比约为 13%；而后依次为 `nav`、`loc_bss` 等八个特征，重要性占比不足 10%，对 AP 发送数据帧序列总时长 `seq_time` 有略微影响；而以 `sta_from_ap` 开头的特征的重要性占比接近于 0，它们对输出变量的影响可以忽略。从网络拓扑、业务流量、门限、节点间 RSSI 四方面看，节点间 RSSI 信息对输出变量的影响程度最大，随后是门限信息，而后是网络拓扑信息，最后是业务流量信息，即节点间 RSSI 信息>门限信息>网络拓扑信息>业务流量信息。就节点间 RSSI 信息内部来看，`ap_form_ap`，即 AP 检测到的来自另一个 AP 的 RSSI 对 `seq_time` 具有显著性影响；随后是 `sta_to_ap`，即 AP 检测到的来自于 STA 的 RSSI；其次是 `eirp`，即 AP 的发送功率；最后是 `sta_form_ap`，即 STA 收到的来自于 AP 的 RSSI，其对 AP 发送数据帧序列总时长 `seq_time` 几乎没有影响。

在 GBDT 中，各特征的排名与随机森林中的各特征排名大致相同，主要相差在 `eirp`，即 AP 发送功率这一特征。在随机森林中，`eirp` 排名第七；而在 GBDT 中，`eirp` 排名第五。因此从节点间 RSSI 信息内部来看，`ap_form_ap` 的重要性>`eirp`>`sta_to_ap`>`sta_form_ap`。但是从网络拓扑、业务流量、门限、节点间 RSSI 四方面信息看，GBDT 重要性排序与随机森林相同，具体为：节点间 RSSI 信息>门限信息>网络拓扑信息>业务流量信息。

4.4.5 预测结果分析

通过比较，本文最终选择随机森林模型为 2ap 情况下的最终预测模型。对于测试集中的非定量特征，采取与 4.3.2 中的相同方法，即采用目标编码、虚拟变量与特征变换等方法对非定量特征进行处理。然后将处理后的数据输入至拟合好的随机森林模型中，得到预测结果。由于本文进行了特征选取操作，并没有使用所有特征作为输入，所以预测值与真实值间会存在细微差别。为了能够更精确地预测 AP 发送数据帧序列总时长 `seq_time`，本文在预测结果的基础上，加上高斯扰动的影响，以表示本文中未考虑的特征对输出变量 `seq_time` 的影响。具体预测结果以及预测值与添加噪声后的预测值对比图如下所示：

表 4-2 2ap 情况下 seq_time 最终预测结果

test_id	seq_time	test_id	seq_time	test_id	seq_time
1	48.28	15	46.33	28	28.92
1	47.57	15	48.85	28	38.71
2	50.92	16	48.81	29	31.05
2	50.55	16	50.27	29	31.74
3	47.92	17	43.46	30	31.80
3	49.43	17	46.33	30	34.16
4	50.97	18	44.35	31	32.19
4	50.45	18	49.64	31	31.82
5	48.10	19	35.25	32	31.02
5	48.05	19	48.23	32	29.95
6	50.80	20	37.16	33	28.16
6	51.08	20	51.35	33	26.77
7	47.69	21	36.71	34	25.25
7	47.88	21	45.43	34	29.04
8	51.32	22	37.29	35	26.35
8	53.08	22	47.49	35	24.70
9	47.01	23	31.61	36	28.64
9	49.37	23	45.05	36	31.10
10	51.73	24	32.27	37	30.04
10	49.33	24	47.93	37	29.82
11	46.50	25	32.50	38	27.83
11	48.13	25	38.56	38	27.98
12	48.64	26	36.20	39	26.83
12	50.25	26	40.47	39	29.05
13	45.85	26	36.20	40	28.50
13	49.69	26	40.47	40	28.54
14	48.13	27	27.40		
14	50.44	27	39.43		

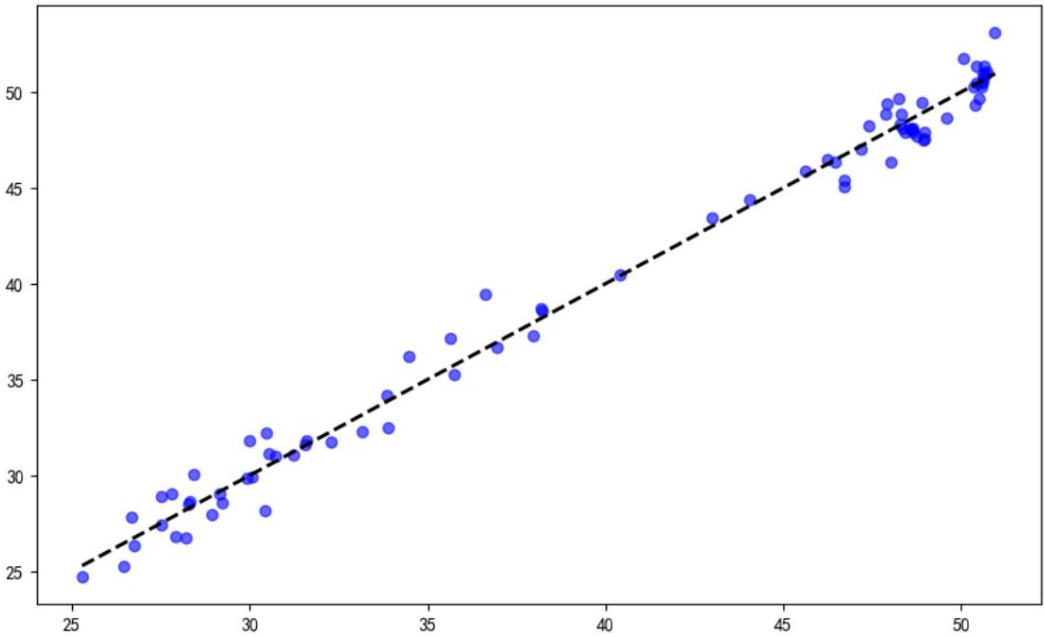


图 4-11 预测值与添加噪声后的预测值对比图

4.5 3ap 情况下模型建立与求解

4.5.1 特征选择与处理

3ap 情况下的特征选取与预处理和 2ap 情况下大致相同，主要有以下区别：将 ap_from_ap_0_max_ant_rssi、ap_from_ap_1_max_ant_rssi 以及 ap_from_ap_2_max_ant_rssi 三个特征融合为两个特征，即 ap_from_ap_x_max_ant_rssi 与 ap_from_ap_y_max_ant_rssi；将 ap_from_ap_0_mean_ant_rssi、ap_from_ap_1_mean_ant_rssi 以及 ap_from_ap_2_mean_ant_rssi 三个特征也融合为两个特征，即 ap_from_ap_x_mean_ant_rssi 与 ap_from_ap_y_mean_ant_rssi。同时 training_set_3ap_loc30_nav86.csv 数据中存在三列缺失值，分别为 ap_from_ap_0_sum_ant_rssi、ap_from_ap_0_max_ant_rssi 以及 ap_from_ap_0_mean_ant_rssi。本文采用自编码器对这三列数据进行填补，但是实验发现填补效果不佳。主要体现在：在原始数据的基础上加入填补数据后拟合模型，模型的预测性能不及仅使用原始数据拟合的预测模型。因此剔除 training_set_3ap_loc30_nav86.csv 中的数据，使用其他 3ap 文件中的数据来拟合预测模型。

4.5.2 指标相关性分析——热力图

与 2ap 的情况相同，计算各特征间以及特征与输出变量间的相关系数，并绘制热力图，如下图所示：

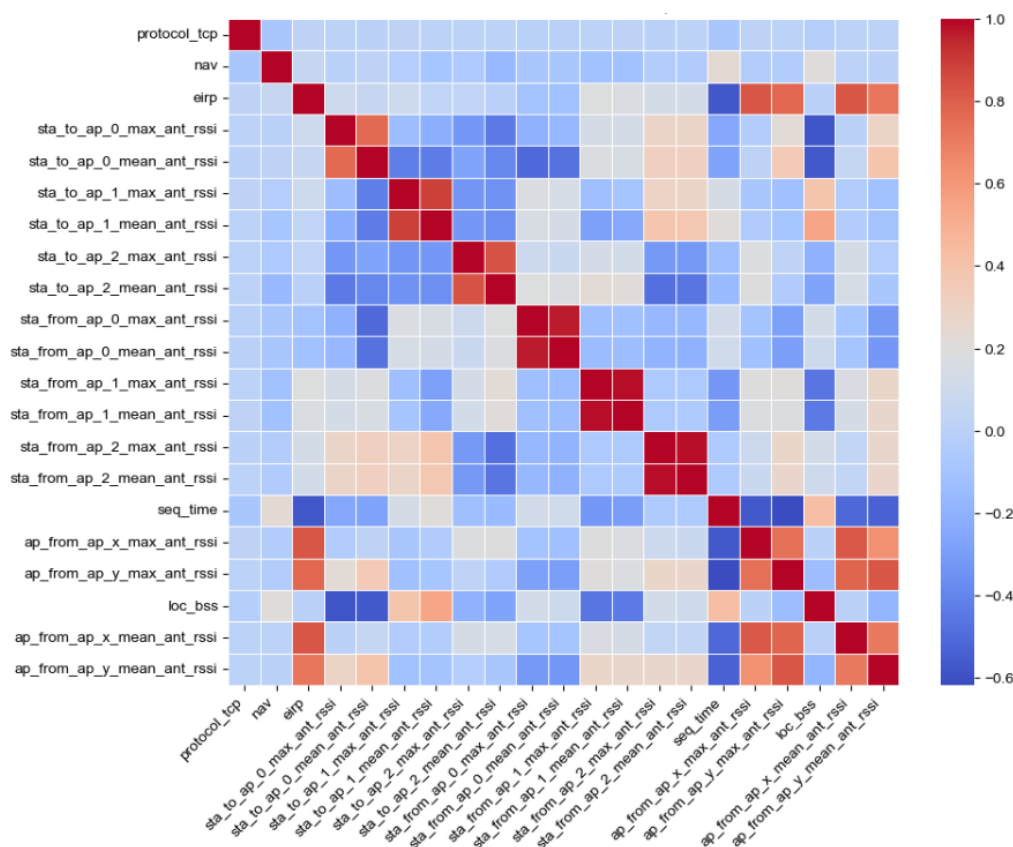


图 4-12 2ap 情况下各特征与标签热力图

通过热力图可以发现，输出变量 AP 发送数据帧序列总时长 seq_time 与 eirp、ap_from_ap_x_max_ant_rssi、ap_from_ap_y_max_ant_rssi、ap_from_ap_x_mean_ant_rssi 以及 ap_from_ap_y_mean_ant_rssi 五个特征间有着较强的相关关系，而这五个特征都属于节点间 RSSI 信息的范畴。而 loc_bss、nav 以及 protocol 三个特征与输出变量间的相关关系较弱，同时这三个特征隶属于网络拓扑信息、业务流量信息以及门限信息的范畴。因此从相关性分析中可以看出，节点间 RSSI 信息相较于其他三种信息而言，对输出变量有着更大程度的影响。

4.5.3 模型建立与重要性排序

按 80%：20%的比例划分训练数据集与测试数据集，然后分别拟合相应的随机森林以及 GBDT 模型，在测试数据集上进行测试，计算预测值与真实值之间的残差平方和 MSE 与拟合优度 R^2 ，最终结果如下表所示：

表 4-3 3ap 情况下随机森林与 GBDT 结果对比

模型	残差平方和MSE	拟合优度 R^2
随机森林	43.10	0.71
GBDT	51.68	0.65

由表可得，随机森林与 GBDT 的拟合效果均没有达到一个较好的水平。但是对比来看，随机森林的拟合效果好于 GBDT：残差平方和较小、拟合优度较高。所以在后续的预测中，本文采用随机森林模型为最终的预测模型。

绘制随机森林以及 GBDT 模型下的特征重要性排序图，如下所示。

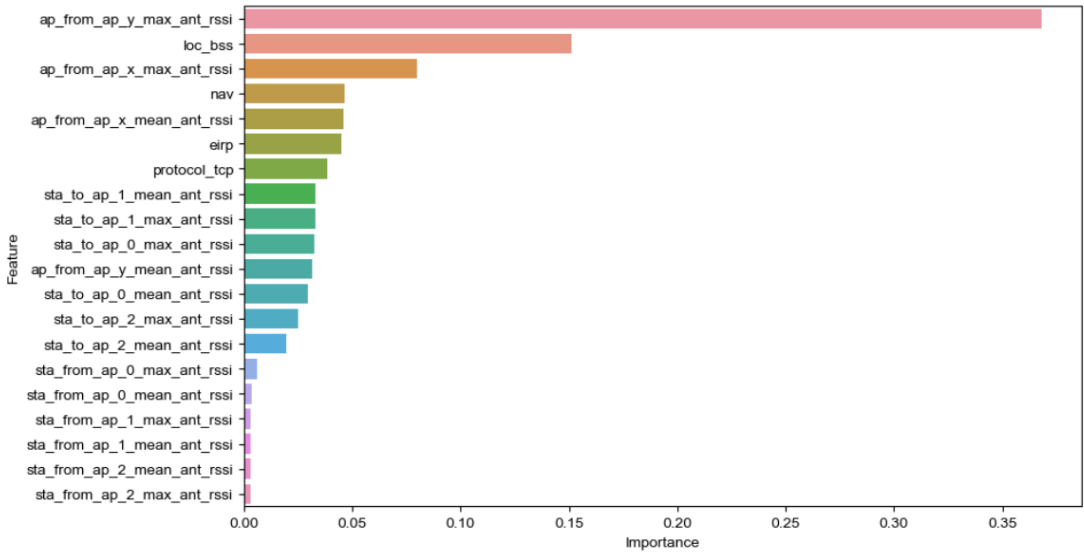


图 4-13 3ap 情况下随机森林特征重要性排序

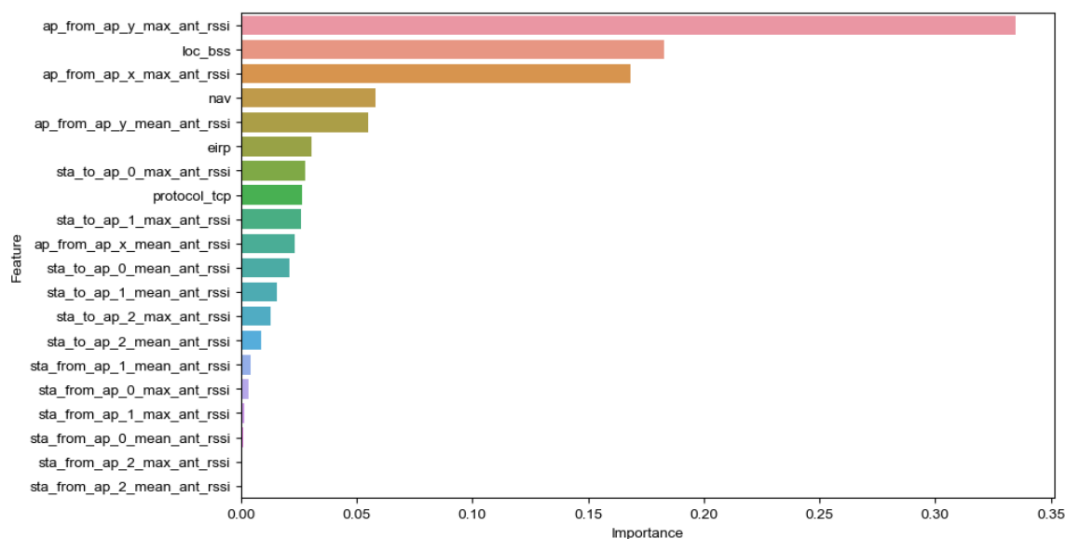


图 4-14 3ap 情况下 GBDT 特征重要性排序

在随机森林中，`ap_from_ap_y_max_ant_rssi` 这一特征对输出变量 AP 发送数据帧序列总时长 `seq_time` 的影响程度最大，重要性占比超过 35%；然后是 `loc_bss` 这一特征，重要性占比约为 15%；其次是 `ap_from_ap_y_max_ant_rssi`，重要性占比约为 8% 左右；其余指标除 `sta_from_ap` 外，对输出变量有着轻微的影响；而 `sta_from_ap` 这一系列的指标的重要性占比可以忽略不计，对 `seq_time` 的影响程度接近于 0。分网络拓扑信息、业务流量信息、门限信息以及节点间 RSSI 信息四大方面看，节点间 RSSI 信息对 `seq_time` 的影响程度最大；然后是网络拓扑信息，表现为 `loc_bss` 特征；其次是门限信息，表现为 `nav` 特征；最后是业务流量信息，表现为 `protocol` 特征。就节点间 RSSI 信息而言，`ap_form_ap` 的重要程度大于 `eirp`，紧接着分别为 `sta_to_ap` 以及 `sta_from_ap`。

在 GBDT 中，各特征重要性排名基本与在随机森林中一致。分网络拓扑信息、业务流量信息、门限信息以及节点间 RSSI 信息四大方面看，重要性排序仍然为节点间 RSSI 信息>网络拓扑信息>门限信息>业务流量信息。就节点间 RSSI 信息而言，各特征重要性排序也仍然为 `ap_form_ap`>`eirp`>`sta_to_ap`>`sta_from_ap`，即 AP 检测到的来自另一个 AP 的 RSSI>AP 的发送功率>AP 检测到的来自于 STA 的 RSSI>STA 收到的来自于 AP 的 RSSI。

4.5.4 预测结果分析

对于文件 `test_set_1_3ap.csv` 中的非定量特征，同样采取虚拟变量、目标编码以及特征选择等方法将其转换为定量特征。然后将数据输入至拟合好的随机森林模型中，得到相应预测值。与 2ap 的情况相同，在得到的预测值的基础上加上高斯扰动，以表示本文中未考虑的特征对输出变量 AP 发送数据帧序列总时长 `seq_time` 的影响。具体预测结果以及预测值与添加噪声后的预测值对比图如下所示：

表 4-4 3ap 情况下 seq_time 最终预测结果

test_id	seq_time	test_id	seq_time	test_id	seq_time	test_id	seq_time
1	29.34	10	25.84	19	22.85	28	14.93
1	42.40	10	38.96	19	28.60	28	17.52
1	41.15	10	31.56	19	26.38	28	22.36
2	35.42	11	25.52	20	18.20	29	22.70
2	38.95	11	39.04	20	29.81	29	22.29
2	40.23	11	26.83	20	12.77	29	24.52
3	26.78	12	25.51	21	18.74	30	12.62
3	42.35	12	35.27	21	24.92	30	19.83
3	41.51	12	27.00	21	16.95	30	21.51
4	27.09	13	21.29	22	20.77	31	16.30
4	42.77	13	30.21	22	21.88	31	25.34
4	40.19	13	26.07	22	14.88	31	20.99
5	29.47	14	23.10	23	18.33	32	19.26
5	40.44	14	27.85	23	21.19	32	17.15
5	40.08	14	25.79	23	20.42	32	20.84
6	26.63	15	21.97	24	21.65	33	20.39
6	40.61	15	41.30	24	26.74	33	23.30
6	37.00	15	17.66	24	22.67	33	23.37
7	18.96	16	22.12	25	15.27	34	13.78
7	40.69	16	27.73	25	17.10	34	22.20
7	35.81	16	17.53	25	18.63	34	20.73
8	26.41	17	24.00	26	15.96	35	18.81
8	41.96	17	27.28	26	18.01	35	26.00
8	31.40	17	20.87	26	20.34	35	24.22
9	23.16	18	23.94	27	20.91		
9	41.75	18	27.97	27	24.08		
9	31.49	18	19.15	27	22.79		

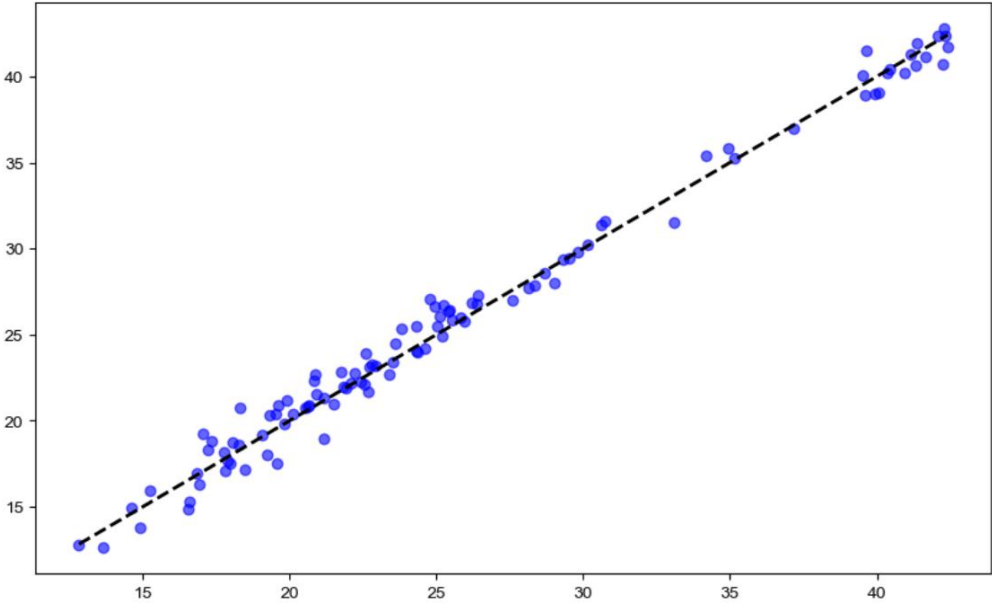


图 4-15 预测值与添加噪声后的预测值对比图

五、问题二：基于过采样方法与卷积神经网络的（MCS,NSS）预测模型

5.1 问题二分析

本题要求对测试中 AP 发送数据选用最多次数的（MCS,NSS）进行建模，通过测试集 test_set_2_2ap 和 test_set_2_3ap（仅提供模型输入信息）预测（MCS, NSS）。

首先，本文选择将**标签 NSS 和 MCS 进行融合**，选择用数字 0-14 对融合后的标签进行编码，为了消除模型对编码大小之分的情况，继而对其采用**独热编码**的操作。接着需要计算一个新的特征 SINR，根据训练集中 sta_from_ap_0_sum_ant_rssi, sta_from_ap_1_sum_ant_rssi 数据结果，对比两个数据长度，对于两者间长的数据采取截断操作，使得两个数据长度保持相同。

然后，需要根据同步传输和异步传输判断每个数据 SINR 特征的计算公式。通过查询相关的文献以及 WLAN 行业的标准，并且分析数据假设环境底噪数据为-99dBm。如果是同步传输则采用 sta_from_ap_0_sum_ant_rssi 数据减去环境底噪-99dBm，如果是异步传输则取 sta_from_ap_0_sum_ant_rssi 数据减去 sta_from_ap_1_sum_ant_rssi 数据，随后分别取同步传输与异步传输的平均值，对同步传输和异步传输数据采取加权平均的方法处理得到最终结果 SINR。在有了标签和特征后分别输入到 **BP 神经网络**和 **CNN 卷积神经网络**模型当中，训练完成加以预测，对比两个模型预测结果的准确率，选择更好的模型预测结果

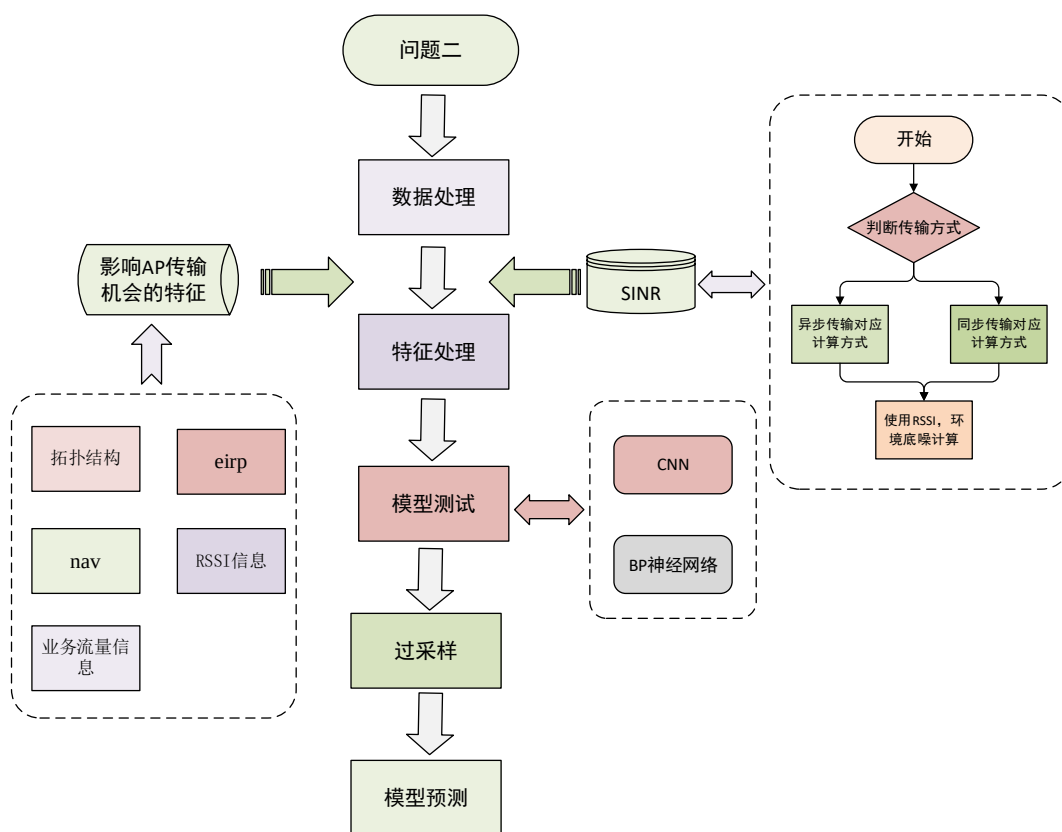


图 5-1 问题二建模流程图

5.2 相关理论

5.2.2 BP 神经网络

BP 神经网络是一种监督学习算法，由输入层、隐藏层和输出层组成，每一层包含多个神经元。其分为两个子过程，分别为工作信号正向传递与误差信号反向传递。BP 神经网络通过误差反向传播算法进行误差校正，最核心的特点就是：信号是前向传播，而误差是反向传播。前向传播过程中，输入信号经过输入层、隐藏层逐层处理，到输出层时，如果结果未到达期望要求，则进入反向传播过程，将误差信号原路返回，修改各层权重。BP 神经网络具体示意图如图 5-2 所示：

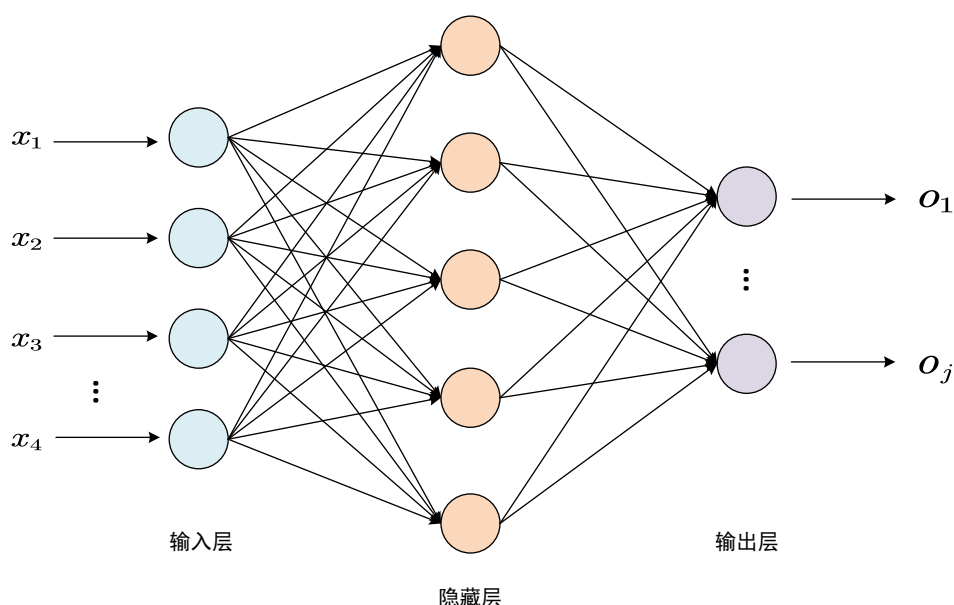


图 5-2 BP 神经网络示意图

5.2.2 CNN 系列

卷积神经网络（CNN, Convolutional Neural Networks）是深度学习中的一种特殊神经网络结构，特别适用于处理具有网格结构的数据，如图像和视频。CNN 的核心思想是通过卷积操作提取数据中的局部特征，逐层抽象出更高层次的表示，广泛应用于图像分类、目标检测、语义分割等任务。CNN 系列包括多个变种和改进模型，涵盖了从经典结构到现代复杂网络的演化。

CNN 的基本结构包括卷积层、池化层和全连接层。卷积层负责提取局部特征，通过卷积核对输入进行滑动并计算特征图；池化层用于下采样，减少特征图的维度并保持重要特征，常见的池化方法有最大池化和平均池化；全连接层则将提取到的特征向量化并用于最终的分类或回归任务。

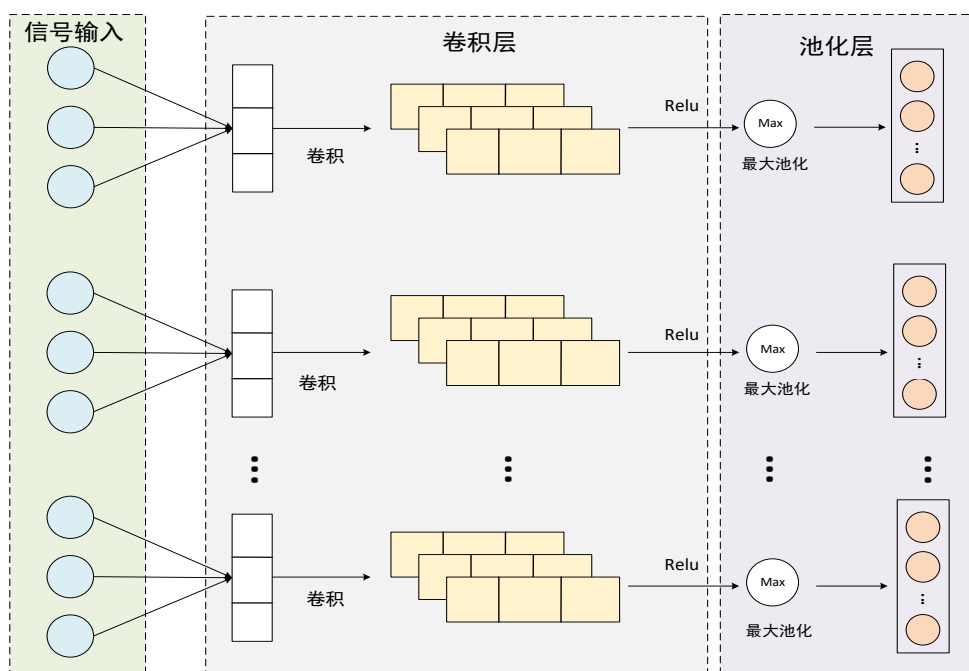


图 5-3 CNN 神经网络示意图

5.2.3 过采样

过采样是一种处理不平衡数据集的技术，特别适用于分类问题中少数类样本数量远少于多数类的情况。通过增加少数类样本的数量，过采样旨在平衡不同类别之间的样本数，从而改善模型对少数类的识别能力。

常见的过采样方法包括随机过采样，即简单地复制少数类中的样本直到达到平衡；合成少数类过采样技术（SMOTE），它不是简单复制现有样本，而是基于少数类样本生成新的合成样本，以减少过拟合的风险并帮助模型更好地理解少数类分布；边界线 SMOTE 专注于那些位于决策边界附近的少数类样本，通过合成更多这类样本来提高决策边界的准确性；ADASYN 则是一种自适应的合成抽样方法，根据少数类样本周围的密度来生成新样本，倾向于为难以学习的区域生成更多样本；聚类后过采样则是先通过聚类算法将少数类样本分成几个簇，然后分别对每个簇进行过采样，以保留数据的内部结构。

使用过采样时需要注意其可能带来的过拟合风险，因为过多复制相同的样本可能导致模型在训练数据上表现良好但在未见数据上的泛化能力较差；此外，生成的新样本会增加数据集大小，这可能会增加训练时间和计算资源的需求。因此，在应用过采样时应结合适当的评估策略，如考虑使用精确度、召回率、F1 分数或 AUC-ROC 曲线等指标，以确保模型的有效性和泛化能力。

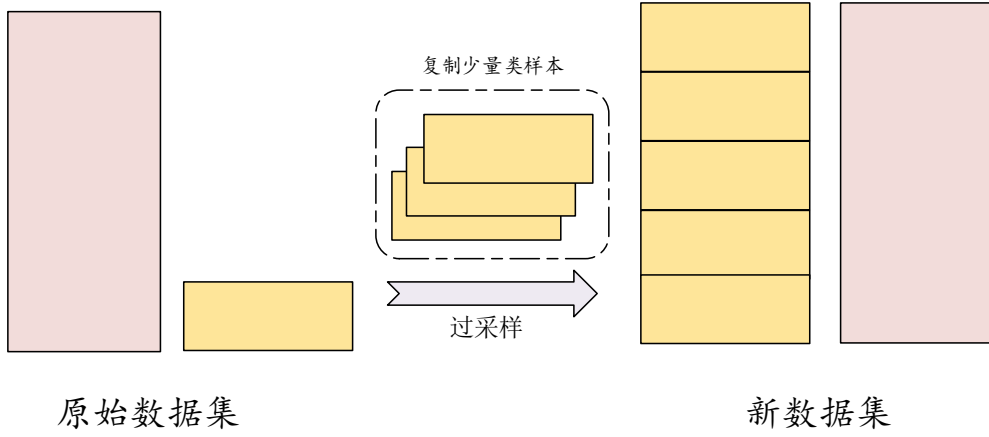


图 5-4 过采样示意图

5.3 模型建立与分析

卷积神经网络一般包括多个卷积层、池化层以及全连接层，本节按照顺序进行建模。卷积操作的公式如下：

$$z_{i,j}^{(l)} = \sum_{m=1}^k \sum_{n=1}^k W_{m,n}^{(l)} \cdot x_{i+m,j+n}^{(l-1)} + b^{(l)} \quad (5.1)$$

在公式(5.1)中， $b^{(l)}$ 表示第 l 层的偏置， $x_{i+m,j+n}^{(l-1)}$ 指的是第 $l-1$ 层的数据， $z_{i,j}^{(l)}$ 是第 l 层卷积输出位置 (i, j) 的值， $W_{m,n}^{(l)}$ 指的是第 l 层的卷积核的权重矩阵。池化层的公式如下：

$$z_{i,j}^{(l)} = \max_{m,n} (x_{i+m,j+n}^{(l-1)}) \quad (5.2)$$

在公式(5.2)中， $x_{i+m,j+n}^{(l-1)}$ 表示池化层的输入值， $z_{i,j}^{(l)}$ 表示池化层的输出层。全连接层可以表示为：

$$z^{(l)} = W^{(l)} \cdot x^{(l-1)} + b^{(l)} \quad (5.3)$$

在公式(5.3)中， $W^{(l)}$ 表示全连接层中的权重矩阵， $b^{(l)}$ 指的是偏置， $x^{(l-1)}$ 指的是上一层的输出， $z^{(l)}$ 表示为全连接层的输出。本团队选取 Relu 激活函数，如公式(5.4)所示。

$$a^{(l)} = \max(0, z^{(l)}) \quad (5.4)$$

BP 神经网络是一种多层前馈神经网络，作用机制是通过反向传播算法调整权重和偏置。

$$z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)} \quad (5.5)$$

$$a^{(l)} = \sigma(z^{(l)}) \quad (5.6)$$

$$L = \frac{1}{2} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \quad (5.7)$$

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \cdot a^{(l-1)T} \quad (5.8)$$

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}} \cdot \sigma'(z^{(l)}) \quad (5.9)$$

公式(5.5)表示 BP 神经网络的前向传播公式，公式(5.6)表示激活函数，公式(5.7)表示使用的均方误差，公式(5.8)表示通过损失函数对于权重进行求导，公式(5.9)表示计算目标层的误差项。

5.3 2ap 情况下模型建立与求解

5.3.1 特征选择与处理

在 2ap 情况下，本小问的特征与问题一相比，本文在问题一的基础上增加了一个特征：SINR。同时将输出变量由 seq_time 改为 (MCS, NSS)。对于输出变量 (MCS, NSS) 的处理，首先将 MCS、NSS 两个变量融合为一个定性变量，如将 MCS 取值为 11，NSS 取值为 2 的变量融合为 (11, 2)；然后将新的定性变量进行数字编码，即用整数 0-13 表示新的定性变量；最后对新的定性变量进行独热编码操作。对于特征 SINR，主要通过 sta_from_ap_0_sum_ant_rssi 以及 sta_from_ap_1_sum_ant_rssi 两个特征解码得到。

具体地，首先判断 sta_from_ap_0_sum_ant_rssi 特征中各数值是属于同步传输还是异步传输。如果属于同步传输，那么 SINR_i 等于 sta_from_ap_0_sum_atn_rssi 减去环境底噪。根据 IEEE802.11 协议，环境底噪统一取值为 -99dBm。如果属于异步传输，那么 SINR_i 等于 sta_from_ap_0_sum_atn_rssi 减去 sta_from_ap_1_sum_atn_rssi。然后将数据按照同步传输与异步传输分为两类，分别计算同步传输以及异步传输条件下 SINR 的平均值 SINR_sync 与 SINR_async。最后统计同步传输以及异步传输占比 sync_percent 与 async_percent，并计算最终的 SINR 取值， $SINR = sync_percent * SINR_sync + async_percent * SINR_async$ 。

5.3.2 建立 BP 神经网络以及 CNN 卷积神经网络模型

得到处理后的特征与标签后，首先对特征进行标准化操作，标准化后的特征服从均值为 0、标准差为 1 的正态分布。然后按 80%: 20% 划分训练数据集以及测试数据集，利用训练数据集分别拟合 BP 神经网络以及 CNN 卷积神经网络模型，而后在测试数据集上进行测试并进行比较。

本文将 BP 神经网络设置为：首先是一个全连接层，即输入层，该层神经元个数为 100，激活函数选择 ReLU；然后添加另一个全连接层，即输出层，该层的神经元个数为 14，激活函数选择 softmax 激活函数，该层神经元个数与类别数量完全相等；最后设置优化器

为 Adam，损失函数为 `categorical_crossentropy`，评估指标为准确率 `acc`。

将 CNN 卷积神经网络设置为：首先是一个一维卷积层，具有 64 个滤波器，卷积核的大小为 2×2 ，使用 ReLU 激活函数；然后是一个最大池化层，尺寸大小也为 2×2 ；接下来是一个 Flatten 层，其作用是将多维输入展平，以便输入到全连接层中；接着是一个全连接层，该层有 100 个神经元，激活函数选择为 ReLU；最后是另一个全连接层，即输出层，其使用 softmax 激活函数，具有与类别数量相等的神经元个数。CNN 中的优化器、损失函数、评估指标与 BP 神经网络中完全相同。

本文将 epoch 设置为 100，batch_size 设置为 32。BP 神经网络以及 CNN 中训练损失和测试损失、训练准确率和测试准确率随 epoch 变化曲线图如下所示：

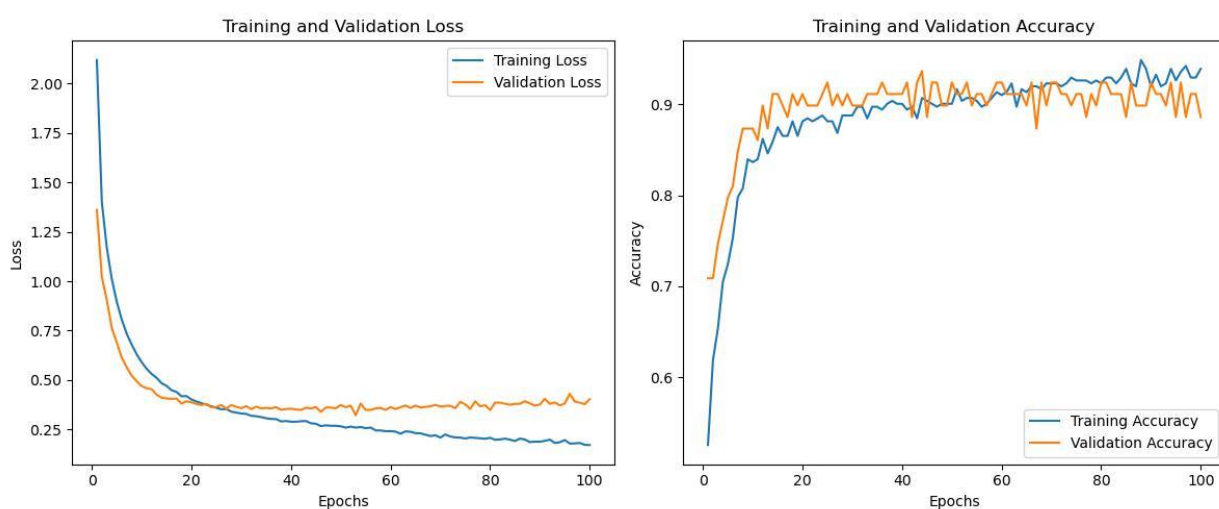


图 5-5 BP 神经网络模型下损失与准确率随 epoch 变化曲线图

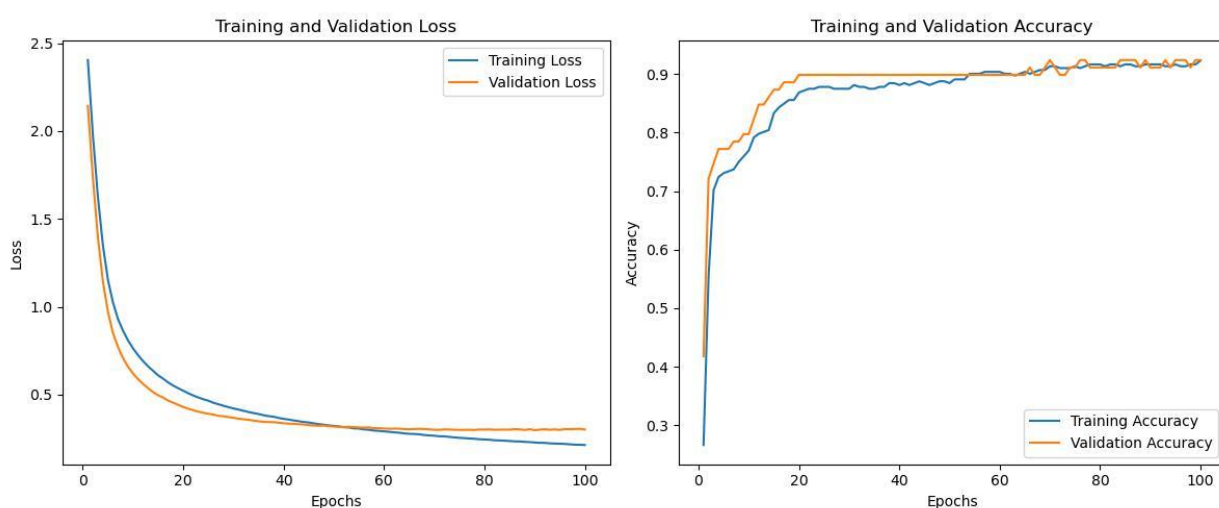


图 5-6 卷积神经网络模型下损失与准确率随 epoch 变化曲线图

可以看出，在 BP 神经网络模型下，随着 epoch 的增加，训练损失最终稳定在 0.15 左右，而测试损失稳定在 0.4 左右，相比训练损失，测试损失偏高。同时随着 epoch 的增加，训练准确率与测试准确率虽刚开始呈现出增长的趋势，但是最终并没有趋于稳定，而是围绕某一特定值上下波动。

在卷积神经网络模型下，随着 epoch 的增加，训练损失与测试损失逐渐下降，最终训练损失稳定在 0.2 左右、测试损失稳定在 0.3 左右。训练准确率与测试准确率随着 epoch 的增加逐渐上升并趋于平稳，最终都稳定在 90% 左右。

最后对比测试集在 BP 神经网络以及卷积神经网络上的准确率 acc，分别为 88.61% 与 92.41%。

综上所述，卷积神经网络 CNN 的预测效果要优于 BP 神经网络。

5.3.3 在 CNN 卷积神经网络中融入过采样方法

在上述建模过程中可以发现，样本各类别数量存在着较为严重的不平衡性。某一类别的数量较少，仅有 1 个样本；而另一类别的数量较多，多达 249 个样本。类别不平衡会导致模型倾向于预测数量较多的类别而忽视数量较少的类别，导致对少数类别的预测准确率较低；同时存在过拟合风险，导致模型在训练过程中对多数类别的样本过拟合，而对少数类别的样本学习不足，导致泛化能力降低。所以需要对类别不平衡这一现象进行处理。

本文采用过采样方法处理类别不平衡问题。通过过采样方法，增加少数类样本的数量，使得各类别数量与最大类别数量保持相等。即过采样后，每一类别都具有 249 个样本，一共存在 3486 个样本。

同样按照 80%: 20% 划分训练数据集与测试数据集，拟合卷积神经网络 CNN 模型，CNN 的模型构造与上一部分中保持一致。由于样本数量的增加，将 epoch 由 100 改为 200。该情况下损失与准确率随 epoch 变化曲线图如下所示：

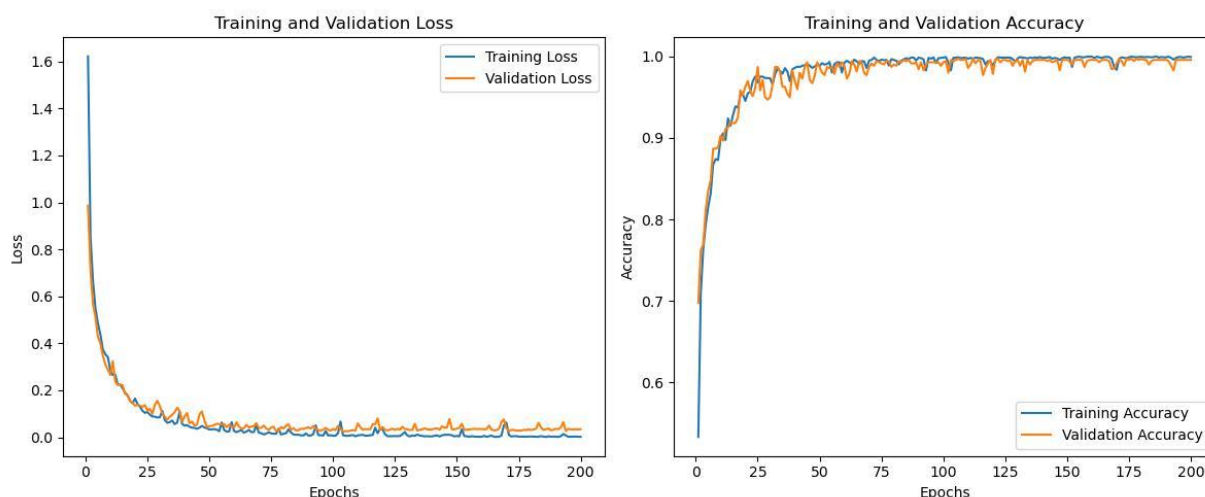


图 5-7 卷积神经网络模型融入过采样方法后损失与准确率随 epoch 变化曲线图

可以看出，随着 epoch 的增加，训练损失与测试损失逐渐下降并趋于平稳，两种损失均趋向于 0；同时训练准确率与测试准确率逐渐上升并且也趋于平稳，准确率逐渐趋向于 1。对比测试数据集的预测值与真实值，可以发现准确率 acc 高达 99.57%，这说明采用过采样方法后所拟合的卷积神经网络 CNN 模型取得了较为好的预测效果。

对于 test_set_2_2ap.csv 文件中的数据，采用相同的方法对其特征进行处理；而后输入至拟合好的融入过采样方法的 CNN 卷积神经网络中，预测其标签 nss_mcs_encoded；最后将 nss_mcs_encoded 转换回 NSS 以及 MCS 特征。最终结果如下表所示：

表 5-1 2ap 情况下 NSS 与 MCS 最终预测结果

test_id	NSS	MCS	test_id	NSS	MCS
1	1	5	17	2	11
1	2	7	17	2	11
2	1	5	18	2	11
2	2	7	18	2	11
3	1	5	19	2	11
3	2	7	19	2	11
4	1	5	20	2	11
4	2	7	20	2	11
5	2	4	21	2	11
5	2	11	21	2	11
6	1	5	22	2	11
6	2	10	22	2	11
7	2	5	23	2	11
7	2	11	23	2	11
8	2	5	24	2	11
8	2	10	24	2	11
9	2	5	25	2	11
9	2	11	25	2	11
10	2	5	26	2	11
10	2	11	26	2	11
11	2	5	27	2	11
11	2	11	27	2	11
12	2	5	28	2	11
12	2	11	28	2	11
13	2	5	29	2	11
13	2	11	29	2	11
14	2	5	30	2	11
14	2	11	30	2	11
15	2	5	31	2	11
15	2	11	31	2	11
16	2	11	32	2	11
16	2	11	32	2	11

5.4 3ap 情况下模型建立与求解

在 3ap 情况下，特征与输出变量的选择与处理与 2ap 情况下或者第一小问中 3ap 情况下几乎相同，主要差异体现在 SINR 特征的计算。在 3ap 情况下，SINR 特征由 `sta_from_ap_0_sum_ant_rssi`、`sta_from_ap_1_sum_ant_rssi` 以及 `sta_from_ap_2_sum_ant_rssi` 三个特征解码得到。如果 `sta_from_ap_0_sum_ant_rssi` 中的数值属于同步传输的范畴，那么 $SINR_i$ 等于 `sta_from_ap_0_sum_atn_rssi` 减去环境底噪，环境底噪的取值同样为 -99dBm；如果 `sta_from_ap_0_sum_ant_rssi` 中的数值属于异步传输的范畴，那么 $SINR_i$ 等于 `sta_from_ap_0_sum_atn_rssi` 减去 `sta_from_ap_1_sum_atn_rssi`，再减去 `sta_from_ap_2_sum_atn_rssi`。其余特征与输出变量的处理以及特征 SINR 的计算与上文中完全相同。

对特征以及输出变量处理后，同样采用过采样方法扩充样本量，增加少数类样本的数量，使得各类别数量与最大类别数量保持相等。然后按照 80%: 20% 划分训练数据集与测试数据集，使用训练数据集拟合卷积神经网络 CNN 模型。CNN 模型的构造也与上文中保持一致，唯一的区别在于将输出层中的神经元个数改为 22，即 3ap 情况下样本的特征数量。设置 epoch 为 300，batch_size 为 32，训练/测试损失与训练/测试准确率随 epoch 变化曲线图如下所示：

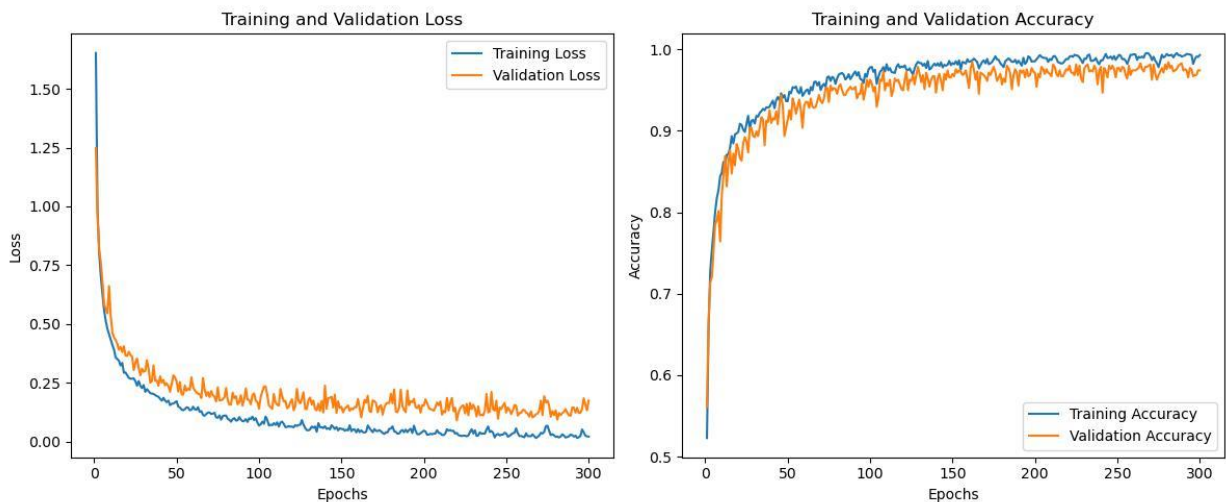


图 5-8 3ap 情况下卷积神经网络模型融入过采样方法后损失与准确率随 epoch 变化曲线图

由上图可以看出，随着 epoch 的增加，训练损失趋向于 0，测试损失稳定在 0.15 左右上下波动；训练准确率趋向于 1，测试准确率也稳定在 0.97 左右。这说明在 3ap 情况下，采用过采样方法后所拟合的卷积神经网络 CNN 模型取得了较为好的预测效果。

对于 `test_set_2_3ap.csv` 文件中的数据，采用相同的方法对其特征进行处理；而后输入至拟合好的融入过采样方法的 CNN 卷积神经网络中，预测其标签 `nss_mcs_encoded`；最后将 `nss_mcs_encoded` 转换回 NSS 以及 MCS 特征。最终预测结果如下表所示：

表 5-2 3ap 情况下 NSS 与 MCS 最终预测结果

test_id	nss	mcs	test_id	nss	mcs
1	2	4	16	2	6
1	2	11	16	2	11
1	1	5	16	2	11
2	2	4	17	2	6
2	2	11	17	2	11
2	2	11	17	2	11
3	2	4	18	2	6
3	2	11	18	2	11
3	2	8	18	2	11
4	2	4	19	2	6
4	2	11	19	2	11
4	2	11	19	2	11
5	2	4	20	2	5
5	2	11	20	2	11
5	2	8	20	2	11
6	2	4	21	2	5
6	2	11	21	2	11
6	2	11	21	2	11
7	2	6	22	2	5
7	2	11	22	2	11
7	2	11	22	2	11
8	2	6	23	2	7
8	2	11	23	2	11
8	2	11	23	2	11
9	2	6	24	2	7
9	2	11	24	2	11
9	2	11	24	2	11
10	2	6	25	2	7
10	2	11	25	2	11
10	2	11	25	2	11
11	2	6	26	2	7
11	2	11	26	2	11
11	2	6	26	2	11
12	2	6	27	2	8
12	2	9	27	2	11
12	2	5	27	2	11
13	2	6	28	2	7
13	2	11	28	2	11
13	2	11	28	2	11
14	2	6	29	2	7
14	2	11	29	2	11
14	2	11	29	2	11
15	2	6			
15	2	11			
15	2	11			

六、问题三：基于双层交叉预测模型的系统吞吐量分析预测

6.1 问题三分析

本题中要求结合一二问的分析，对于系统吞吐量（即 throughput）进行预测分析并且由于 RSSI 信息属于大尺度信息，不足以反应真实的信道状态，本题允许使用真实的（MCS, NSS）信息。

结合题目中所给出的相关信息，吞吐量的计算中仅 MSDU 的总字节数属于有效传输数据，但是 AMSDU 和 AMPDU 聚合的准确评估对吞吐量预测也有影响，由此可以明显判断 AMSDU 和 AMPDU 聚合的准确评估会影响 num_ppdu 和 ppdu_dur (s)，通过观察数据集中 pkt_len 代表的列的意思为数据报文的长度，即说明 AMSDU 和 AMPDU 聚合的准确性会直接影响到数据帧的传输时间和时长。

所以本题中，我们团队认为 throughput 受到 nss, mcs, per, num_ppdu, ppdu_dur (s), seq_time (s), other_air_time (s) 的影响，而这些 num_ppdu 数据等是作为测试过程中的观测值，测试集中并没有标出，所以团队认为该类数据近似受到网络拓扑、业务流量、门限、节点间 RSSI 等信息的影响，本题将建立一个**双层交叉预测模型**，单层中我们使用经过第一问检验的**随机森林模型**进行数据帧统计信息的预测，通过验证之后，我们采用**深度残差神经网络**进行数据帧统计信息和部分门限等信息来进行系统吞吐量的判断。

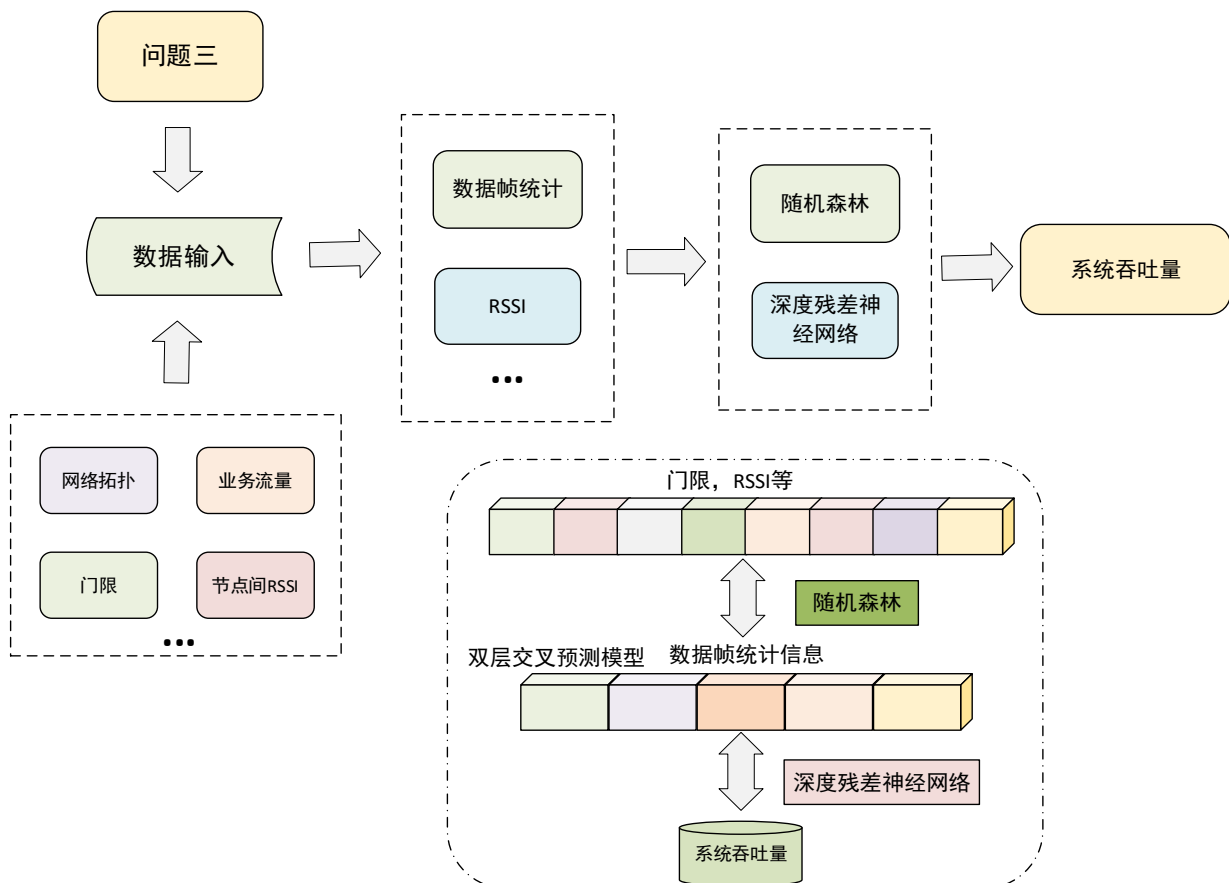


图 6-1 问题三建模流程图

6.2 深度残差神经网络相关理论

深度残差神经网络通过加入残差连接解决了传统神经网络中梯度消失和梯度爆炸的问题。具体的公式如下：

$$Y = \mathcal{F}(X, \{W_i\}) \quad (6.1)$$

$$Y = \mathcal{F}(X, \{W_i\}) + X \quad (6.2)$$

在公式(6.1)和公式(6.2)中， X 为模型的输入变量， $\mathcal{F}(X, \{W_i\})$ 是指经过卷积层之后的变换，而加入的 $+X$ 为残差连接，意味着直接将输入加到输出上。激活函数如(6.3)所示。

$$a^{(l)} = \max(0, z^{(l)}) \quad (6.3)$$

$$Y^{(l)} = \mathcal{F}(X^{(l)}, \{W_i^{(l)}\}) + X^{(l)} \quad (6.4)$$

$$Y = \mathcal{F}_L(\mathcal{F}_{L-1}(\dots \mathcal{F}_2(\mathcal{F}_1(X)))) \quad (6.5)$$

公式(6.4)表示 L 层的深度残差神经网络，并且包含的若干残差块的输出。公式(6.5)表示整个深度残差神经网络的输出，每一层都有残差连接。我们使用均方误差作为损失函数，如公式(6.6)所示。

$$L = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \quad (6.6)$$

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \cdot a^{(l-1)} \quad (6.7)$$

公式(6.7)中表示模型通过反向传播算法，更新每一层的权重和偏置， $\delta^{(l)}$ 表示第 l 层的误差。

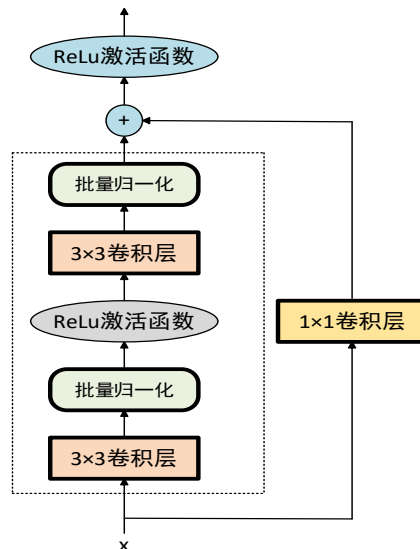


图 6-2 深度残差神经网络示意图

6.3 2ap 情况下模型建立与求解

6.3.1 特征选取与处理

第一阶段的特征选取与问题一相同，输出变量包括 num_ppdu，一个数据帧的聚合个数；ppdu_dur，一个数据帧的时长；seq_time，AP 发送数据的帧序列的总时长以及 other_air_time，AP 除发送数据帧外，发送管理帧等其他帧和接收所占用的时间。第二阶段的特征除了第一阶段的四个输出变量外，还包括（MCS，NSS）以及丢包率 per；第二阶段的输出变量为 throughput，即每个 AP 的吞吐量。除（MCS，NSS）外，其余变量均为定量变量。对于（MCS，NSS），根据下表将其转为定量变量 PHYRate。

表 6-1 （MCS，NSS）与 PHYRate 对应表

MCS	0	1	2	3	4	5
NSS1	8.6	17.2	25.8	34.4	51.6	68.8
NSS2	17.2	34.4	51.6	68.8	103.2	137.6
MCS	6	7	8	9	10	11
NSS1	77.4	86.0	103.2	114.7	129.0	143.4
NSS2	154.9	172.1	206.5	229.4	258.1	286.8

6.3.2 第一阶段——预测中间变量

根据第一问的结论，本文使用随机森林模型来预测 num_ppdu、ppdu_dur、seq_time 以及 other_air_time 四个目标变量。与第一问中预测 AP 发送数据的帧序列的总时长 seq_time 相似，首先按 80%：20%的比例划分训练数据集与测试数据集；然后使用训练数据集拟合相对应的随机森林模型，并在测试数据集上进行测试，计算预测值与真实值之间的残差平方和 MSE 以及拟合优度 R^2 ，最终结果如表 6-2 所示：

表 6-2 各变量随机森林拟合效果

输出变量	残差平方和 MSE	拟合优度 R^2
num_ppdu	1.9743	0.7357
ppdu_dur	0.7804	0.6123
seq_time	6.9639	0.9378
other air time	7.9694	0.9144

可以看出，对于预测 AP 发送数据的帧序列的总时长 seq_time 以及 AP 除发送数据帧外，发送管理帧等其他帧和接收所占用的时间 other_air_time，随机森林有着较好的预测效果，相应模型的拟合优度可以达到 90%以上。对于数据帧的聚合个数 num_ppdu 以及数据帧的时长 ppdu_dur 而言，随机森林的效果并不显著，拟合优度 R^2 大约在 70%与 60%左右。

绘制各变量与各特征间的热力图，如图 6-3 所示。通过图 6-3，可以看出与变量相关性程度最大的特征以及各特征之间的相互影响程度。

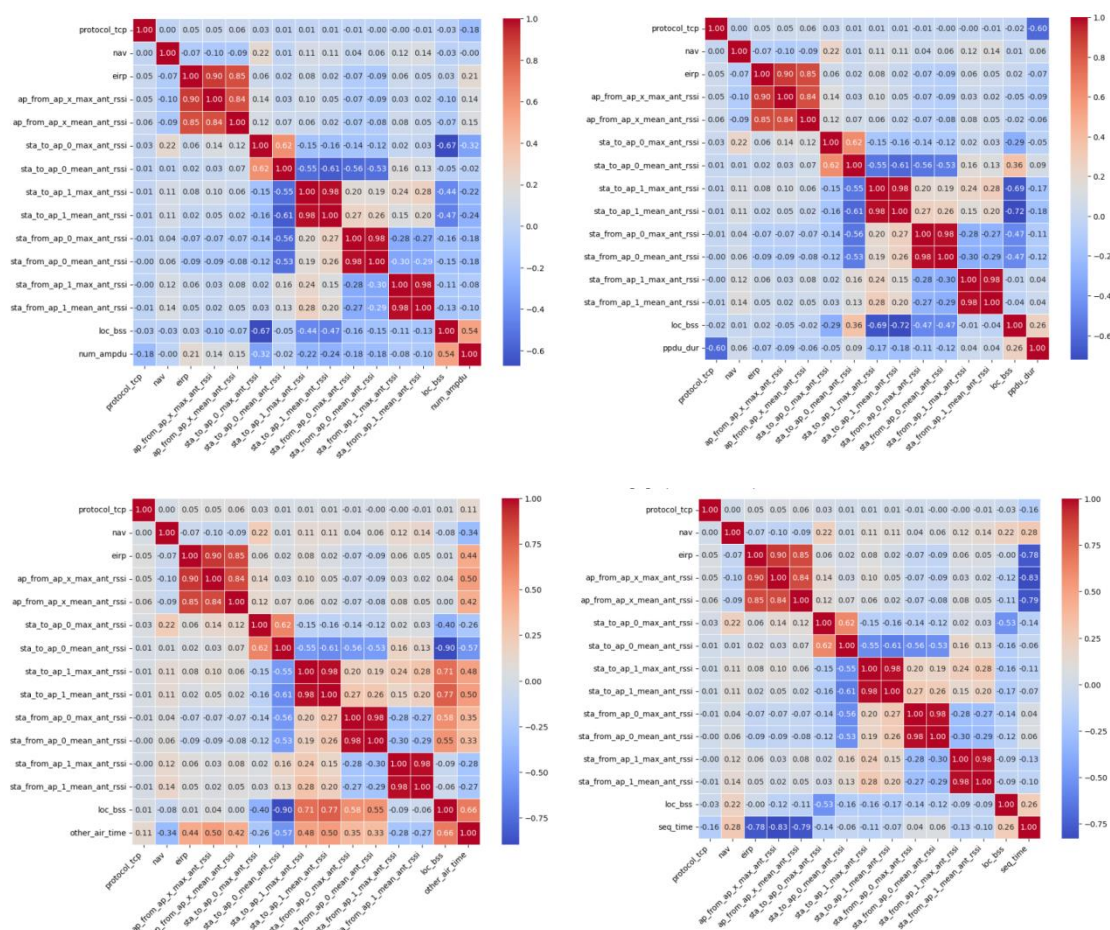


图 6-3 各变量与各特征间热力图

对于 test_set_1_2ap.csv 文件中的数据，对其特征进行处理后，使用上述拟合好的随机森林模型分别预测各样本的 num_ppdu、ppdu_dur、seq_time 以及 other_air_time，部分预测数据如下表所示：

表 6-3 num_ppdu 等四个变量预测结果

test_id	num_ppdu	ppdu_dur	other air time	seq_time
1	18.45	0.0036	12.48	48.82
1	18.10	0.0034	5.14	49.40
2	18.25	0.0041	8.99	50.74
2	18.44	0.0041	3.84	50.39
.....				
39	18.14	0.0035	20.71	27.67
39	18.67	0.0036	10.67	29.88
40	16.92	0.0040	17.74	28.89
40	17.73	0.0040	9.39	29.82

6.3.3 第二阶段——预测网络吞吐量

在上一阶段中，已经得到 num_ppdu、ppdu_dur、seq_time 以及 other_air_time 四个变量的预测结果。在该阶段中，将这四个变量作为输入变量，同时 per 和 PHYRate 两个已知的变量也作为输入变量；输出变量为网络吞吐量 throughput。同样按照 80%：20%的比例

划分训练数据集以及测试数据集，使用训练数据集建立深度残差神经网络模型。模型的残差平方和 MSE 以及拟合优度 R^2 随 epoch 变化的曲线图如下所示：

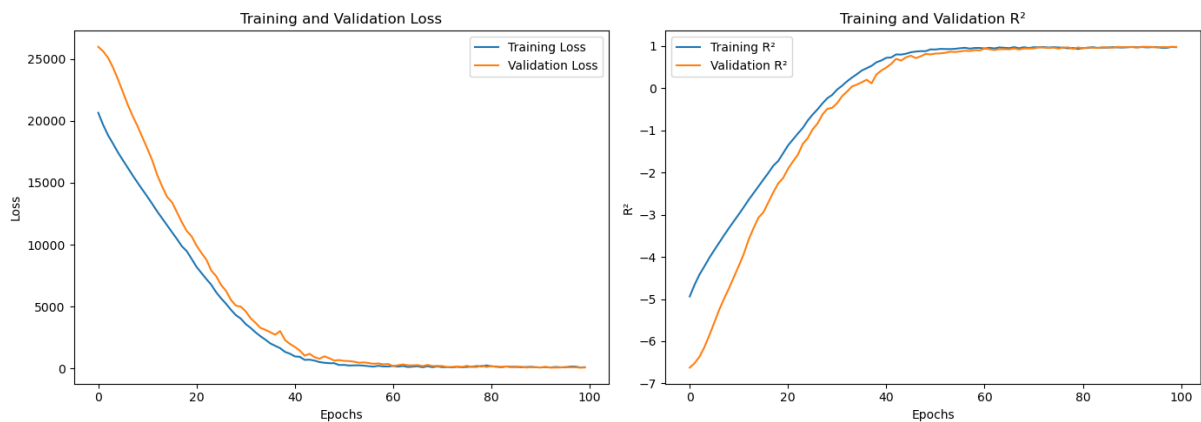


图 6-4 损失与拟合优度随 epoch 变化曲线图

随着 epoch 的增加，训练损失与测试损失均下降至 0 左右同时保持稳定；而训练数据拟合优度与测试数据拟合优度均上升至 1 左右同时也趋向于稳定。这可以看出，本文所拟合的深度残差神经网络模型有着较好的预测效果。

将 test_set_1_2ap.csv 文件中的数据输入至拟合好的深度残差神经网络模型中，可以预测出最终的网络吞吐量，具体结果如下表所示：

表 6-4 网络吞吐量最终预测结果

test_id	throughput	test_id	throughput
1	107.78	21	139.53
1	114.45	21	120.05
2	136.32	22	157.20
2	139.55	22	169.53
3	115.93	23	118.65
3	117.41	23	129.30
4	135.12	24	130.80
4	138.76	24	118.99
5	115.63	25	141.46
5	117.23	25	87.18
6	135.29	26	153.26
6	132.87	26	111.02
7	112.89	27	118.49
7	131.79	27	83.03
8	131.15	28	130.17
8	130.51	28	96.49
9	128.92	29	122.89
9	114.96	29	83.25
10	155.44	30	147.54
10	159.88	30	81.49
11	132.47	31	121.24

11	139.90	31	77.01
12	152.67	32	134.72
12	128.21	32	75.64
13	124.09	33	125.88
13	156.83	33	74.00
14	165.97	34	111.52
14	179.30	34	76.82
15	139.93	35	123.82
15	150.36	35	75.68
16	162.11	36	109.12
16	173.63	36	85.99
17	140.40	37	126.56
17	146.97	37	91.69
18	154.37	38	106.22
18	181.91	38	87.67
19	110.20	39	119.30
19	154.30	39	84.64
20	121.54	40	138.29
20	175.51	40	76.63

6.4 3ap 情况下模型建立与求解

在 3ap 情况下，网络吞吐量的预测与 2ap 情况下几乎相同，同样需要先预测出 num_pdu、ppdu_dur、seq_time 以及 other_air_time 四个中间变量的数值，然后再预测最终网络吞吐量的大小。

划分完训练数据集与测试数据集后，拟合相应的随机森林模型，使用残差平方和 MSE 以及拟合优度 R^2 来评估所拟合的随机森林模型的优劣，具体结果如表 6-5 所示：

表 6-5 3ap 情况下各变量随机森林拟合效果

输出变量	残差平方和 MSE	拟合优度 R^2
num_ppdu	2.0231	0.7379
ppdu_dur	1.4902×10^{-7}	0.7399
seq_time	37.4314	0.7829
other_air_time	60.9694	0.7144

可以看出，在四个变量上所拟合的随机森林模型的拟合优度均不足 80%，说明随机森林的预测效果并不显著。

同样绘制各变量与各特征间的热力图，如图 6-5 所示。通过图 6-5，可以看出与变量相关性程度最大的特征以及各特征之间的相互影响程度。

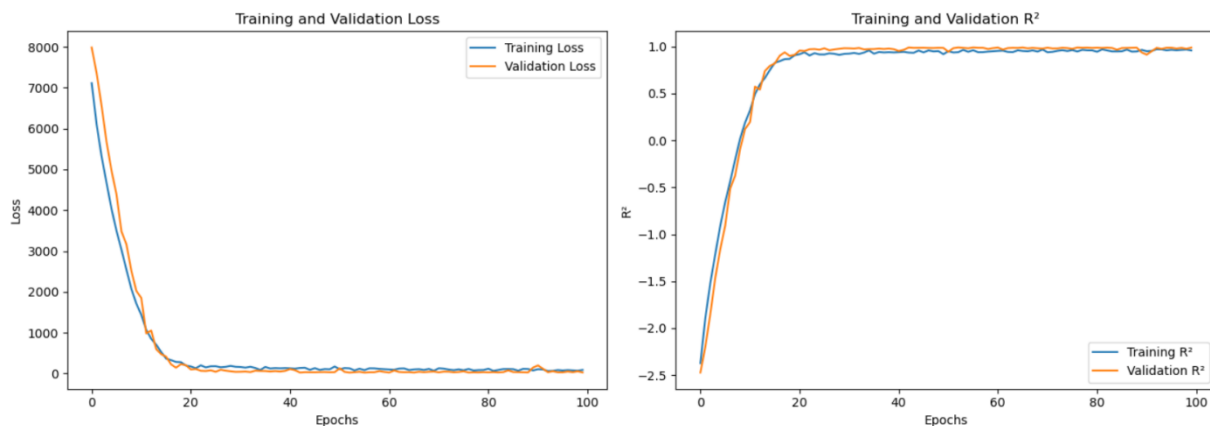


图 6-6 3ap 情况下损失与拟合优度随 epoch 变化曲线图

随着 epoch 的增加，训练损失与测试损失均较早地收敛到 0 左右；而训练数据拟合优度与测试数据拟合优度也均较早地收敛到 1 左右。从这两幅变化趋势图可以看出，本文所拟合的深度残差神经网络模型有着较好的预测性能。

将 test_set_1_3ap.csv 文件中的数据输入至拟合好的深度残差神经网络模型中，可以预测出最终的网络吞吐量，具体结果如下表所示：

表 6-7 3ap 情况下网络吞吐量最终预测结果

test_id	throughput	test_id	throughput
1	23.62	18	61.64
1	41.19	19	57.77
1	48.40	19	53.17
2	29.23	19	84.15
2	40.13	20	39.52
2	44.88	20	93.22
3	19.97	20	49.99
3	41.65	21	40.93
3	45.37	21	51.11
4	24.87	21	45.45
4	41.75	22	54.00
4	42.31	22	69.77
5	36.64	22	56.52
5	41.49	23	59.22
5	45.60	23	70.46
6	28.47	23	60.71
6	43.28	24	48.71
6	43.64	24	60.30
7	46.18	24	64.41
7	43.55	25	53.71
8	62.51	25	69.27
8	52.15	25	54.38
8	35.64	26	56.23
9	51.20	26	72.39

9	42.97	26	65.35
9	84.36	27	36.07
10	51.90	27	62.79
10	68.65	27	54.58
10	75.86	28	54.97
11	46.79	28	71.47
11	41.55	28	57.93
11	83.41	29	35.16
12	59.08	29	31.13
12	52.84	29	65.83
12	40.60	30	57.10
13	53.61	30	69.05
13	52.80	30	67.26
13	59.49	31	36.51
14	59.30	31	61.90
14	52.13	31	59.33
14	59.01	32	61.56
15	56.19	32	65.83
15	42.73	32	67.50
15	59.33	33	44.98
16	56.79	33	59.83
16	59.50	33	56.95
16	58.86	34	54.71
17	55.22	34	74.56
17	60.42	34	62.06
17	51.22	35	34.77
18	53.43	35	33.15
18	50.76	35	68.37

七、模型评价与改进

7.1 模型的优点

针对问题一而言，本文使用了随机森林模型来预测 AP 发送数据帧序列的总时长（seq_time），并结合 DBSCAN 算法对同步传输和异步传输进行区分，并且添加了一部分扰动来描述和处理偶发性的随机因素。我们使用了 GBDT 和随机森林两种模型进行了数据分析和比较，发现随机森林对于该数据的拟合优度更好，通过 DBSCAN 分类处理和特征工程之后的数据训练出来的模型有良好的效果。

针对问题二而言，我们首先分情况计算了 SINR 的信息，将 SINR 以及网络拓扑、业务流量、门限、节点间 RSSI 等信息进行特征工程处理之后，我们选择使用了卷积神经网络和 BP 神经网络进行对比分析，由于卷积神经网络（CNN）则在局部特征提取、参数共享、降维能力表现良好，以及在训练集上的数据表现，选择卷积神经网络作为我们的模型进行预测。

针对问题三而言，本文使用了一种双层交叉预测模型，使用随机森林和深度残差神经网络（ResNet）结合的方式来预测 WLAN 的吞吐量。深度残差神经网络（ResNet）通过引入残差块解决了深层网络梯度消失/爆炸的问题，使得网络可以更深，从而提高模型的表达能力，有更好的拟合效果，而随机森林在问题一中对于本文中初步的特征处理有良好的表现，两者交叉结合对于模型的预测有更好的效果。

7.2 模型的缺点

（1）随机森林模型可以通过多棵树捕捉部分特征交互，但是如果特征间存在复杂的非线性交互关系，随机森林模型可能无法处理和捕捉。

（2）卷积神经网络（CNN）的效果高度依赖于输入数据的质量，如果数据存在噪声或不一致性，可能会严重影响模型性能。

（3）深度残差神经网络（ResNet）一般需要较大的计算资源，特别是在处理大规模数据集时。

（4）采用的双层交叉预测模型由于结构的复杂性，其计算效率会相对低一些。

参考文献

- [1] Nazabal J A, Iturri P L, Azpilicueta L, et al. Performance analysis of IEEE 802.15. 4 compliant wireless devices for heterogeneous indoor home automation environments[J]. International Journal of Antennas and Propagation, 2012, 2012(1): 176383.
- [2] Radhakrishna K S, Lee Y S, You K Y, et al. Study of Obstacles Effect on Mobile Network and WLAN Signal Strength[J]. International Journal of Electronics and Telecommunications, 2023, 69.
- [3] Qureshi I A, Asghar S. A systematic review of the IEEE-802.11 standard's enhancements and limitations[J]. Wireless Personal Communications, 2023, 131(4): 2539-2572.
- [4] 宋向东. 802.11ac wave 2 规范发布 Wi-Fi 连接速度显著提升 [J]. 通信世界, 2016, (24): 47. DOI:10.13571/j.cnki.cww.2016.24.028.
- [5] 徐勇军,田秦语,陈前斌,等. RIS 辅助认知反向散射通信网络吞吐量最大化算法 [J]. 中国科学:信息科学, 2024, 54 (08): 1970-1982.
- [6] 李爽. 参数估计与假设检验在软件性能测试中的应用——以随机变量的角度描述和处理网络性能指标 [J]. 科学技术创新, 2024, (13): 62-68.
- [7] 曹莹,陈旭,张跃博,等. 基于 ARIMA-BP 神经网络组合预测的港口吞吐量预测 [J]. 物流技术, 2023, 42 (12): 84-91.
- [8] 何剑,张广兴,谢高岗.基于多关联模型的软件定义高密 WLAN EDCA 机制的吞吐量优化 [J].高技术通讯,2018,28(01):15-21.
- [9] 陈禹,赵静雅,朱庆华,等. 基于强化学习的 WLAN 节点自适应调整技术 [J]. 计算机工程与设计, 2019, 40 (09): 2422-2427. DOI:10.16208/j.issn1000-7024.2019.09.005.

附录

附录 A：各小题预测结果

1、2ap 情况下 seq_time 预测结果

test_id	seq_time	test_id	seq_time	test_id	seq_time
1	48.28	15	46.33	28	28.92
1	47.57	15	48.85	28	38.71
2	50.92	16	48.81	29	31.05
2	50.55	16	50.27	29	31.74
3	47.92	17	43.46	30	31.80
3	49.43	17	46.33	30	34.16
4	50.97	18	44.35	31	32.19
4	50.45	18	49.64	31	31.82
5	48.10	19	35.25	32	31.02
5	48.05	19	48.23	32	29.95
6	50.80	20	37.16	33	28.16
6	51.08	20	51.35	33	26.77
7	47.69	21	36.71	34	25.25
7	47.88	21	45.43	34	29.04
8	51.32	22	37.29	35	26.35
8	53.08	22	47.49	35	24.70
9	47.01	23	31.61	36	28.64
9	49.37	23	45.05	36	31.10
10	51.73	24	32.27	37	30.04
10	49.33	24	47.93	37	29.82
11	46.50	25	32.50	38	27.83
11	48.13	25	38.56	38	27.98
12	48.64	26	36.20	39	26.83
12	50.25	26	40.47	39	29.05
13	45.85	26	36.20	40	28.50
13	49.69	26	40.47	40	28.54
14	48.13	27	27.40		
14	50.44	27	39.43		

2、2ap 情况下（MCS，NSS）预测结果

test_id	NSS	MCS	test_id	NSS	MCS
1	1	5	17	2	11
1	2	7	17	2	11
2	1	5	18	2	11
2	2	7	18	2	11
3	1	5	19	2	11
3	2	7	19	2	11
4	1	5	20	2	11
4	2	7	20	2	11
5	2	4	21	2	11
5	2	11	21	2	11
6	1	5	22	2	11
6	2	10	22	2	11
7	2	5	23	2	11
7	2	11	23	2	11
8	2	5	24	2	11
8	2	10	24	2	11
9	2	5	25	2	11
9	2	11	25	2	11
10	2	5	26	2	11
10	2	11	26	2	11
11	2	5	27	2	11
11	2	11	27	2	11
12	2	5	28	2	11
12	2	11	28	2	11
13	2	5	29	2	11
13	2	11	29	2	11
14	2	5	30	2	11
14	2	11	30	2	11
15	2	5	31	2	11
15	2	11	31	2	11
16	2	11	32	2	11
16	2	11	32	2	11

3、2ap 情况下吞吐量 throughput 预测结果

test_id	throughput	test_id	throughput
1	107.78	21	139.53
1	114.45	21	120.05
2	136.32	22	157.20
2	139.55	22	169.53
3	115.93	23	118.65
3	117.41	23	129.30
4	135.12	24	130.80
4	138.76	24	118.99
5	115.63	25	141.46
5	117.23	25	87.18
6	135.29	26	153.26
6	132.87	26	111.02
7	112.89	27	118.49
7	131.79	27	83.03
8	131.15	28	130.17
8	130.51	28	96.49
9	128.92	29	122.89
9	114.96	29	83.25
10	155.44	30	147.54
10	159.88	30	81.49
11	132.47	31	121.24
11	139.90	31	77.01
12	152.67	32	134.72
12	128.21	32	75.64
13	124.09	33	125.88
13	156.83	33	74.00
14	165.97	34	111.52
14	179.30	34	76.82
15	139.93	35	123.82
15	150.36	35	75.68
16	162.11	36	109.12
16	173.63	36	85.99
17	140.40	37	126.56
17	146.97	37	91.69
18	154.37	38	106.22
18	181.91	38	87.67
19	110.20	39	119.30
19	154.30	39	84.64
20	121.54	40	138.29
20	175.51	40	76.63

4、3ap 情况下 seq_time 预测结果

test_id	seq_time	test_id	seq_time	test_id	seq_time	test_id	seq_time
1	29.34	10	25.84	19	22.85	28	14.93
1	42.40	10	38.96	19	28.60	28	17.52
1	41.15	10	31.56	19	26.38	28	22.36
2	35.42	11	25.52	20	18.20	29	22.70
2	38.95	11	39.04	20	29.81	29	22.29
2	40.23	11	26.83	20	12.77	29	24.52
3	26.78	12	25.51	21	18.74	30	12.62
3	42.35	12	35.27	21	24.92	30	19.83
3	41.51	12	27.00	21	16.95	30	21.51
4	27.09	13	21.29	22	20.77	31	16.30
4	42.77	13	30.21	22	21.88	31	25.34
4	40.19	13	26.07	22	14.88	31	20.99
5	29.47	14	23.10	23	18.33	32	19.26
5	40.44	14	27.85	23	21.19	32	17.15
5	40.08	14	25.79	23	20.42	32	20.84
6	26.63	15	21.97	24	21.65	33	20.39
6	40.61	15	41.30	24	26.74	33	23.30
6	37.00	15	17.66	24	22.67	33	23.37
7	18.96	16	22.12	25	15.27	34	13.78
7	40.69	16	27.73	25	17.10	34	22.20
7	35.81	16	17.53	25	18.63	34	20.73
8	26.41	17	24.00	26	15.96	35	18.81
8	41.96	17	27.28	26	18.01	35	26.00
8	31.40	17	20.87	26	20.34	35	24.22
9	23.16	18	23.94	27	20.91		
9	41.75	18	27.97	27	24.08		
9	31.49	18	19.15	27	22.79		

5、3ap 情况下（MCS，NSS）预测结果

test_id	nss	mcs	test_id	nss	mcs
1	2	4	16	2	6
1	2	11	16	2	11
1	1	5	16	2	11
2	2	4	17	2	6
2	2	11	17	2	11
2	2	11	17	2	11
3	2	4	18	2	6
3	2	11	18	2	11
3	2	8	18	2	11
4	2	4	19	2	6
4	2	11	19	2	11
4	2	11	19	2	11
5	2	4	20	2	5
5	2	11	20	2	11
5	2	8	20	2	11
6	2	4	21	2	5
6	2	11	21	2	11
6	2	11	21	2	11
7	2	6	22	2	5
7	2	11	22	2	11
7	2	11	22	2	11
8	2	6	23	2	7
8	2	11	23	2	11
8	2	11	23	2	11
9	2	6	24	2	7
9	2	11	24	2	11
9	2	11	24	2	11
10	2	6	25	2	7
10	2	11	25	2	11
10	2	11	25	2	11
11	2	6	26	2	7
11	2	11	26	2	11
11	2	6	26	2	11
12	2	6	27	2	8
12	2	9	27	2	11
12	2	5	27	2	11
13	2	6	28	2	7
13	2	11	28	2	11
13	2	11	28	2	11
14	2	6	29	2	7
14	2	11	29	2	11
14	2	11	29	2	11
15	2	6			
15	2	11			
15	2	11			

6、3ap 情况下吞吐量 throughput 预测结果

test_id	throughput	test_id	throughput
1	23.62	18	61.64
1	41.19	19	57.77
1	48.40	19	53.17
2	29.23	19	84.15
2	40.13	20	39.52
2	44.88	20	93.22
3	19.97	20	49.99
3	41.65	21	40.93
3	45.37	21	51.11
4	24.87	21	45.45
4	41.75	22	54.00
4	42.31	22	69.77
5	36.64	22	56.52
5	41.49	23	59.22
5	45.60	23	70.46
6	28.47	23	60.71
6	43.28	24	48.71
6	43.64	24	60.30
7	46.18	24	64.41
7	43.55	25	53.71
8	62.51	25	69.27
8	52.15	25	54.38
8	35.64	26	56.23
9	51.20	26	72.39
9	42.97	26	65.35
9	84.36	27	36.07
10	51.90	27	62.79
10	68.65	27	54.58
10	75.86	28	54.97
11	46.79	28	71.47
11	41.55	28	57.93
11	83.41	29	35.16
12	59.08	29	31.13
12	52.84	29	65.83
12	40.60	30	57.10
13	53.61	30	69.05
13	52.80	30	67.26
13	59.49	31	36.51
14	59.30	31	61.90
14	52.13	31	59.33
14	59.01	32	61.56
15	56.19	32	65.83

15	42.73	32	67.50
15	59.33	33	44.98
16	56.79	33	59.83
16	59.50	33	56.95
16	58.86	34	54.71
17	55.22	34	74.56
17	60.42	34	62.06
17	51.22	35	34.77
18	53.43	35	33.15
18	50.76	35	68.37

附录 B：代码

Question1：拟合随机森林及 GBDT 模型并进行比较，同时对特征重要性进行排序。

```
1. import numpy as np
2. import pandas as pd
3. import ast
4. import matplotlib.pyplot as plt
5. import seaborn as sns
6. import matplotlib
7. from sklearn.preprocessing import OneHotEncoder
8. from sklearn.ensemble import RandomForestRegressor
9. from sklearn.ensemble import GradientBoostingRegressor
10. from sklearn.metrics import mean_squared_error, r2_score
11. from sklearn.model_selection import train_test_split
12. from sklearn.tree import plot_tree
13. from itertools import chain
14. from category_encoders import TargetEncoder
15.
16. df_data = pd.read_excel('2ap_train.xlsx')
17. pd.set_option('display.max_columns', None)
18. df_data
19.
20. def convert_to_list(element):
21.     try:
22.         return ast.literal_eval(element)
23.     except (ValueError, SyntaxError):
24.         return element
25.
26. for i in range(4,14):
27.     df_data.iloc[:,i] = df_data.iloc[:,i].apply(convert_to_list)
28.
29. def calc_proportion_above_threshold(rssi_list, threshold1=-62, threshold2=-82):
30.     if isinstance(rssi_list, list):
31.         total_count = len(rssi_list)
32.         above_threshold_count = sum(1 for rssi in rssi_list if (rssi > threshold2 and rssi < threshold
1))
```

```

33.         return above_threshold_count / total_count if total_count > 0 else 0
34.     return 0
35.
36. for i in range(4,14):
37.     df_data.iloc[:,i] = df_data.iloc[:,i].apply(calc_proportion_above_threshold)
38.
39. df_data
40.
41. # 定义数据字典
42. lookup_table = {
43.     1: [8.6, 17.2, 25.8, 34.4, 51.6, 68.8, 77.4, 86.0, 103.2, 114.7, 129.0, 143.4],
44.     2: [17.2, 34.4, 51.6, 68.8, 103.2, 137.6, 154.9, 172.1, 206.5, 229.4, 258.1, 286.8]
45. }
46.
47. # 定义查找函数
48. def lookup_value(row):
49.     return lookup_table[row['nss']][row['mcs']]
50.
51. # 应用查找
52. df_data['PHYRate'] = df_data[['nss','mcs']].apply(lookup_value, axis=1)
53.
54. columns_to_drop = ['nss', 'mcs']
55. df_data = df_data.drop(columns=columns_to_drop)
56.
57. df_data.to_excel('Q3_2ap_train.xlsx',index = False)
58.
59. # 初始化目标编码器
60. encoder = TargetEncoder()
61.
62. encoded_loc_bss_id = encoder.fit_transform(df_data[['loc_bss_id']], df_data[['seq_time']])
63. df_encoded = pd.DataFrame(encoded_loc_bss_id, columns=encoder.get_feature_names_out(['loc_bss_id']))
64. df_data_1 = pd.concat([df_data, df_encoded], axis=1)
65.
66. df_data_1.columns.values[-1] = 'loc_bss'

```

```

67.
68. df_data_1 = df_data_1.drop(df_data_1.columns[1], axis=1)
69.
70. df_data_1
71.
72. columns_to_drop = ['num_ampdu', 'ppdu_dur', 'other_air_time']
73. df_data_1_corr = df_data_1.drop(columns=columns_to_drop)
74. corr_matrix = df_data_1_corr.corr()
75. plt.figure(figsize=(10, 8))
76. sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
77. plt.xticks(rotation=45, ha='right')
78. plt.show()
79.
80. # df_data_1.to_excel('output2.xlsx', index=False)
81.
82. columns_to_drop = ['num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time']
83. X = df_data_1.drop(columns=columns_to_drop)
84. y = df_data_1['seq_time']
85.
86. # 划分训练集和测试集
87. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
88.
89. # 初始化随机森林回归模型
90. rf = RandomForestRegressor(n_estimators=100, random_state=42)
91.
92. # 训练模型
93. rf.fit(X_train, y_train)
94.
95. estimator = rf.estimators_[50]
96.
97. plt.figure(figsize=(40, 40))
98. plot_tree(estimator, feature_names=X.columns, filled=True, rounded=True, fontsize=10)
99. plt.show()
100.

```

```

101. # 进行预测
102. y_pred = rf.predict(X_test)
103.
104. # 评估模型
105. mse = mean_squared_error(y_test, y_pred)
106. r2 = r2_score(y_test, y_pred)
107.
108. mse, r2
109.
110. # 随机森林特征重要性条形图
111. feature_importances = rf.feature_importances_
112. feature_names = X_train.columns
113.
114. feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': feature_importances})
115. feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
116. feature_importance_df = feature_importance_df
117.
118. plt.figure(figsize=(10, 6))
119. sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
120. plt.savefig('rf_import.png')
121. plt.show()
122.
123. gbdt = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
124.
125. gbdt.fit(X_train, y_train)
126.
127. # 对测试集进行预测
128. y_pred = gbdt.predict(X_test)
129.
130. # 计算均方误差 (MSE)
131. mse = mean_squared_error(y_test, y_pred)
132. r2 = r2_score(y_test, y_pred)
133.
134. mse, r2

```


135.

```
136.importances = gbd_t.feature_importances_
```

137.

```
138.feature_names = X_train.columns
```

```
139.feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
```

140.

```
141.feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
```

```
142.feature_importance_df = feature_importance_df
```

143.

```
144.plt.figure(figsize=(10, 6))
```

```
145.sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
```

```
146.plt.savefig('gbd_t_import.png')
```

```
147.plt.show()
```

148.

```
149.df_testdata = pd.read_excel('2ap_test.xlsx')
```

```
150.df_testdata = df_testdata.drop(df_testdata.columns[[5, 7]], axis=1)
```

151.

```
152.for i in range(4,14):
```

```
153.    df_testdata.iloc[:,i] = df_testdata.iloc[:,i].apply(convert_to_list)
```

154.

```
155.for i in range(4,14):
```

```
156.    df_testdata.iloc[:,i] = df_testdata.iloc[:,i].apply(calc_proportion_above_threshold)
```

157.

```
158.df_testdata
```

159.

```
160.loc_bss = encoder.transform(df_testdata['loc_bss_id'])
```

161.

```
162.loc_bss = pd.DataFrame(loc_bss)
```

```
163.df_testdata_1 = pd.concat([df_testdata, loc_bss], axis=1)
```

164.

```
165.df_testdata_1.columns.values[-1] = 'loc_bss'
```

166.

```
167.columns_to_drop = ['loc_bss_id', 'seq_time']
```

```
168.x_test = df_testdata_1.drop(columns=columns_to_drop)
```

```

169.x_test
170.
171.y_pred = rf.predict(X_test)
172.y_pred
173.
174.noise_mean = 0
175.noise_std = 1
176.noise = np.random.normal(loc=noise_mean, scale=noise_std, size=y_pred.shape)
177.y_pred_noisy = y_pred + noise
178.
179.y_pred_noisy = pd.DataFrame(y_pred_noisy)
180.y_pred_noisy.to_excel('ap2_output.xlsx', index=False)
181.
182.# 绘制原始预测值与添加噪声后的预测值对比
183.matplotlib.rcParams['font.sans-serif'] = ['SimHei']
184.plt.figure(figsize=(10, 6))
185.plt.scatter(y_pred, y_pred_noisy, color='blue', alpha=0.6)
186.plt.plot([y_pred.min(), y_pred.max()], [y_pred.min(), y_pred.max()], 'k--', lw=2)
187.plt.title('随机森林预测值与添加噪声后的预测值对比')
188.plt.show()
189.
190.df_data_3ap = pd.read_excel('3ap_train.xlsx')
191.df_data_3ap
192.
193.df_data_3ap['merged_column_1'] = df_data_3ap['ap_from_ap_0_max_ant_rssi'].combine_first(df_data_3ap['ap_f
rom_ap_1_max_ant_rssi'])
194.df_data_3ap['merged_column_2'] = df_data_3ap['ap_from_ap_2_max_ant_rssi'].combine_first(df_data_3ap['ap_f
rom_ap_1_max_ant_rssi'])
195.
196.df_data_3ap
197.
198.df_data_3ap.columns.values[-1] = 'ap_from_ap_y_max_ant_rssi'
199.df_data_3ap.columns.values[-2] = 'ap_from_ap_x_max_ant_rssi'
200.

```

```

201.encoder = TargetEncoder()

202.encoded_loc_bss_id = encoder.fit_transform(df_data_3ap[['loc_bss_id']], df_data_3ap[['seq_time']])

203.df_encoded = pd.DataFrame(encoded_loc_bss_id, columns=encoder.get_feature_names_out(['loc_bss_id']))

204.df_data_3ap = pd.concat([df_data_3ap, df_encoded], axis=1)

205.

206.df_data_3ap.columns.values[-1] = 'loc_bss'

207.

208.columns_to_drop = ['ap_from_ap_0_max_ant_rssi', 'ap_from_ap_1_max_ant_rssi', 'ap_from_ap_2_max_ant_rssi']

209.df_data_3ap = df_data_3ap.drop(columns=columns_to_drop)

210.

211.df_data_3ap['ap_from_ap_x_mean_ant_rssi'] = df_data_3ap['ap_from_ap_0_mean_ant_rssi'].combine_first(df_data_3ap['ap_from_ap_1_mean_ant_rssi'])

212.df_data_3ap['ap_from_ap_y_mean_ant_rssi'] = df_data_3ap['ap_from_ap_2_mean_ant_rssi'].combine_first(df_data_3ap['ap_from_ap_1_mean_ant_rssi'])

213.

214.df_data_3ap.to_excel('ap3_train_q2.xlsx', index=False)

215.

216.columns_to_drop = ['ap_from_ap_0_mean_ant_rssi', 'ap_from_ap_1_mean_ant_rssi', 'ap_from_ap_2_mean_ant_rssi']

217.df_data_3ap = df_data_3ap.drop(columns=columns_to_drop)

218.

219.df_data_3ap

220.

221.for i in chain(range(4, 16), range(24, 28)):

222.    df_data_3ap.iloc[:,i] = df_data_3ap.iloc[:,i].apply(convert_to_list)

223.

224.for i in chain(range(4, 16), range(24, 28)):

225.    df_data_3ap.iloc[:,i] = df_data_3ap.iloc[:,i].apply(calc_proportion_above_threshold)

226.

227.df_data_3ap.to_excel('Q3_3ap_train.xlsx', index=False)

228.

229.pd.set_option('display.max_columns', None)

230.df_data_3ap

```

231.

```
232.corr_matrix = df_data_3ap.corr()
```

```
233.plt.figure(figsize=(10, 8))
```

```
234.plt.rcParams['font.family'] = 'Arial'
```

```
235.sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', fmt='.2f', linewidths=0.5)
```

```
236.plt.xticks(rotation=45, ha='right')
```

```
237.plt.title('Correlation Matrix Heatmap')
```

```
238.plt.show()
```

239.

```
240.X = df_data_3ap.drop(columns=['seq_time'])
```

```
241.y = df_data_3ap['seq_time']
```

242.

```
243.# 划分训练集和测试集
```

```
244.X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

245.

```
246.# 初始化随机森林回归模型
```

```
247.rf = RandomForestRegressor(n_estimators=100, random_state=42)
```

248.

```
249.# 训练模型
```

```
250.rf.fit(X_train, y_train)
```

251.

```
252.# 进行预测
```

```
253.y_pred = rf.predict(X_test)
```

254.

```
255.# 评估模型
```

```
256.mse = mean_squared_error(y_test, y_pred)
```

```
257.r2 = r2_score(y_test, y_pred)
```

258.

```
259.mse, r2
```

260.

```
261.# 随机森林特征重要性条形图
```

```
262.feature_importances = rf.feature_importances_
```

```
263.feature_names = X_train.columns
```

264.

```

265.feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': feature_importances})
266.feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
267.feature_importance_df = feature_importance_df
268.
269.plt.figure(figsize=(10, 6))
270.sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
271.plt.savefig('rf_import.png')
272.plt.show()
273.
274.gbdt = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
275.
276.gbdt.fit(X_train, y_train)
277.
278.# 对测试集进行预测
279.y_pred = gbdt.predict(X_test)
280.
281.# 计算均方误差 (MSE)
282.mse = mean_squared_error(y_test, y_pred)
283.r2 = r2_score(y_test, y_pred)
284.
285.mse,r2
286.
287.importances = gbdt.feature_importances_
288.
289.feature_names = X_train.columns
290.feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
291.
292.feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
293.feature_importance_df = feature_importance_df
294.
295.plt.figure(figsize=(10, 6))
296.sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
297.plt.savefig('gbdt_import.png')
298.plt.show()

```

299.

```
300.df_testdata_3ap = pd.read_excel('3ap_test.xlsx')
```

```
301.df_testdata_3ap
```

302.

```
303.df_testdata_3ap['ap_from_ap_x_max_ant_rssi'] = df_testdata_3ap['ap_from_ap_0_max_ant_rssi'].combine_first  
(df_testdata_3ap['ap_from_ap_1_max_ant_rssi'])
```

```
304.df_testdata_3ap['ap_from_ap_y_max_ant_rssi'] = df_testdata_3ap['ap_from_ap_2_max_ant_rssi'].combine_first  
(df_testdata_3ap['ap_from_ap_1_max_ant_rssi'])
```

305.

```
306.df_testdata_3ap['ap_from_ap_x_mean_ant_rssi'] = df_testdata_3ap['ap_from_ap_0_mean_ant_rssi'].combine_fir  
st(df_testdata_3ap['ap_from_ap_1_mean_ant_rssi'])
```

```
307.df_testdata_3ap['ap_from_ap_y_mean_ant_rssi'] = df_testdata_3ap['ap_from_ap_2_mean_ant_rssi'].combine_fir  
st(df_testdata_3ap['ap_from_ap_1_mean_ant_rssi'])
```

308.

```
309.loc_bss = encoder.transform(df_testdata_3ap['loc_bss_id'])
```

310.

```
311.loc_bss = pd.DataFrame(loc_bss)
```

```
312.df_testdata_3ap = pd.concat([df_testdata_3ap, loc_bss], axis=1)
```

313.

```
314.df_testdata_3ap.columns.values[-1] = 'loc_bss'
```

315.

```
316.columns_to_drop = [
```

```
317.         'ap_from_ap_0_max_ant_rssi', 'ap_from_ap_1_max_ant_rssi', 'ap_from_ap_2_max_ant_rssi'  
,
```

```
318.         'ap_from_ap_0_mean_ant_rssi', 'ap_from_ap_1_mean_ant_rssi', 'ap_from_ap_2_mean_ant_rssi'  
,
```

```
319.df_testdata_3ap = df_testdata_3ap.drop(columns=columns_to_drop)
```

320.

```
321.for i in chain(range(4, 16), range(19, 23)):
```

```
322.     df_testdata_3ap.iloc[:,i] = df_testdata_3ap.iloc[:,i].apply(convert_to_list)
```

323.

```
324.for i in chain(range(4, 16), range(19, 23)):
```

```
325.     df_testdata_3ap.iloc[:,i] = df_testdata_3ap.iloc[:,i].apply(calc_proportion_above_threshold)
```

326.

```
327.df_testdata_3ap.to_excel('Q3_3ap_test.xlsx', index=False)
328.
329.train_columns = X_train.columns
330.df_testdata_3ap = df_testdata_3ap[train_columns]
331.
332.df_testdata_3ap
333.
334.y_pred = rf.predict(df_testdata_3ap)
335.
336.y_pred
337.
338.noise_mean = 0
339.noise_std = 1
340.noise = np.random.normal(loc=noise_mean, scale=noise_std, size=y_pred.shape)
341.y_pred_noisy = y_pred + noise
342.
343.y_pred_noisy = pd.DataFrame(y_pred_noisy)
344.y_pred_noisy.to_excel('3ap_output.xlsx', index=False, header=False)
345.
346.# 绘制原始预测值与添加噪声后的预测值对比
347.plt.figure(figsize=(10, 6))
348.plt.scatter(y_pred, y_pred_noisy, color='blue', alpha=0.6)
349.plt.plot([y_pred.min(), y_pred.max()], [y_pred.min(), y_pred.max()], 'k--', lw=2)
350.plt.show()
```

Question2: 拟合融入过采样方法的卷积神经网络对（MCS, NSS）进行预测。

```
1. import numpy as np
2. import pandas as pd
3. import ast
4. import matplotlib.pyplot as plt
5. import seaborn as sns
6. import matplotlib
7. from sklearn.metrics import mean_squared_error, r2_score
8. from sklearn.model_selection import train_test_split
9. from itertools import chain
10. from category_encoders import TargetEncoder
11. from sklearn.preprocessing import LabelEncoder
12. from tensorflow.keras.models import Sequential
13. from tensorflow.keras.layers import Dense, Conv1D, MaxPooling1D, Flatten
14. from tensorflow.keras.utils import to_categorical
15. from sklearn.model_selection import train_test_split
16. from sklearn.preprocessing import StandardScaler
17. from imblearn.over_sampling import RandomOverSampler, SMOTE
18. from collections import Counter
19. from tensorflow.keras.optimizers import Adam
20.
21. data_2ap = pd.read_excel('2ap_train.xlsx')
22. data_2ap
23.
24. def convert_to_list(element):
25.     try:
26.         return ast.literal_eval(element)
27.     except (ValueError, SyntaxError):
28.         return element
29.
30. def calc_proportion_above_threshold(rssi_list, threshold1=-62, threshold2=-82):
31.     if isinstance(rssi_list, list):
32.         total_count = len(rssi_list)
33.         above_threshold_count = sum(1 for rssi in rssi_list if (rssi > threshold2 and rssi < threshold
1))
```



```

34.         return above_threshold_count / total_count if total_count > 0 else 0
35.     return 0
36.
37. def calculate_sinr_max(row):
38.     # 获取两个列中的数值
39.     list_a = row['sta_from_ap_0_max_ant_rssi']
40.     list_b = row['sta_from_ap_1_max_ant_rssi']
41.
42.     # 截断较长的列表
43.     min_length = min(len(list_a), len(list_b))
44.     list_a = list_a[:min_length]
45.     list_b = list_b[:min_length]
46.
47.     sync_sinr = [] # 同步传输的 SINR 值
48.     async_sinr = [] # 异步传输的 SINR 值
49.
50.     for a, b in zip(list_a, list_b):
51.         if a > -62: # 同步传输
52.             sinr_value = a + 99
53.             sync_sinr.append(sinr_value)
54.         elif a < -82: # 异步传输
55.             sinr_value = a - b
56.             async_sinr.append(sinr_value)
57.         else:
58.             category = np.random.choice(['sync', 'async'], p=[0.3, 0.7])
59.             if category == 'sync':
60.                 sinr_value = a + 99
61.                 sync_sinr.append(sinr_value)
62.             else:
63.                 sinr_value = a - b
64.                 async_sinr.append(sinr_value)
65.
66.     # 计算同步传输的平均 SINR
67.     if sync_sinr:

```

```

68.         SINR1 = np.mean(sync_sinnr)
69.     else:
70.         SINR1 = 0
71.
72.     # 计算异步传输的平均 SINR
73.     if async_sinnr:
74.         SINR2 = np.mean(async_sinnr)
75.     else:
76.         SINR2 = 0
77.
78.     # 计算同步传输和异步传输的占比
79.     total_count = len(list_a)
80.     sync_count = len(sync_sinnr)
81.     async_count = len(async_sinnr)
82.
83.     c_percent = sync_count / total_count if total_count > 0 else 0
84.     d_percent = async_count / total_count if total_count > 0 else 0
85.
86.     # 计算最终 SINR
87.     sinnr_max = c_percent * SINR1 + d_percent * SINR2
88.
89.     return pd.Series([sinnr_max], index=['sinnr_max'])
90.
91. def calculate_sinnr_mean(row):
92.     # 获取两个列中的数值
93.     list_a = row['sta_from_ap_0_max_ant_rssi']
94.     list_b = row['sta_from_ap_1_max_ant_rssi']
95.
96.     # 截断较长的列表
97.     min_length = min(len(list_a), len(list_b))
98.     list_a = list_a[:min_length]
99.     list_b = list_b[:min_length]
100.
101.     sync_sinnr = [] # 同步传输的 SINR 值

```

```

102.     async_sinr = [] # 异步传输的 SINR 值
103.
104.     for a, b in zip(list_a, list_b):
105.         if a > -62: # 同步传输
106.             sinr_value = a + 99
107.             sync_sinr.append(sinr_value)
108.         elif a < -82: # 异步传输
109.             sinr_value = a - b
110.             async_sinr.append(sinr_value)
111.         else:
112.             category = np.random.choice(['sync', 'async'], p=[0.3, 0.7])
113.             if category == 'sync':
114.                 sinr_value = a + 99
115.                 sync_sinr.append(sinr_value)
116.             else:
117.                 sinr_value = a - b
118.                 async_sinr.append(sinr_value)
119.
120.     # 计算同步传输的平均 SINR
121.     if sync_sinr:
122.         SINR1 = np.mean(sync_sinr)
123.     else:
124.         SINR1 = 0
125.
126.     # 计算异步传输的平均 SINR
127.     if async_sinr:
128.         SINR2 = np.mean(async_sinr)
129.     else:
130.         SINR2 = 0
131.
132.     # 计算同步传输和异步传输的占比
133.     total_count = len(list_a)
134.     sync_count = len(sync_sinr)
135.     async_count = len(async_sinr)

```

```

136.
137.     c_percent = sync_count / total_count if total_count > 0 else 0
138.     d_percent = async_count / total_count if total_count > 0 else 0
139.
140.     # 计算最终 SINR
141.     sinr_mean = c_percent * SINR1 + d_percent * SINR2
142.
143.     return pd.Series([sinr_mean], index=['sinr_mean'])
144.
145. for i in range(4,14):
146.     data_2ap.iloc[:,i] = data_2ap.iloc[:,i].apply(convert_to_list)
147.
148. # 对 DataFrame 中的每一行应用 calculate_sinr 函数
149. data_2ap[['sinr_max']] = data_2ap.apply(calculate_sinr_max, axis=1)
150. data_2ap[['sinr_mean']] = data_2ap.apply(calculate_sinr_mean, axis=1)
151.
152. for i in range(4,14):
153.     data_2ap.iloc[:,i] = data_2ap.iloc[:,i].apply(calc_proportion_above_threshold)
154.
155. data_2ap['nss_mcs'] = data_2ap.apply(lambda row: f"({row['nss']},{row['mcs']})", axis=1)
156.
157. encoder = LabelEncoder()
158. data_2ap['nss_mcs_encoded'] = encoder.fit_transform(data_2ap['nss_mcs'])
159.
160. data_2ap
161.
162. encoder_target = TargetEncoder()
163.
164. encoded_loc_bss_id = encoder_target.fit_transform(data_2ap[['loc_bss_id']], data_2ap[['nss_mcs_encoded
' ]])
165. df_encoded = pd.DataFrame(encoded_loc_bss_id, columns=encoder_target.get_feature_names_out(['loc_bss_id
' ]))
166. data_2ap_1 = pd.concat([data_2ap, df_encoded], axis=1)
167.

```

```

168.data_2ap_1.columns.values[-1] = 'loc_bss'

169.data_2ap_1 = data_2ap_1.drop(data_2ap_1.columns[1], axis=1)

170.

171.data_2ap_1

172.

173.columns_to_drop = ['nss', 'mcs', 'nss_mcs', 'nss_mcs_encoded']

174.X = data_2ap_1.drop(columns=columns_to_drop)

175.

176.y = data_2ap_1['nss_mcs_encoded']

177.

178.Counter(y)

179.

180.n_samples = 391

181.n_features = 16

182.n_classes = 14

183.

184.y = to_categorical(y, num_classes=n_classes)

185.

186.scaler = StandardScaler()

187.X = scaler.fit_transform(X)

188.X = X.reshape((n_samples, n_features, 1))

189.

190.X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

191.

192.def bp_model():

193.     model = Sequential()

194.     model.add(Dense(100, input_dim=X.shape[1], activation='relu'))

195.     model.add(Dense(14, activation='softmax'))

196.     model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])

197.     return model

198.

199.model_bp = bp_model()

200.history = model_bp.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=100, batch_size=32)

201.

```

```

202.loss, accuracy = model_bp.evaluate(X_test, y_test)

203.accuracy

204.

205.# 获取训练过程中的损失和准确率

206.train_loss = history.history['loss']

207.val_loss = history.history['val_loss']

208.train_accuracy = history.history['accuracy']

209.val_accuracy = history.history['val_accuracy']

210.

211.# 创建图像

212.epochs = range(1, len(train_loss) + 1)

213.

214.# 绘制损失曲线

215.plt.figure(figsize=(12, 5))

216.

217.# Subplot 1: 绘制损失曲线

218.plt.subplot(1, 2, 1)

219.plt.plot(epochs, train_loss, label='Training Loss')

220.plt.plot(epochs, val_loss , label='Validation Loss')

221.plt.title('Training and Validation Loss')

222.plt.xlabel('Epochs')

223.plt.ylabel('Loss')

224.plt.legend()

225.

226.# Subplot 2: 绘制准确率曲线

227.plt.subplot(1, 2, 2)

228.plt.plot(epochs, train_accuracy, label='Training Accuracy')

229.plt.plot(epochs, val_accuracy, label='Validation Accuracy')

230.plt.title('Training and Validation Accuracy')

231.plt.xlabel('Epochs')

232.plt.ylabel('Accuracy')

233.plt.legend()

234.

235.# 显示图像

```

```

236.plt.tight_layout()

237.plt.savefig('ap2_bp.jpg')

238.plt.show()

239.

240.def create_cnn_model():

241.     model = Sequential()

242.     model.add(Conv1D(filters=64, kernel_size=2, activation='relu', input_shape=(n_features, 1)))

243.     model.add(MaxPooling1D(pool_size=2))

244.     model.add(Flatten())

245.     model.add(Dense(100, activation='relu'))

246.     model.add(Dense(n_classes, activation='softmax'))

247.     model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

248.     return model

249.

250.model_ori = create_cnn_model()

251.history = model_ori.fit(X_train, y_train, epochs=100, batch_size=32, validation_data=(X_test, y_test))

252.

253.# 评估模型

254.loss, accuracy = model_ori.evaluate(X_test, y_test)

255.accuracy

256.

257.# 获取训练过程中的损失和准确率

258.train_loss = history.history['loss']

259.val_loss = history.history['val_loss']

260.train_accuracy = history.history['accuracy']

261.val_accuracy = history.history['val_accuracy']

262.

263.# 创建图像

264.epochs = range(1, len(train_loss) + 1)

265.

266.# 绘制损失曲线

267.plt.figure(figsize=(12, 5))

268.

269.# Subplot 1: 绘制损失曲线

```

```

270.plt.subplot(1, 2, 1)

271.plt.plot(epochs, train_loss, label='Training Loss')

272.plt.plot(epochs, val_loss , label='Validation Loss')

273.plt.title('Training and Validation Loss')

274.plt.xlabel('Epochs')

275.plt.ylabel('Loss')

276.plt.legend()

277.

278.# Subplot 2: 绘制准确率曲线

279.plt.subplot(1, 2, 2)

280.plt.plot(epochs, train_accuracy, label='Training Accuracy')

281.plt.plot(epochs, val_accuracy, label='Validation Accuracy')

282.plt.title('Training and Validation Accuracy')

283.plt.xlabel('Epochs')

284.plt.ylabel('Accuracy')

285.plt.legend()

286.

287.# 显示图像

288.plt.tight_layout()

289.plt.savefig('ap2_cnn.jpg')

290.plt.show()

291.

292.# 欠采样

293.ros = RandomOverSampler(random_state=42)

294.x_ros, y_ros = ros.fit_resample(X, y)

295.

296.Counter(y_ros)

297.

298.x_ros

299.

300.n_samples = 3486

301.n_features = 16

302.n_classes = 14

303.

```



```

304.y_ros = to_categorical(y_ros, num_classes=n_classes)

305.

306.scaler = StandardScaler()

307.X_ros = scaler.fit_transform(X_ros)

308.X_ros = X_ros.reshape((n_samples, n_features, 1))

309.

310.X_ros_train, X_ros_test, y_ros_train, y_ros_test = train_test_split(X_ros, y_ros, test_size=0.2, random_s
tate=42)

311.

312.model_ros = create_cnn_model()

313.history_ros = model_ros.fit(X_ros_train, y_ros_train, epochs=200, batch_size=32, validation_data=(X_ros_t
est, y_ros_test))

314.

315.loss, accuracy = model_ros.evaluate(X_ros_test, y_ros_test)

316.accuracy

317.

318.# 获取训练过程中的损失和准确率

319.train_loss = history_ros.history['loss']

320.val_loss = history_ros.history['val_loss']

321.train_accuracy = history_ros.history['accuracy']

322.val_accuracy = history_ros.history['val_accuracy']

323.

324.# 创建图像

325.epochs = range(1, len(train_loss) + 1)

326.

327.# 绘制损失曲线

328.plt.figure(figsize=(12, 5))

329.

330.# Subplot 1: 绘制损失曲线

331.plt.subplot(1, 2, 1)

332.plt.plot(epochs, train_loss, label='Training Loss')

333.plt.plot(epochs, val_loss , label='Validation Loss')

334.plt.title('Training and Validation Loss')

335.plt.xlabel('Epochs')

```

```

336.plt.ylabel('Loss')
337.plt.legend()
338.
339.# Subplot 2: 绘制准确率曲线
340.plt.subplot(1, 2, 2)
341.plt.plot(epochs, train_accuracy, label='Training Accuracy')
342.plt.plot(epochs, val_accuracy, label='Validation Accuracy')
343.plt.title('Training and Validation Accuracy')
344.plt.xlabel('Epochs')
345.plt.ylabel('Accuracy')
346.plt.legend()
347.
348.# 显示图像
349.plt.tight_layout()
350.plt.savefig('ap2_cnn_ros.jpg')
351.plt.show()
352.
353.data_2ap_test = pd.read_excel('2ap_test_2.xlsx')
354.data_2ap_test
355.
356.for i in range(4,14):
357.    data_2ap_test.iloc[:,i] = data_2ap_test.iloc[:,i].apply(convert_to_list)
358.
359.# 对 DataFrame 中的每一行应用 calculate_snr 函数
360.data_2ap_test[['sinr_max']] = data_2ap_test.apply(calculate_snr_max, axis=1)
361.data_2ap_test[['sinr_mean']] = data_2ap_test.apply(calculate_snr_mean, axis=1)
362.
363.for i in range(4,14):
364.    data_2ap_test.iloc[:,i] = data_2ap_test.iloc[:,i].apply(calc_proportion_above_threshold)
365.
366.data_2ap_test
367.
368.loc_bss = encoder_target.transform(data_2ap_test['loc_bss_id'])
369.loc_bss = pd.DataFrame(loc_bss)

```

```

370.data_2ap_test_1 = pd.concat([data_2ap_test, loc_bss], axis=1)

371.data_2ap_test_1.columns.values[-1] = 'loc_bss'

372.columns_to_drop = ['loc_bss_id', 'nss', 'mcs']

373.x_test = data_2ap_test_1.drop(columns=columns_to_drop)

374.

375.x_test

376.

377.x_test_scale = scaler.transform(X_test)

378.

379.x_test_scale = X_test_scale.reshape((X_test_scale.shape[0], X_test_scale.shape[1], 1))

380.y_pred_prob = model_ros.predict(X_test_scale)

381.y_pred_labels = np.argmax(y_pred_prob, axis=1)

382.y_pred_labels

383.

384.y_pred_labels = encoder.inverse_transform(y_pred_labels)

385.y_pred_labels

386.

387.y_pred_labels = pd.DataFrame(y_pred_labels)

388.

389.y_pred_labels.columns = ['nss_mcs']

390.

391.data_3ap = pd.read_excel('ap3_train_q2.xlsx')

392.data_3ap

393.

394.def convert_to_list(element):

395.     try:

396.         return ast.literal_eval(element)

397.     except (ValueError, SyntaxError):

398.         return element

399.

400.def calc_proportion_above_threshold(rssi_list, threshold1=-62, threshold2=-82):

401.     if isinstance(rssi_list, list):

402.         total_count = len(rssi_list)

403.         above_threshold_count = sum(1 for rssi in rssi_list if (rssi > threshold2 and rssi < threshold
1))

```

```

404.         return above_threshold_count / total_count if total_count > 0 else 0
405.     return 0
406.
407. def calculate_sinr_max(row):
408.     # 获取两个列中的数值
409.     list_a = row['sta_from_ap_0_max_ant_rssi']
410.     list_b = row['sta_from_ap_1_max_ant_rssi']
411.     list_c = row['sta_from_ap_2_max_ant_rssi']
412.
413.     # 截断较长的列表
414.     min_length = min(len(list_a), len(list_b), len(list_c))
415.     list_a = list_a[:min_length]
416.     list_b = list_b[:min_length]
417.     list_c = list_c[:min_length]
418.
419.     sync_sinr = [] # 同步传输的 SINR 值
420.     async_sinr = [] # 异步传输的 SINR 值
421.
422.     for a, b, c in zip(list_a, list_b, list_c):
423.         if a > -62: # 同步传输
424.             sinr_value = a + 99
425.             sync_sinr.append(sinr_value)
426.         elif a < -82: # 异步传输
427.             sinr_value = a - b - c
428.             async_sinr.append(sinr_value)
429.         else:
430.             category = np.random.choice(['sync', 'async'], p=[0.3, 0.7])
431.             if category == 'sync':
432.                 sinr_value = a + 99
433.                 sync_sinr.append(sinr_value)
434.             else:
435.                 sinr_value = a - b - c
436.                 async_sinr.append(sinr_value)
437.

```

```

438.     # 计算同步传输的平均 SINR
439.     if sync_snr:
440.         SINR1 = np.mean(sync_snr)
441.     else:
442.         SINR1 = 0
443.
444.     # 计算异步传输的平均 SINR
445.     if async_snr:
446.         SINR2 = np.mean(async_snr)
447.     else:
448.         SINR2 = 0
449.
450.     # 计算同步传输和异步传输的占比
451.     total_count = len(list_a)
452.     sync_count = len(sync_snr)
453.     async_count = len(async_snr)
454.
455.     c_percent = sync_count / total_count if total_count > 0 else 0
456.     d_percent = async_count / total_count if total_count > 0 else 0
457.
458.     # 计算最终 SINR
459.     sinr_max = c_percent * SINR1 + d_percent * SINR2
460.
461.     return pd.Series([sinr_max], index=['sinr_max'])
462.
463. def calculate_sinr_mean(row):
464.     # 获取两个列中的数值
465.     list_a = row['sta_from_ap_0_max_ant_rssi']
466.     list_b = row['sta_from_ap_1_max_ant_rssi']
467.     list_c = row['sta_from_ap_2_max_ant_rssi']
468.
469.     # 截断较长的列表
470.     min_length = min(len(list_a), len(list_b), len(list_c))
471.     list_a = list_a[:min_length]

```

```

472.     list_b = list_b[:min_length]
473.     list_c = list_c[:min_length]
474.
475.     sync_sinr = [] # 同步传输的 SINR 值
476.     async_sinr = [] # 异步传输的 SINR 值
477.
478.     for a, b, c in zip(list_a, list_b, list_c):
479.         if a > -62: # 同步传输
480.             sinr_value = a + 99
481.             sync_sinr.append(sinr_value)
482.         elif a < -82: # 异步传输
483.             sinr_value = a - b - c
484.             async_sinr.append(sinr_value)
485.         else:
486.             category = np.random.choice(['sync', 'async'], p=[0.3, 0.7])
487.             if category == 'sync':
488.                 sinr_value = a + 99
489.                 sync_sinr.append(sinr_value)
490.             else:
491.                 sinr_value = a - b - c
492.                 async_sinr.append(sinr_value)
493.
494.     # 计算同步传输的平均 SINR
495.     if sync_sinr:
496.         SINR1 = np.mean(sync_sinr)
497.     else:
498.         SINR1 = 0
499.
500.     # 计算异步传输的平均 SINR
501.     if async_sinr:
502.         SINR2 = np.mean(async_sinr)
503.     else:
504.         SINR2 = 0
505.

```

```

506.     # 计算同步传输和异步传输的占比
507.     total_count = len(list_a)
508.     sync_count = len(sync_sinr)
509.     async_count = len(async_sinr)
510.
511.     c_percent = sync_count / total_count if total_count > 0 else 0
512.     d_percent = async_count / total_count if total_count > 0 else 0
513.
514.     # 计算最终 SINR
515.     sinr_mean = c_percent * SINR1 + d_percent * SINR2
516.
517.     return pd.Series([sinr_mean], index=['sinr_mean'])
518.
519. for i in range(4,20):
520.     data_3ap.iloc[:,i] = data_3ap.iloc[:,i].apply(convert_to_list)
521.
522. # 对 DataFrame 中的每一行应用 calculate_sinr 函数
523. data_3ap[['sinr_max']] = data_3ap.apply(calculate_sinr_max, axis=1)
524. data_3ap[['sinr_mean']] = data_3ap.apply(calculate_sinr_mean, axis=1)
525.
526. for i in range(4,20):
527.     data_3ap.iloc[:,i] = data_3ap.iloc[:,i].apply(calc_proportion_above_threshold)
528.
529. data_3ap['nss_mcs'] = data_3ap.apply(lambda row: f"({row['nss']},{row['mcs']})", axis=1)
530.
531. encoder = LabelEncoder()
532. data_3ap['nss_mcs_encoded'] = encoder.fit_transform(data_3ap['nss_mcs'])
533.
534. encoder_target = TargetEncoder()
535. encoded_loc_bss_id = encoder_target.fit_transform(data_3ap[['loc_bss_id']], data_3ap[['nss_mcs_encoded
' ]])
536. df_encoded = pd.DataFrame(encoded_loc_bss_id, columns=encoder_target.get_feature_names_out(['loc_bss_id
' ]))
537. data_3ap_1 = pd.concat([data_3ap, df_encoded], axis=1)

```

```

538.
539.data_3ap_1.columns.values[-1] = 'loc_bss'
540.data_3ap_1 = data_3ap_1.drop(data_3ap_1.columns[1], axis=1)
541.
542.columns_to_drop = ['nss', 'mcs', 'nss_mcs', 'nss_mcs_encoded']
543.x = data_3ap_1.drop(columns=columns_to_drop)
544.
545.x
546.
547.y = data_3ap_1['nss_mcs_encoded']
548.
549.y
550.
551.ros = RandomOverSampler(random_state=42)
552.X_ros, y_ros = ros.fit_resample(X, y)
553.
554.Counter(y_ros)
555.
556.X_ros
557.
558.n_samples = 3472
559.n_features = 22
560.n_classes = 14
561.
562.y_ros = to_categorical(y_ros, num_classes=n_classes)
563.
564.scaler = StandardScaler()
565.X_ros = scaler.fit_transform(X_ros)
566.X_ros = X_ros.reshape((n_samples, n_features, 1))
567.
568.X_ros_train, X_ros_test, y_ros_train, y_ros_test = train_test_split(X_ros, y_ros, test_size=0.2, random_s
tate=42)
569.def create_cnn_model():
570.    model = Sequential()

```



```

571.     model.add(Conv1D(filters=64, kernel_size=2, activation='relu', input_shape=(n_features, 1)))
572.     model.add(MaxPooling1D(pool_size=2))
573.     model.add(Flatten())
574.     model.add(Dense(100, activation='relu'))
575.     model.add(Dense(n_classes, activation='softmax'))
576.     model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
577.     return model
578.
579.model_ros = create_cnn_model()
580.history_ros = model_ros.fit(X_ros_train, y_ros_train, epochs=300, batch_size=32, validation_data=(X_ros_t
est, y_ros_test))
581.
582.loss, accuracy = model_ros.evaluate(X_ros_test, y_ros_test)
583.accuracy
584.
585.# 获取训练过程中的损失和准确率
586.train_loss = history_ros.history['loss']
587.val_loss = history_ros.history['val_loss']
588.train_accuracy = history_ros.history['accuracy']
589.val_accuracy = history_ros.history['val_accuracy']
590.
591.# 创建图像
592.epochs = range(1, len(train_loss) + 1)
593.
594.# 绘制损失曲线
595.plt.figure(figsize=(12, 5))
596.
597.# Subplot 1: 绘制损失曲线
598.plt.subplot(1, 2, 1)
599.plt.plot(epochs, train_loss, label='Training Loss')
600.plt.plot(epochs, val_loss , label='Validation Loss')
601.plt.title('Training and Validation Loss')
602.plt.xlabel('Epochs')
603.plt.ylabel('Loss')

```

```

604.plt.legend()

605.

606.# Subplot 2: 绘制准确率曲线

607.plt.subplot(1, 2, 2)

608.plt.plot(epochs, train_accuracy, label='Training Accuracy')

609.plt.plot(epochs, val_accuracy, label='Validation Accuracy')

610.plt.title('Training and Validation Accuracy')

611.plt.xlabel('Epochs')

612.plt.ylabel('Accuracy')

613.plt.legend()

614.

615.# 显示图像

616.plt.tight_layout()

617.plt.savefig('ap3_cnn_ros.jpg')

618.plt.show()

619.

620.data_3ap_test = pd.read_excel('3ap_test_2.xlsx')

621.data_3ap_test

622.

623.data_3ap_test['ap_from_ap_x_max_ant_rssi'] = data_3ap_test['ap_from_ap_0_max_ant_rssi'].combine_first(data_3ap_test['ap_from_ap_1_max_ant_rssi'])

624.data_3ap_test['ap_from_ap_y_max_ant_rssi'] = data_3ap_test['ap_from_ap_2_max_ant_rssi'].combine_first(data_3ap_test['ap_from_ap_1_max_ant_rssi'])

625.

626.data_3ap_test['ap_from_ap_x_mean_ant_rssi'] = data_3ap_test['ap_from_ap_0_mean_ant_rssi'].combine_first(data_3ap_test['ap_from_ap_1_mean_ant_rssi'])

627.data_3ap_test['ap_from_ap_y_mean_ant_rssi'] = data_3ap_test['ap_from_ap_2_mean_ant_rssi'].combine_first(data_3ap_test['ap_from_ap_1_mean_ant_rssi'])

628.

629.columns_to_drop = ['ap_from_ap_0_max_ant_rssi', 'ap_from_ap_1_max_ant_rssi', 'ap_from_ap_2_max_ant_rssi',
'ap_from_ap_0_mean_ant_rssi', 'ap_from_ap_1_mean_ant_rssi', 'ap_from_ap_2_mean_ant_rssi',
'nss', 'mcs']

```

```

632.data_3ap_test = data_3ap_test.drop(columns=columns_to_drop)
633.
634.for i in range(4,20):
635.    data_3ap_test.iloc[:,i] = data_3ap_test.iloc[:,i].apply(convert_to_list)
636.
637.# 对 DataFrame 中的每一行应用 calculate_sinr 函数
638.data_3ap_test[['sinr_max']] = data_3ap_test.apply(calculate_sinr_max, axis=1)
639.data_3ap_test[['sinr_mean']] = data_3ap_test.apply(calculate_sinr_mean, axis=1)
640.
641.for i in range(4,20):
642.    data_3ap_test.iloc[:,i] = data_3ap_test.iloc[:,i].apply(calc_proportion_above_threshold)
643.
644.loc_bss = encoder_target.transform(data_3ap_test['loc_bss_id'])
645.loc_bss = pd.DataFrame(loc_bss)
646.data_3ap_test_1 = pd.concat([data_3ap_test, loc_bss], axis=1)
647.data_3ap_test_1.columns.values[-1] = 'loc_bss'
648.columns_to_drop = ['loc_bss_id']
649.X_test = data_3ap_test_1.drop(columns=columns_to_drop)
650.
651.X_test_scale = scaler.transform(X_test)
652.
653.X_test_scale = X_test_scale.reshape((X_test_scale.shape[0], X_test_scale.shape[1], 1))
654.y_pred_prob = model_ros.predict(X_test_scale)
655.y_pred_labels = np.argmax(y_pred_prob, axis=1)
656.y_pred_labels
657.
658.y_pred_labels = encoder.inverse_transform(y_pred_labels)
659.y_pred_labels
660.
661.y_pred_labels = pd.DataFrame(y_pred_labels)
662.
663.y_pred_labels.columns = ['nss_mcs']
664.
665.y_pred_labels.to_excel("Q2_3ap_output.xlsx")

```


Question3: 拟合双层交叉预测模型对 WLAN 吞吐量进行预测。

```
1. import numpy as np
2. import pandas as pd
3. import ast
4. import matplotlib.pyplot as plt
5. import seaborn as sns
6. import matplotlib
7. from sklearn.ensemble import RandomForestRegressor
8. from sklearn.model_selection import train_test_split
9. from itertools import chain
10. from category_encoders import TargetEncoder
11. from sklearn.model_selection import train_test_split
12. from sklearn.preprocessing import StandardScaler
13. from tensorflow.keras.optimizers import Adam
14. from sklearn.metrics import mean_squared_error, r2_score
15. from keras.models import Model
16. from keras.layers import Input, Dense, Add, Activation, BatchNormalization
17. from keras.optimizers import Adam
18. from keras import backend as K
19. import warnings
20.
21. warnings.filterwarnings("ignore")
22.
23. Q3_2ap_data = pd.read_excel('Q3_2ap_train.xlsx')
24. pd.set_option('display.max_columns', None)
25. Q3_2ap_data
26.
27. Q3_2ap_testdata = pd.read_excel('2ap_test.xlsx')
28.
29. def convert_to_list(element):
30.     try:
31.         return ast.literal_eval(element)
32.     except (ValueError, SyntaxError):
33.         return element
```

```

34.
35. def calc_proportion_above_threshold(rssi_list, threshold1=-62, threshold2=-82):
36.     if isinstance(rssi_list, list):
37.         total_count = len(rssi_list)
38.         above_threshold_count = sum(1 for rssi in rssi_list if (rssi > threshold2 and rssi < threshold
1))
39.         return above_threshold_count / total_count if total_count > 0 else 0
40.     return 0
41.
42. for i in range(4,14):
43.     Q3_2ap_testdata.iloc[:,i] = Q3_2ap_testdata.iloc[:,i].apply(convert_to_list)
44.
45. for i in range(4,14):
46.     Q3_2ap_testdata.iloc[:,i] = Q3_2ap_testdata.iloc[:,i].apply(calc_proportion_above_threshold)
47.
48. Q3_2ap_testdata
49.
50. result = pd.DataFrame()
51.
52. def rf_model(feature):
53.     encoder = TargetEncoder()
54.     encoded_loc_bss_id = encoder.fit_transform(Q3_2ap_data[['loc_bss_id']], Q3_2ap_data[['feature']])
55.     df_encoded = pd.DataFrame(encoded_loc_bss_id, columns=encoder.get_feature_names_out(['loc_bss_id
']))
56.     Q3_2ap_data_1 = pd.concat([Q3_2ap_data, df_encoded], axis=1)
57.
58.     Q3_2ap_data_1.columns.values[-1] = 'loc_bss'
59.     Q3_2ap_data_1 = Q3_2ap_data_1.drop(Q3_2ap_data_1.columns[1], axis=1)
60.
61.     columns_to_drop = ['per', 'num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time', 'throughput', 'PHYRate
']
62.     x = Q3_2ap_data_1.drop(columns=columns_to_drop)
63.     y = Q3_2ap_data_1[feature]
64.

```

```

65.     X_y = pd.concat([X, y], axis=1)
66.     corr_matrix = X_y.corr()
67.     plt.figure(figsize=(10, 8))
68.     sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
69.     plt.xticks(rotation=45, ha='right')
70.     plt.show()
71.
72.     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
73.     rf = RandomForestRegressor(n_estimators=100, random_state=42)
74.     rf.fit(X_train, y_train)
75.     y_pred = rf.predict(X_test)
76.
77.     mse = mean_squared_error(y_test, y_pred)
78.     r2 = r2_score(y_test, y_pred)
79.
80.     loc_bss = encoder.transform(Q3_2ap_testdata['loc_bss_id'])
81.     loc_bss = pd.DataFrame(loc_bss)
82.     Q3_2ap_testdata_1 = pd.concat([Q3_2ap_testdata, loc_bss], axis=1)
83.     Q3_2ap_testdata_1.columns.values[-1] = 'loc_bss'
84.     columns_to_drop = ['loc_bss_id']
85.     X_test = Q3_2ap_testdata_1.drop(columns=columns_to_drop)
86.
87.     y_pred = rf.predict(X_test)
88.     y_pred = y_pred.tolist()
89.     result[feature] = y_pred
90.
91.     return mse, r2
92.
93. for i in ['num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time']:
94.     rf_model(i)
95.
96. result
97.
98. for i in ['num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time']:

```

```

99.     mse, r2 = rf_model(i)
100.     print(f"Feature: {i}, MSE: {mse}, R²: {r2}")
101.
102. def r2_score(y_true, y_pred):
103.     ss_res = K.sum(K.square(y_true - y_pred))
104.     ss_tot = K.sum(K.square(y_true - K.mean(y_true)))
105.     return 1 - ss_res / (ss_tot + K.epsilon())
106.
107. def residual_block(x, units):
108.     shortcut = x
109.     x = Dense(units)(x)
110.     x = BatchNormalization()(x)
111.     x = Activation('relu')(x)
112.     x = Dense(units)(x)
113.     x = BatchNormalization()(x)
114.
115.     # 添加残差连接
116.     x = Add()([x, shortcut])
117.     x = Activation('relu')(x)
118.     return x
119.
120. def create_resnet_model(input_shape, num_blocks, units):
121.     inputs = Input(shape=input_shape)
122.     x = Dense(units)(inputs)
123.     x = BatchNormalization()(x)
124.     x = Activation('relu')(x)
125.
126.     for _ in range(num_blocks):
127.         x = residual_block(x, units)
128.
129.     x = Dense(1)(x) # 输出层
130.     model = Model(inputs=inputs, outputs=x)
131.
132.     model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error', metrics=['r2_score'])

```



```

133.     return model
134.
135.x = Q3_2ap_data[['per', 'num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time', 'PHYRate']]
136.
137.y = Q3_2ap_data[['throughput']]
138.
139.scaler = StandardScaler()
140.x = scaler.fit_transform(X)
141.
142.X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
143.
144.model = create_resnet_model(input_shape=(6,), num_blocks=5, units=64)
145.history = model.fit(X_train, y_train, epochs=100, batch_size=32, validation_data=(X_test, y_test))
146.
147.def plot_history(history):
148.     plt.figure(figsize=(14, 5))
149.
150.     # 绘制损失图
151.     plt.subplot(1, 2, 1)
152.     plt.plot(history.history['loss'], label='Training Loss')
153.     plt.plot(history.history['val_loss'], label='Validation Loss')
154.     plt.title('Training and Validation Loss')
155.     plt.xlabel('Epochs')
156.     plt.ylabel('Loss')
157.     plt.legend()
158.
159.     # 绘制 R2 图
160.     plt.subplot(1, 2, 2)
161.     plt.plot(history.history['r2_score'], label='Training R²')
162.     plt.plot(history.history['val_r2_score'], label='Validation R²')
163.     plt.title('Training and Validation R²')
164.     plt.xlabel('Epochs')
165.     plt.ylabel('R²')
166.     plt.legend()

```

```

167.
168.     # 显示图像
169.     plt.tight_layout()
170.     plt.show()
171.
172.plot_history(history)
173.
174.result1 = pd.read_excel('2ap_test_.xlsx')
175.result1
176.
177.# 定义数据字典
178.lookup_table = {
179.    1: [8.6, 17.2, 25.8, 34.4, 51.6, 68.8, 77.4, 86.0, 103.2, 114.7, 129.0, 143.4],
180.    2: [17.2, 34.4, 51.6, 68.8, 103.2, 137.6, 154.9, 172.1, 206.5, 229.4, 258.1, 286.8]
181.}
182.
183.# 定义查找函数
184.def lookup_value(row):
185.    return lookup_table[row['nss']][row['mcs']]
186.
187.result1['PHYRate'] = result1[['nss', 'mcs']].apply(lookup_value, axis=1)
188.
189.columns_to_drop = ['nss', 'mcs']
190.result1 = result1.drop(columns=columns_to_drop)
191.
192.result = pd.concat([result1, result], axis=1)
193.result
194.
195.second_col = result.pop('PHYRate')
196.result['PHYRate'] = second_col
197.result
198.
199.x_test_scale = scaler.transform(result)
200.

```

```

201.pred_throughput = model.predict(X_test_scale)
202.pred_throughput
203.
204.pred_throughput = pd.DataFrame(pred_throughput)
205.pred_throughput.to_excel('Q3_ap2_output.xlsx',index = False)
206.
207.Q3_3ap_data = pd.read_excel('Q3_3ap_train.xlsx')
208.pd.set_option('display.max_columns', None)
209.Q3_3ap_data
210.
211.Q3_3ap_testdata = pd.read_excel('Q3_3ap_test.xlsx')
212.Q3_3ap_testdata
213.
214.result = pd.DataFrame()
215.
216.def rf_model(feature):
217.    encoder = TargetEncoder()
218.    encoded_loc_bss_id = encoder.fit_transform(Q3_3ap_data[['loc_bss_id']], Q3_3ap_data[[feature]])
219.    df_encoded = pd.DataFrame(encoded_loc_bss_id, columns=encoder.get_feature_names_out(['loc_bss_id
' ]))
220.    Q3_3ap_data_1 = pd.concat([Q3_3ap_data, df_encoded], axis=1)
221.
222.    Q3_3ap_data_1.columns.values[-1] = 'loc_bss'
223.    Q3_3ap_data_1 = Q3_3ap_data_1.drop(Q3_3ap_data_1.columns[1], axis=1)
224.
225.    columns_to_drop = ['per', 'num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time','throughput','nss','m
cs']
226.    X = Q3_3ap_data_1.drop(columns=columns_to_drop)
227.    y = Q3_3ap_data_1[feature]
228.
229.    X_y = pd.concat([X, y], axis=1)
230.    corr_matrix = X_y.corr()
231.    plt.figure(figsize=(10, 8))
232.    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)

```

```

233. plt.xticks(rotation=45, ha='right')
234. plt.show()
235.
236. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
237. rf = RandomForestRegressor(n_estimators=100, random_state=42)
238. rf.fit(X_train, y_train)
239. y_pred = rf.predict(X_test)
240.
241. mse = mean_squared_error(y_test, y_pred)
242. r2 = r2_score(y_test, y_pred)
243.
244. loc_bss = encoder.transform(Q3_3ap_testdata['loc_bss_id'])
245. loc_bss = pd.DataFrame(loc_bss)
246. Q3_3ap_testdata_1 = pd.concat([Q3_3ap_testdata, loc_bss], axis=1)
247. Q3_3ap_testdata_1.columns.values[-1] = 'loc_bss'
248. columns_to_drop = ['loc_bss_id', 'nss', 'mcs', 'per']
249. X_test = Q3_3ap_testdata_1.drop(columns=columns_to_drop)
250.
251. y_pred = rf.predict(X_test)
252. y_pred = y_pred.tolist()
253. result[feature] = y_pred
254.
255. return mse, r2
256.
257. for i in ['num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time']:
258.     rf_model(i)
259.
260. result
261.
262. for i in ['num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time']:
263.     mse, r2 = rf_model(i)
264.     print(f"Feature: {i}, MSE: {mse}, R²: {r2}")
265.
266. def r2_score(y_true, y_pred):

```

```

267.     ss_res = K.sum(K.square(y_true - y_pred))
268.     ss_tot = K.sum(K.square(y_true - K.mean(y_true)))
269.     return 1 - ss_res / (ss_tot + K.epsilon())
270.
271. def residual_block(x, units):
272.     shortcut = x
273.     x = Dense(units)(x)
274.     x = BatchNormalization()(x)
275.     x = Activation('relu')(x)
276.     x = Dense(units)(x)
277.     x = BatchNormalization()(x)
278.
279.     # 添加残差连接
280.     x = Add()([x, shortcut])
281.     x = Activation('relu')(x)
282.     return x
283.
284. def create_resnet_model(input_shape, num_blocks, units):
285.     inputs = Input(shape=input_shape)
286.     x = Dense(units)(inputs)
287.     x = BatchNormalization()(x)
288.     x = Activation('relu')(x)
289.
290.     for _ in range(num_blocks):
291.         x = residual_block(x, units)
292.
293.     x = Dense(1)(x) # 输出层
294.     model = Model(inputs=inputs, outputs=x)
295.
296.     model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error', metrics=['r2_score'])
297.     return model
298.
299. # 定义数据字典
300. lookup_table = {

```

```

301.     1: [8.6, 17.2, 25.8, 34.4, 51.6, 68.8, 77.4, 86.0, 103.2, 114.7, 129.0, 143.4],
302.     2: [17.2, 34.4, 51.6, 68.8, 103.2, 137.6, 154.9, 172.1, 206.5, 229.4, 258.1, 286.8]
303.}
304.
305.# 定义查找函数
306.def lookup_value(row):
307.    return lookup_table[row['nss']][row['mcs']]
308.
309.Q3_3ap_data['PHYRate'] = Q3_3ap_data[['nss', 'mcs']].apply(lookup_value, axis=1)
310.
311.x = Q3_3ap_data[['per', 'num_ampdu', 'ppdu_dur', 'other_air_time', 'seq_time', 'PHYRate']]
312.
313.x
314.
315.y = Q3_3ap_data[['throughput']]
316.
317.y
318.
319.scaler = StandardScaler()
320.x = scaler.fit_transform(X)
321.
322.X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
323.
324.model = create_resnet_model(input_shape=(6,), num_blocks=5, units=64)
325.history = model.fit(X_train, y_train, epochs=100, batch_size=32, validation_data=(X_test, y_test))
326.
327.def plot_history(history):
328.    plt.figure(figsize=(14, 5))
329.
330.    # 绘制损失图
331.    plt.subplot(1, 2, 1)
332.    plt.plot(history.history['loss'], label='Training Loss')
333.    plt.plot(history.history['val_loss'], label='Validation Loss')
334.    plt.title('Training and Validation Loss')

```

```

335. plt.xlabel('Epochs')
336. plt.ylabel('Loss')
337. plt.legend()
338.
339. # 绘制 R2 图
340. plt.subplot(1, 2, 2)
341. plt.plot(history.history['r2_score'], label='Training R2')
342. plt.plot(history.history['val_r2_score'], label='Validation R2')
343. plt.title('Training and Validation R2')
344. plt.xlabel('Epochs')
345. plt.ylabel('R2')
346. plt.legend()
347.
348. # 显示图像
349. plt.tight_layout()
350. plt.show()
351.
352.plot_history(history)
353.
354.result1 = Q3_3ap_testdata[['nss', 'mcs', 'per']]
355.
356.result = pd.concat([result1, result], axis=1)
357.result
358.
359.result.drop(index=20, inplace=True)
360.result = result.reset_index(drop=True)
361.result['PHYRate'] = result[['nss', 'mcs']].apply(lookup_value, axis=1)
362.
363.columns_to_drop = ['nss', 'mcs']
364.result = result.drop(columns=columns_to_drop)
365.result
366.
367.x_test_scale = scaler.transform(result)
368.

```

```
369.pred_throughput = model.predict(X_test_scale)
370.pred_throughput
371.
372.pred_throughput = pd.DataFrame(pred_throughput)
373.pred_throughput.to_excel('Q3_ap3_output.xlsx',index = False)
```